

A diferencia de los niveles 0 y 1, que funcionan con tiras de sectores, RAID nivel 2 funciona con base en palabras, posiblemente incluso con base en bytes. Imagine dividir cada byte del disco virtual único en un par de "nibbles" de cuatro bits, y luego agregar un código de Hamming a cada uno para formar una palabra de siete bits, de la cual los bits 1, 2 y 4 son bits de paridad. Imagine además que las siete unidades de la figura 2-23(c) se sincronizan en términos de posición del brazo y posición rotacional. Entonces sería posible escribir la palabra de siete bits con código de Hamming distribuida en las siete unidades, un bit por unidad.

La computadora CM-2 de Thinking Machines usaba este esquema, tomando palabras de datos de 32 bits y añadiendo seis bits de paridad para formar una palabra de Hamming de 38 bits, más un bit extra para la paridad de palabras, distribuyendo cada palabra entre 39 unidades de disco. El rendimiento total era inmenso, porque en un tiempo de sector se podían escribir 32 sectores de datos. Además, perder una unidad no causaba problemas, porque equivalía a perder un bit de cada palabra de 39 bits leída, algo que el código Hamming podía manejar sobre la marcha.

En el lado negativo, este esquema requiere que todas las unidades estén sincronizadas rotacionalmente, y sólo tiene sentido si el número de unidades es considerable (aun con 32 unidades de datos y 6 unidades de paridad, el gasto extra es de 19%). También se exige mucho al controlador, ya que debe efectuar una suma de verificación de Hamming cada tiempo de bit.

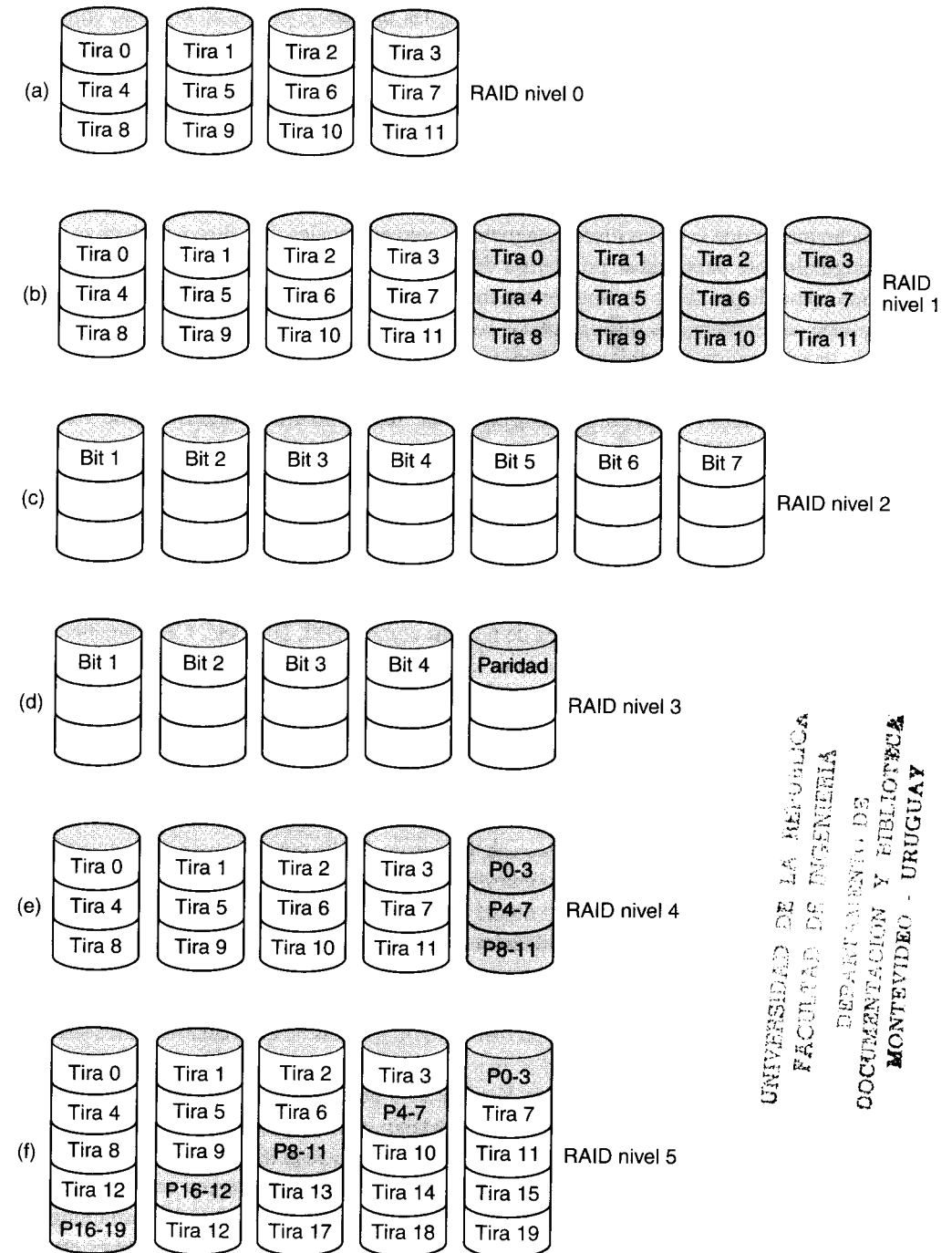
RAID nivel 3 es una versión simplificada de RAID nivel 2, y se ilustra en la figura 2-23(d). Aquí se calcula un solo bit de paridad para cada palabra de datos y se escribe en una unidad de disco de paridad. Al igual que en el RAID nivel 2, las unidades deben estar sincronizadas con exactitud, puesto que las palabras de datos individuales se distribuyen entre varias unidades.

A primera vista, podría parecer que un solo bit de paridad permite detectar errores, pero no corregirlos. En el caso de errores aleatorios no detectados esta observación es correcta, pero en el caso de una unidad que se descompone ofrece plena corrección de errores de un bit, ya que se sabe la posición del bit erróneo. Si una unidad falla, el controlador simplemente supone que todos sus bits son 0. Si una palabra tiene error de paridad, el bit de la unidad inactiva tiene que haber sido un 1, así que se corrige. Aunque ambos niveles de RAID, 2 y 3, ofrecen tasas de datos muy altas, el número de solicitudes de E/S distintas que pueden manejar por segundo no es mayor que con una sola unidad de disco.

Los niveles RAID 4 y 5 trabajan otra vez con tiras, no palabras individuales con paridad, y no requieren unidades sincronizadas. RAID nivel 4 [vea la figura 2-23(e)] se parece a RAID nivel 0, sólo que se escribe una paridad tira por tira en una unidad extra. Por ejemplo, si cada tira tiene  $k$  bytes de longitud, se calcula el OR EXCLUSIVO de todas las tiras para obtener una tira de paridad de  $k$  bytes. Si una unidad falla, los bytes perdidos se pueden recalcular a partir de la unidad de paridad.

Este diseño protege contra la pérdida de una unidad de disco pero tiene un desempeño pobre en el caso de actualizaciones pequeñas. Si un sector se modifica, es necesario leer todas las unidades para recalculer la paridad, que luego debe reescribirse. Como alternativa, se pueden leer los datos del usuario y de paridad antiguos y recalculer la nueva paridad a partir de ellos. Incluso con esta optimización, una actualización pequeña requiere dos lecturas y dos escrituras.

Lo pesado de la carga sobre la unidad de disco de paridad podría hacer que ésta se convirtiera en un cuello de botella. Esto se resolvería en el RAID nivel 5 distribuyendo los



**Figura 2-23.** RAID niveles 0 a 5. Las unidades de respaldo y paridad se muestran sombreadas.

bits de paridad de manera uniforme entre todas las unidades, por turno circular, como se muestra en la figura 2-23(f). Sin embargo, si una unidad fallara, la reconstrucción de su contenido sería un proceso complejo.

### 2.3.7 CD-ROM

En años recientes han aparecido discos ópticos (no magnéticos) cuya densidad de grabación es mucho mayor que la de los discos magnéticos convencionales. Los discos ópticos se crearon originalmente para grabar programas de televisión, pero se les puede dar un uso más estético como dispositivos de almacenamiento para computadora. A causa de su capacidad, que puede ser enorme, los discos ópticos han sido tema de muchas investigaciones y han sufrido una evolución increíblemente rápida.

Los discos ópticos de primera generación fueron inventados por el conglomerado de electrónica holandés Philips para grabar películas. Tenían 30 cm de diámetro y se vendían con el nombre LaserVision, pero no se popularizaron fuera de Japón.

En 1980 Philips, junto con Sony, desarrolló el CD (disco compacto, *Compact Disc*), que rápidamente sustituyó al disco de vinilo de 33 1/3 rpm para música grabada. Los detalles técnicos precisos del CD se publicaron en una Norma Internacional oficial (IS 10149) conocido popularmente como el **Libro Rojo**, por el color de su portada. (Las Normas Internacionales son emitidas por la Organización Internacional de Estandarización, que es la contraparte internacional de los grupos nacionales de normas como ANSI, DIN, etc. Cada una tiene un número IS.) Lo que se buscaba al publicar las especificaciones del disco y la unidad como Norma Internacional fue permitir que un CD de diferentes editores de música y reproductores de diferentes compañías electrónicas funcionaran juntos. Todos los CD tienen 120 mm de diámetro y 1.2 mm de espesor, con un orificio central de 15 mm. El CD de audio fue el primer medio de almacenamiento digital que tuvo éxito en el mercado masivo. Se supone que los CD duran 100 años. No olvide preguntar en 2080 qué tan bueno salió el primer lote.

Un CD se prepara utilizando un láser de infrarrojo de alta potencia para quemar orificios de 0.8 micras de diámetro en un disco maestro de vidrio recubierto. Este disco maestro sirve para hacer un molde, con salientes en donde estarán los orificios del láser. En este molde se inyecta resina de policarbonato fundida para formar un CD con el mismo patrón de orificios que el máster de vidrio. Luego se deposita una capa muy delgada de aluminio reflejante sobre el policarbonato, seguida de una laca protectora y finalmente una etiqueta. Las depresiones en el sustrato de policarbonato se llaman **fosos** (*pits*); las áreas no quemadas entre los fosos se llaman **lands**.

Durante la reproducción, un diodo láser de baja potencia dirige luz infrarroja con una longitud de onda de 0.78 micras hacia los fosos y lands que van pasando. El láser incide en el lado del policarbonato, por lo que los fosos se proyectan hacia el láser como salientes de una superficie por lo demás plana. Dado que los fosos tienen una altura de una cuarta parte de la longitud de onda de la luz láser, la luz que se refleja de un foso está defasada media longitud de onda respecto a la luz que se refleja de la superficie circundante. El resultado es que las dos partes se interfieren destructivamente y devuelven menos luz al fotodetector del reproductor que la luz que se refleja de un land. Es así como el reproductor distingue un foso de un land.

Aunque podría parecer que lo más fácil es usar un foso para registrar un 0 y un land para registrar un 1, es más confiable utilizar una transición foso/land o land/foso para registrar un 1, y su ausencia para registrar un 0, y éste es el esquema que se usa.

Los fosos y lands se escriben en una sola espiral continua que comienza cerca del agujero y llega hasta una distancia de 32 mm del borde. La espiral describe 22,188 revoluciones alrededor del disco (unas 600 por mm). Si se desenrollara, tendría una longitud de 5.6 km. La espiral se ilustra en la figura 2-24.

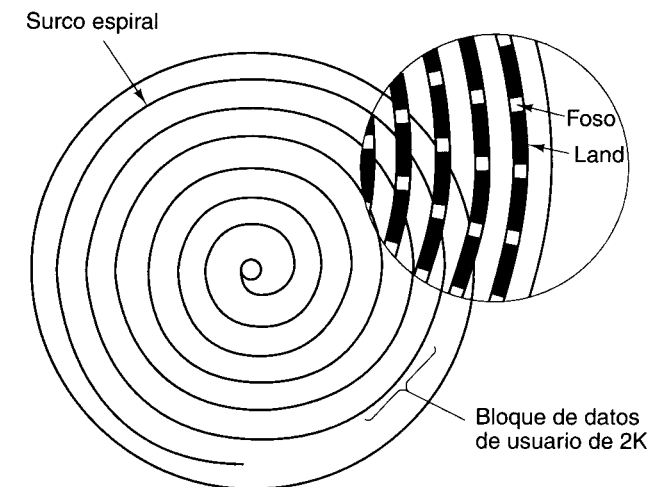


Figura 2-24. Estructura de grabación de un disco compacto o CD-ROM.

Para poder reproducir la música con una rapidez uniforme es preciso que los fosos y lands pasen bajo el láser con una velocidad lineal constante. Por consiguiente, es preciso reducir continuamente la rapidez de rotación del CD a medida que la cabeza de lectura se mueve del interior del CD hacia el exterior. En el interior, la rapidez de rotación es de 530 rpm para lograr la velocidad lineal deseada de 120 cm/s; en el exterior tiene que bajar a 200 rpm para dar la misma velocidad lineal en la cabeza. Una unidad de disco con velocidad lineal constante es muy diferente de una unidad de disco magnético, que opera a velocidad angular constante, independiente de la posición de la cabeza. Además, 530 rpm es muy diferente de las 3600 a 7200 rpm con que gira la mayor parte de los discos magnéticos.

En 1984, Philips y Sony se percataron del potencial que tenían los CD para almacenar datos de computadora, así que publicaron el **Libro Amarillo** que define una norma precisa para lo que ahora llamamos **CD-ROM (disco compacto-memoria sólo de lectura, Compact Disc - Read Only Memory)**. A fin de aprovechar el para entonces floreciente mercado de los CD de audio, los CD-ROM tendrían el mismo tamaño físico que los CD de audio, serían mecánica y ópticamente compatibles con ellos, y se producirían empleando las mismas máquinas de moldeo por inyección de policarbonato. Las consecuencias de esta decisión fueron

que se requerirían motores lentos de velocidad variable, pero también que el costo de fabricación de un CD-ROM sería muy inferior a un dólar en volúmenes moderados.

Lo que el Libro Amarillo definió fue el formato de los datos para computadora, y también mejoró las capacidades de corrección de errores del sistema. Este paso era indispensable porque, si bien a los aficionados a la música no les importa perder un bit de vez en cuando, los aficionados a la computación suelen ser muy melindrosos al respecto. El formato básico de un CD-ROM consiste en codificar cada byte en un símbolo de 14 bits. Como vimos antes, 14 bits bastan para codificar por Hamming un byte de 8 bits y hasta sobran dos bits. De hecho, se utiliza un sistema de codificación más potente. La transformación de 14 a 8 durante la lectura se efectúa en hardware mediante tablas.

En el siguiente nivel superior, un grupo de 42 símbolos consecutivos constituye un **cuadro** de 588 bits. Cada cuadro contiene 192 bits de datos (24 bytes). Los 396 bits restantes sirven para corrección de errores y control. Hasta aquí, el esquema es idéntico para los CD de audio y los CD-ROM.

Lo que el Libro Amarillo añade es el agrupamiento de 98 cuadros en un **sector de CD-ROM**, como se muestra en la figura 2-25. Cada sector de CD-ROM comienza con un preámbulo de 16 bytes, de los cuales los primeros 12 son 00FFFFFFFFFFFFFFFFF00 (hexadecimal), para que el reproductor pueda reconocer el inicio de un sector de CD-ROM. Los siguientes tres bytes contienen el número de sector, que es necesario porque buscar en un CD-ROM con su espiral de datos única es mucho más difícil que en un disco magnético con sus pistas concéntricas uniformes. Para buscar, el software de la unidad calcula aproximadamente a dónde debe mover la cabeza, coloca la cabeza ahí, y comienza a buscar un preámbulo para ver cuán cerca está de donde debería estar. El último byte del preámbulo es el modo.

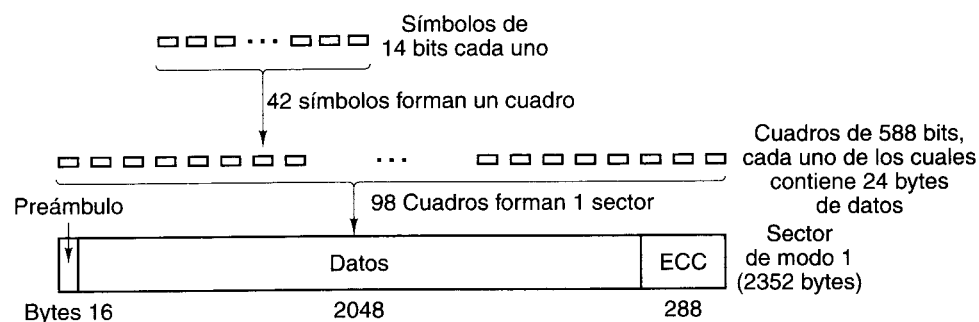


Figura 2-25. Organización lógica de datos en un CD-ROM.

El Libro Amarillo define dos modos. El modo 1 utiliza la organización de la figura 2-25, con un preámbulo de 16 bytes, 2048 bytes de datos y un código de corrección de errores de 288 bytes (un código Reed-Solomon con intercalación cruzada). El modo 2 combina los campos de datos y de ECC en un campo de datos de 2336 bytes para las aplicaciones que no necesitan (o que no tienen tiempo para efectuar) corrección de errores, como audio y video. Cabe señalar que, a fin de lograr una confiabilidad excelente, se emplean tres esquemas de corrección de errores distintos: dentro de un símbolo, dentro de un cuadro y dentro de un

sector de CD-ROM. Los errores de un solo bit se corrigen en el nivel más bajo, los errores de ráfaga corta se corrigen en el nivel de cuadro y cualesquier errores residuales se atrapan en el nivel de sector. El precio que se paga por esta confiabilidad es que se requieren 98 cuadros de 588 bits (7203 bytes) para llevar una carga útil de 2048 bytes, una eficiencia de sólo 28%.

Las unidades de CD-ROM de velocidad sencilla operan a 75 sectores/s, lo que da una tasa de datos de 153,600 bytes/s en el modo 1 y 175,200 bytes/s en el modo 2. Las unidades de doble velocidad son dos veces más rápidas, y así hasta la velocidad más alta. Un CD de audio estándar tiene capacidad para 74 minutos de música que, si se usara para datos en modo 1 sería de 681,984,000 bytes. Esta cifra suele informarse como 650 MB porque 1 MB es  $2^{20}$  bytes (1,048,576 bytes), no 1,000,000 bytes.

Cabe señalar que ni siquiera una unidad CD-ROM 32x (4,915,200 bytes/s) es rival para una unidad de disco magnético SCSI-2 rápida que opera a 10 MB/s, aunque muchas unidades CD-ROM utilizan la interfaz SCSI (también existen unidades CD-ROM IDE). Si nos damos cuenta de que el tiempo de búsqueda suele ser de varios cientos de milisegundos, es evidente que las unidades de CD-ROM no están en la misma categoría de desempeño que las de disco magnético, a pesar de su gran capacidad.

En 1986, Philips dio un nuevo golpe con el **Libro Verde**, que añadía gráficos y la capacidad para intercalar audio, video y datos en el mismo sector, función indispensable en los CD-ROM de multimedia.

La última pieza del rompecabezas de los CD-ROM es el sistema de archivos. Para poder usar el mismo CD-ROM en diferentes computadoras, era necesario llegar a un acuerdo respecto a los sistemas de archivos de los CD-ROM. Para lograr este acuerdo, representantes de muchas compañías de computación se reunieron en Lake Tahoe, en la frontera entre California y Nevada, e idearon un sistema de archivos que llamaron **High Sierra** (por el nombre que recibe la región) y que evolucionó hasta convertirse en una Norma Internacional (IS 9660). La norma tiene tres niveles. En el nivel 1 se usan nombres de archivo de hasta ocho caracteres seguidos opcionalmente de hasta tres caracteres (la convención para nombres de archivo en MS-DOS). Los nombres de archivo sólo pueden contener letras mayúsculas, dígitos y el carácter de subrayado. Los directorios pueden anidarse hasta una profundidad de ocho, pero los nombres de directorio no pueden contener extensiones. El nivel 1 requiere que todos los archivos sean contiguos, lo cual no es un problema en un medio que se escribe sólo una vez. Cualquier CD-ROM que se ajuste a la IS 9660 nivel 1 se puede leer con MS-DOS, una computadora Apple, una computadora UNIX o casi cualquier otra computadora. Los productores de CD-ROM consideran esta propiedad como una gran ventaja.

IS 9660 nivel 2 permite nombres de hasta 32 caracteres, y el nivel 3 permite archivos no contiguos. Las extensiones Rock Ridge (nombre que se les dio caprichosamente para recordar el pueblo que sale en la película *Blazing Saddles* de Gene Wilder) permiten nombres muy largos (para UNIX), UID, GID y vínculos simbólicos, pero los CD-ROM que no se ajustan al nivel 1 no pueden leerse en todas las computadoras.

Los CD-ROM se han vuelto extremadamente populares para publicar juegos, películas, enciclopedias, atlas y obras de referencia de todo tipo. Casi todo el software comercial se vende ahora en CD-ROM. Su combinación de alta capacidad y bajo costo de fabricación los hace ideales para un sinnúmero de aplicaciones.

### 2.3.8 CD grabables

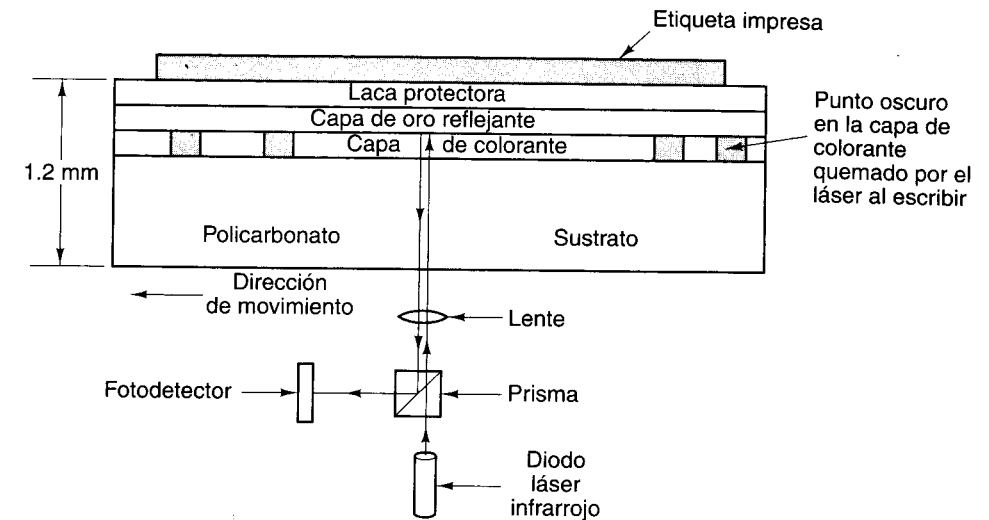
Inicialmente, el equipo requerido para producir un máster de CD-ROM (o de un CD de audio, para el caso) era extremadamente costoso. Pero como suele suceder en la industria de la computación, todos los precios bajan tarde o temprano. Para mediados de los noventa las grabadoras de CD no más grandes que un reproductor de CD, eran un periférico común que podía adquirirse en casi cualquier tienda de computación. Estos dispositivos seguían siendo diferentes de los discos magnéticos porque, una vez escritos, los CD-ROM no podían borrarse. No obstante, encontraron rápidamente un nicho como medio de respaldo para discos duros grandes y también permitieron a individuos o compañías incipientes fabricar sus propios CD-ROM en cantidades pequeñas o producir másters para entregar a plantas de duplicación de CD comerciales de alto volumen. Estas unidades se denominan **CD-R (CD grabables, CD-Recordables)**.

Físicamente, los CD-R parten de discos de policarbonato en blanco de 120 mm que son parecidos a los CD-ROM, sólo que contienen un surco de 0.6 mm de ancho para guiar al láser durante la escritura. El surco tiene una excursión senoidal de 0.3 mm a una frecuencia de exactamente 22.05 kHz, a fin de proporcionar una retroalimentación continua que permita vigilar con exactitud la velocidad de rotación, y ajustarla si es necesario. Los CD-R tienen un aspecto parecido al de los CD-ROM normales, excepto que son dorados por la parte de arriba, en lugar de plateados. El color dorado proviene del uso de oro en lugar de aluminio para la capa reflejante. A diferencia de los CD plateados, que tienen depresiones físicas, en los CD-R la diferencia en reflectividad entre los fosos y los lands tiene que simularse. Esto se hace añadiendo una capa de colorante entre el policarbonato y la capa de oro reflejante, como se muestra en la figura 2-26. Se utilizan dos tipos de colorante: cianina, que es verde, y ftalocianina, que es de color anaranjado amarillento. Los químicos pueden discutir horas y horas acerca de cuál es mejor. Dichos colorantes son similares a los que se utilizan en fotografía, lo que explica por qué Kodak y Fuji son fabricantes importantes de discos CD-R.

En su estado inicial, la capa de colorante es transparente y permite a la luz láser pasar a través y reflejarse en la capa de oro. Para escribir, el láser del CD-R se alimenta con alta potencia (8-16 mW). Cuando el haz incide sobre un punto con colorante, lo calienta y rompe uno de sus enlaces químicos. Este cambio en la estructura molecular crea un punto oscuro. Durante la reproducción (a 0.5 mW), el fotodetector percibe una diferencia entre los puntos oscuros donde el colorante fue quemado y las áreas transparentes donde está intacto. Esta diferencia se interpreta como la diferencia entre fosos y lands, cuando se reproduce con un lector de CD-ROM normal o con un reproductor de CD de audio.

Un tipo nuevo de CD sentiría una vergüenza enorme si no tuviera su propio libro de color, así que el CD-R tiene el **Libro Anaranjado**, que se publicó en 1989. Este documento define el CD-R y también un nuevo formato, **CD-ROM XA**, que permite escribir los CD-R de forma incremental: unos cuantos sectores hoy, otros pocos mañana, y unos más el mes próximo. Un grupo de sectores consecutivos que se escriben en una sola operación se llama **pista de CD-ROM**.

Uno de los primeros usos del CD-R fue el PhotoCD de Kodak. Con este sistema, el cliente lleva al procesador fotográfico un rollo de película expuesta y su antiguo PhotoCD, y luego recoge el mismo PhotoCD al que se han añadido las nuevas imágenes después de las



**Figura 2-26.** Sección transversal de un disco CD-R y láser (no está a escala). Un CD-ROM plateado tiene una estructura similar, excepto que no tiene capa de colorante y tiene una capa de aluminio con fosos en lugar de una capa de oro.

antiguas. El nuevo lote, que se crea digitalizando los negativos, se escribe en el PhotoCD como pista de CD-ROM aparte. Se requiere una escritura incremental porque los discos de CD-R en blanco son demasiado costosos como para gastar uno en cada rollo de película.

Sin embargo, la escritura incremental crea un problema nuevo. Antes del Libro Anaranjado, todos los CD-ROM tenían una sola **VTOC (tabla de contenido del volumen, Volume Table of Contents)** al principio. Ese esquema no funciona con las escrituras incrementales (es decir, de múltiples pistas). La solución del Libro Anaranjado es dar a cada pista de CD-ROM su propia VTOC. Los archivos enumerados en la VTOC pueden incluir algunos de los archivos de pistas anteriores, o todos. Una vez que el CD-R se inserta en la unidad, el sistema operativo examina todas las pistas de CD-ROM hasta encontrar la VTOC más reciente, que contiene la situación actual del disco. Al incluir algunos de los archivos de pistas anteriores, pero no todos, en la VTOC vigente, es posible crear la ilusión de que se han borrado archivos. Las pistas pueden agruparse en **sesiones**, lo que da pie a **CD-ROM multisesión**. Los reproductores de CD de audio estándar no pueden manejar los CD multisesión porque esperan una sola VTOC al principio.

Cada pista tiene que escribirse en una sola operación continua sin pausas. Por consiguiente, el disco duro del cual provienen los datos tiene que ser lo bastante rápido como para suministrarlos a tiempo. Si los archivos que se van a copiar están dispersos por todo el disco duro, los tiempos de búsqueda podrían hacer que el flujo de datos al CD-R se interrumpa y propicie una insuficiencia de buffer. El resultado es un bonito y brillante (pero un tanto costoso) posavasos para sus bebidas, o un frisbee (disco volador) dorado de 120 mm. El software de CD-R generalmente ofrece la opción de reunir todos los archivos de entrada en una sola imagen de CD-ROM contigua de 650 MB antes de quemar el CD-R, pero este proceso nor-

malmente duplica el tiempo de escritura efectivo, requiere 650 MB de espacio libre en disco y aun así no protege contra las recalibraciones térmicas repentinas que efectúan los discos duros cuando se calientan demasiado.

El CD-R permite a individuos y compañías copiar fácilmente los CD-ROM (y los CD de audio), generalmente con violación de los derechos de autor del productor. Se han ideado varios esquemas para obstaculizar tal piratería y dificultar la lectura de un CD-ROM utilizando otra cosa que no sea el software del productor. Uno de ellos implica registrar la longitud de los archivos del CD-ROM como de varios gigabytes, lo que frustra cualquier intento por copiar los archivos en el disco duro empleando software de copiado estándar. Las verdaderas longitudes están incorporadas en el software del productor u ocultas (tal vez en forma cifrada) en el CD-ROM en un lugar inesperado. Otro esquema utiliza ECC intencionalmente incorrectos en sectores selectos, con la esperanza de que el software de copiado de CD "corrija" los errores. El software de aplicación verifica los ECC por sí mismo, y se niega a funcionar si son correctos. Otras posibilidades son el uso de separaciones no estándar entre las pistas y otros "defectos" físicos.

### 2.3.9 CD reescribibles

Aunque estamos acostumbrados a varios medios de escritura única, como el papel y la película fotográfica, existe demanda por un CD-ROM reescribible. Una tecnología que ya está disponible es **CD-RW** (**CD reescribible**, *CD-ReWritable*), que utiliza medios del mismo tamaño que los CD-R, sólo que en lugar de un colorante como cianina o ftalocianina, el CD-RW emplea una aleación de plata, indio, antimonio y telurio para la capa de grabación. Esta aleación tiene dos estados estables: cristalina y amorfa, con diferente reflectividad.

Las unidades de CD-RW emplean láseres con tres potencias distintas. A la potencia alta, el láser funde la aleación, transformándola del estado cristalino de alta reflectividad, al estado amorfo de baja reflectividad. Esto representa un foso. A la potencia media, la aleación se funde y se recristaliza en su estado natural para convertirse otra vez en un land. A la potencia baja, el estado del material se detecta (para lectura), pero no ocurre ninguna transición de fase.

La razón por la que el CD-RW no ha suplantado al CD-R es que los discos CD-RW en blanco son mucho más costosos que los de CD-R. Además, cuando el disco se usa para crear copias de seguridad de discos duros, el hecho de que una vez escrito un CD-R no puede borrarse accidentalmente es una gran ventaja.

### 2.3.10 DVD

El formato de CD/CD-ROM básico se estableció desde 1980. Desde entonces la tecnología ha mejorado y ha hecho factibles económicamente discos ópticos de mayor capacidad, para los cuales hay gran demanda. A Hollywood le encantaría sustituir las videocintas analógicas por discos digitales, ya que éstos tienen más alta calidad, su fabricación cuesta menos, duran más, ocupan menos espacio en los anaqueles de las tiendas de video y no tienen que rebobinarse. Las compañías de electrónica para el consumidor están buscando un nuevo producto que se

venda mucho, y un gran número de compañías de computación quieren añadir características multimedia a su software.

Esta combinación de tecnología y demanda por parte de tres industrias inmensamente ricas y poderosas ha dado origen al **DVD**, que originalmente era un acrónimo de **videodisco digital** (*Digital Video Disc*) pero ahora significa oficialmente **disco digital versátil** (*Digital Versatile Disc*). Los DVD tienen el mismo diseño general que los CD, con discos de policarbonato de 120 mm moldeados por inyección que contienen fosos y lands y que se iluminan con un diodo láser para ser leídos por un fotodetector. Lo que es nuevo es el uso de

1. Fosos más pequeños (0.4 micras en lugar de 0.8 micras para los CD).
2. Una espiral más cerrada (0.74 micras entre pistas en lugar de 1.6 micras para los CD).
3. Un láser rojo (de 0.65 micras en vez de 0.78 micras para los CD).

Juntas, estas mejoras multiplican la capacidad siete veces, a 4.7 GB. Una unidad de DVD 1x opera a 1.4 MB/s (compárese con 150 KB/s para los CD). Desafortunadamente, el cambio a los láseres rojos empleados en los supermercados implica que los reproductores de DVD requieren un segundo láser o una óptica de conversión precisa para poder leer los CD y CD-ROM existentes, algo que tal vez no hagan algunos de ellos. Además, quizá podrían no leerse los CD-R y los CD-RW en una unidad DVD.

¿Es suficiente 4.7 GB? Tal vez. Utilizando compresión MPEG-2 (estandarizada en IS 13346), un DVD de 4.7 GB puede contener 133 minutos de video de pantalla completa con pleno movimiento y alta definición (720 × 480), así como pistas sonoras en hasta ocho idiomas y subtítulos en 32 más. Cerca de 92% de las películas que ha hecho Hollywood duran menos de 133 minutos. No obstante, algunas aplicaciones como los juegos de multimedia o las obras de referencia podrían necesitar más, y a Hollywood le gustaría grabar varias películas en el mismo disco. Por ello, se han definido cuatro formatos:

1. Un solo lado, una sola capa (4.7 GB).
2. Un solo lado, capa dual (8.5 GB).
3. Dos lados, una sola capa (9.4 GB).
4. Dos lados, capa dual (17 GB).

¿Por qué tantos formatos? En una palabra: política. Philips y Sony querían discos de un solo lado y capa dual para la versión de alta capacidad, pero Toshiba y Time Warner querían discos de dos lados y una sola capa. Philips y Sony no creían que la gente estaría dispuesta a darle vuelta al disco, y Time Warner no creía que fuera factible colocar dos capas en un mismo lado. La concesión: todas las combinaciones, pero el mercado decidirá cuáles han de sobrevivir.

La tecnología de capa dual tiene una capa reflejante abajo, cubierta por una capa semirreflejante. Dependiendo de la distancia focal del láser, el haz se refleja en una capa o en la otra. La capa inferior necesita fosos y lands un poco mayores para que su lectura sea confiable, por lo que su capacidad es un poco menor que la de la capa superior.

Los discos de dos lados se fabrican tomando dos discos de un solo lado, de 0.6 mm, y pegándolos espalda con espalda. Para que el espesor de todas las versiones sea el mismo, un disco de un solo lado consiste en un disco de 0.6 mm pegado a un sustrato en blanco (o tal vez, en el futuro, uno que contenga 133 minutos de anuncios, en la esperanza de que la gente sentirá curiosidad por ver qué hay ahí abajo). La estructura del disco de dos lados y capa dual se ilustra en la figura 2-27.

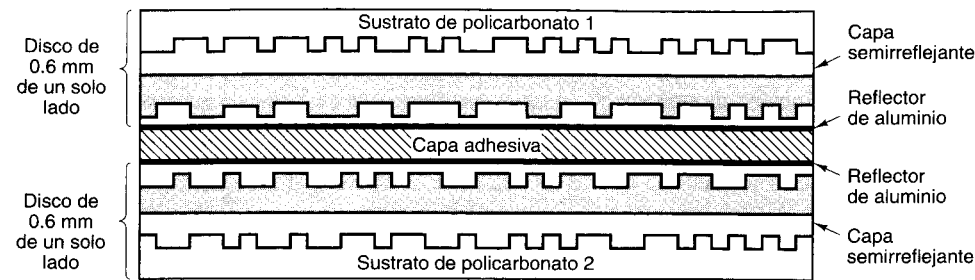


Figura 2-27. Disco DVD de dos lados y capa dual.

El DVD fue inventado por un consorcio de 10 compañías de electrónica para el consumidor, siete de ellas japonesas, en estrecha cooperación con los principales estudios de Hollywood (algunos de los cuales son propiedad de las compañías japonesas de electrónica del consorcio). Las industrias de las computadoras y telecomunicaciones no fueron invitadas al picnic, y el enfoque radicó en el uso del DVD para la renta de películas y exposiciones de ventas. Por ejemplo, entre las funciones estándar están la omisión en tiempo real de escenas fuertes (para que los padres puedan convertir una película para adultos en una que toda la familia pueda ver), sonido de seis canales, y apoyo de Pan-and-Scan. Esta última función permite al reproductor de DVD decidir dinámicamente cómo debe recortar los bordes izquierdo y derecho de las películas (cuya relación anchura:altura es de 3:2) de modo que quepan en las pantallas de televisión actuales (cuya relación es 4:3).

Otro aspecto sobre el que tal vez no se había hablado en la industria de las computadoras es una incompatibilidad intencional entre los discos hechos para distribuirse en Estados Unidos y aquéllos para Europa, además de otros estándares para otros continentes. Hollywood exigió esta "función" porque las películas nuevas siempre se exhiben primero en Estados Unidos y luego se envían a Europa cuando los videos salen a la venta en Estados Unidos. Lo que se buscaba era evitar que las tiendas de video europeas compraran videos en Estados Unidos antes de tiempo, lo cual reduciría los ingresos por la exhibición de esas películas en los cines europeos. Si Hollywood controlara la industria de las computadoras, se usarían disquetes de 3.5" en Estados Unidos y de 9cm en Europa.

Si el DVD tiene mucho éxito, en poco tiempo se producirán en masa los DVD-R (grabables) y DVD reescribibles. Sin embargo, el éxito de los DVD no está garantizado, ya que las compañías de cable tienen un plan muy diferente para distribuir películas —video por demanda de cable— y la batalla apenas comienza.

## 2.4 ENTRADA/SALIDA

Como mencionamos al principio del capítulo, un sistema de computación tiene tres componentes principales: la CPU, las memorias (primaria y secundaria) y el equipo de E/S (**entrada/salida**, *Input/Output*) que incluye impresoras, escáneres y módems. Ya hemos hablado de la CPU y las memorias. Ha llegado el momento de examinar el equipo de E/S y ver cómo se conecta con el resto del sistema.

### 2.4.1 Buses

Físicamente, la mayor parte de las computadoras personales y estaciones de trabajo tiene una estructura similar a la que se muestra en la figura 2-28. La disposición usual es una caja metálica con una tarjeta grande de circuitos impresos en su base, llamada **tarjeta madre** o *motherboard* (o tarjeta familia o *parentboard*, para quienes quieren ser diplomáticamente correctos). La tarjeta madre contiene el chip de CPU, algunas ranuras en las que pueden insertarse módulos DIMM, y diversos chips de apoyo. Además, contiene un bus grabado a todo su largo, y zócalos en los que pueden insertarse los conectores de arista de tarjetas de E/S. A veces hay dos buses, uno de alta velocidad (para las tarjetas de E/S modernas) y uno de baja velocidad (para las tarjetas de E/S más viejas).

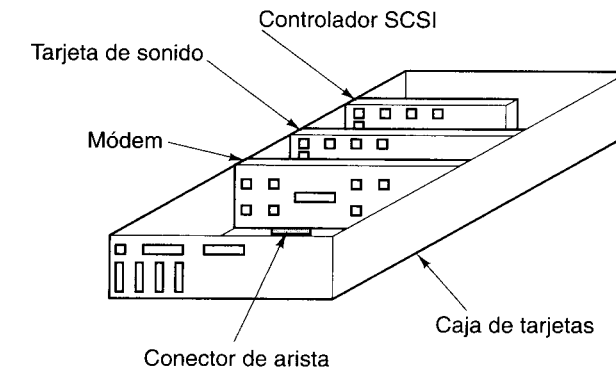


Figura 2-28. Estructura física de una computadora personal.

En la figura 2-29 se muestra la estructura lógica de una sencilla computadora personal. Ésta tiene un solo bus que sirve para conectar la CPU, la memoria y los dispositivos de E/S; casi todos los sistemas tienen dos o más buses. Cada dispositivo de E/S consta de dos partes: una que contiene casi todos los circuitos electrónicos, llamada **controlador**, y una que contiene el dispositivo de E/S propiamente dicho, como una unidad de disco. El controlador suele estar contenido en una tarjeta que se inserta en una ranura desocupada, con excepción de los controladores que no son opcionales (como el del teclado), los cuales a veces se encuentran en la tarjeta madre. Aunque la pantalla (monitor) no es opcional, el controlador de video a veces se encuentra en una tarjeta insertable para que el usuario pueda escoger entre tarjetas con o sin aceleradores de gráficos, memoria adicional, etc. El controlador se conecta a su dispositivo con un cable unido a un conector en la parte de trasera del gabinete.

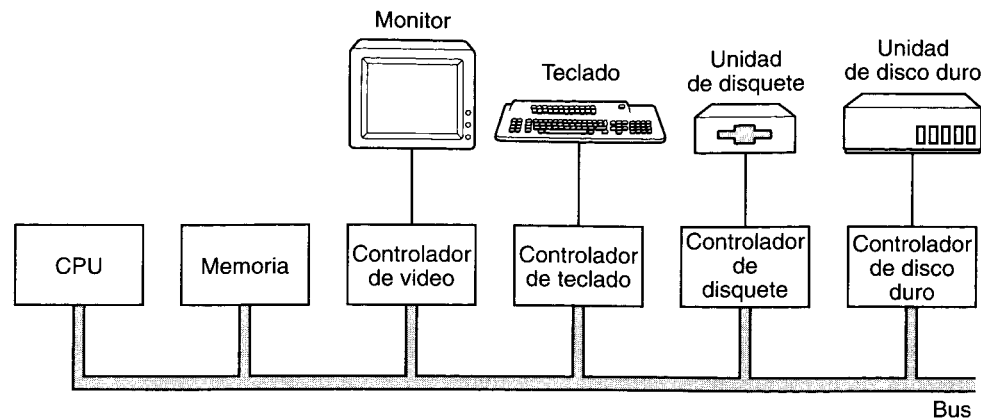


Figura 2-29. Estructura lógica de una computadora personal sencilla.

La tarea de un controlador es dominar su dispositivo de E/S y manejar su acceso al bus. Por ejemplo, cuando un programa quiere datos del disco, envía un comando al controlador del disco, que a su vez emite comandos de búsqueda y de otro tipo a la unidad de disco. Una vez que se localizan la pista y el sector apropiados, la unidad de disco comienza a enviar los datos como un flujo de bits en serie al controlador. Corresponde a éste dividir el flujo de bits en unidades y escribir cada unidad en la memoria conforme se ensambla. Las unidades constan normalmente de una o más palabras. Un controlador que lee datos de la memoria o los escribe en ella sin intervención de la CPU está efectuando **acceso directo a la memoria**, mejor conocido por sus iniciales en inglés, **DMA (Direct Memory Access)**. Una vez completada la transferencia, el controlador normalmente genera una **interrupción** que obliga a la CPU a dejar de ejecutar su programa actual y comenzar a ejecutar un procedimiento especial, llamado **manejador de interrupciones**, para verificar la presencia de errores, efectuar cualquier acción especial que se requiera, e informar al sistema operativo sobre la finalización de la E/S. Cuando el manejador de interrupciones concluye su intervención, la CPU continúa con el programa que se suspendió cuando ocurrió la interrupción.

El bus no sólo es utilizado por los controladores de E/S, sino también por la CPU para obtener instrucciones y datos. ¿Qué sucede si la CPU y un controlador de E/S quieren usar el bus al mismo tiempo? La respuesta es que un chip llamado **árbitro de bus** decide quién tendrá el acceso. En general, los dispositivos de E/S tienen preferencia sobre la CPU, porque los discos y otros dispositivos móviles no pueden detenerse, y obligarlos a esperar podría causar la pérdida de datos. Si no se está efectuando E/S, la CPU puede aprovechar todos los ciclos de bus para hacer referencia a la memoria. Pero si también está funcionando un dispositivo de E/S, éste solicitará el bus cuando lo necesite, y se le otorgará. Este proceso se llama **robo de ciclos** y hace más lenta a la computadora.

Este diseño funcionaba bien en las primeras computadoras personales, porque todos los componentes estaban más o menos balanceados. Pero a medida que aumentó la rapidez de las CPU, memorias y dispositivos de E/S, surgió un problema: el bus ya no podía manejar la carga que se le presentaba. En un sistema cerrado, como una estación de trabajo de ingeniería,

la solución era diseñar un bus nuevo más rápido para el siguiente modelo. Puesto que a nadie se le ocurría pasar dispositivos de E/S de un modelo viejo a uno nuevo, la estrategia funcionaba bien.

En cambio, en el mundo de las PC era común que la gente modernizara su CPU pero quisiera seguir usando su impresora, su escáner y su módem anteriores con el nuevo sistema. Además, había surgido una enorme industria dedicada a proveer una inmensa gama de dispositivos de E/S para el bus de la IBM PC, y esta industria no tenía ningún interés en tirar por la borda toda su inversión y comenzar otra vez desde el principio. IBM aprendió esta lección por las malas cuando sacó la sucesora de la IBM PC, la serie PS/2. La PS/2 tenía un bus nuevo, más rápido, pero casi todos los fabricantes de clones siguieron usando el viejo bus de la PC, ahora llamado bus **ISA (arquitectura estándar de la industria, Industry Standard Architecture)**. También, la mayoría de los fabricantes de discos y dispositivos de E/S siguieron produciendo controladores para ese bus, e IBM se encontró en la peculiar situación de ser el único fabricante de PC que no era compatible con IBM. Finalmente, IBM se vio obligada a apoyar otra vez el bus ISA. Por cierto, cabe señalar que ISA significa arquitectura del conjunto de instrucciones (*Instruction Set Architecture*) en el contexto de niveles de máquina, pero quiere decir arquitectura estándar de la industria en el ámbito de los buses.

No obstante, a pesar de la presión del mercado contra los cambios, el viejo bus realmente era demasiado lento, por lo que había que hacer algo. Esta situación hizo que otras compañías crearan máquinas con múltiples buses, uno de los cuales era el viejo bus ISA, o su sucesor compatible con lo existente, el **EISA (ISA extendido)**. El más popular de éstos es actualmente el bus **PCI (interconexión de componentes periféricos, Peripheral Component Interconnect)**. Intel diseñó este bus, pero decidió colocar todas las patentes en el dominio público para animar a toda la industria (incluidos sus competidores) a que lo adoptaran.

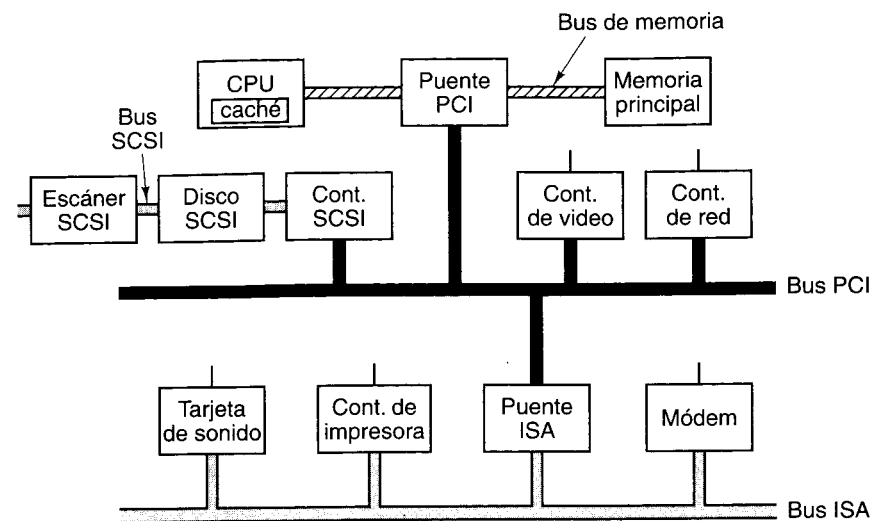
El bus PCI se puede usar en muchas configuraciones, pero la que se ilustra en la figura 2-30 es típica. Aquí la CPU se comunica con un controlador de memoria por una conexión dedicada de alta velocidad. El controlador se comunica con la memoria y con el bus PCI directamente, de modo que el tráfico entre la CPU y la memoria no usa el bus PCI. En cambio, los periféricos con gran ancho de banda (es decir, tasa de datos elevada), como los discos SCSI, se pueden conectar directamente al bus PCI. Además, el bus PCI tiene un puente con el bus ISA, lo que permite seguir usando controladores ISA y sus dispositivos. Una máquina con este diseño incluiría por lo regular tres o cuatro ranuras PCI vacías y otras tres o cuatro ranuras ISA vacías, para que los clientes pudieran insertar tarjetas de E/S ISA viejas (generalmente para dispositivos lentos) y también tarjetas de E/S PCI nuevas (generalmente para dispositivos rápidos).

Actualmente se venden muchas clases de dispositivos de E/S. A continuación describiremos algunos de los más comunes.

## 2.4.2 Terminales

Las terminales de computadora constan de dos partes: un teclado y un monitor. En el mundo de las mainframes, estas partes a menudo se integran en un solo dispositivo y se conectan a la computadora principal con una línea en serie o una línea telefónica. En las industrias de las reservaciones de vuelos, bancos y otras que están orientadas hacia mainframes, estos dispositivos se siguen usando ampliamente. En el mundo de las computadoras personales el teclado y el monitor son dispositivos independientes. En ambos casos, la tecnología de las dos partes es la misma.





**Figura 2-30.** PC moderna representativa con un bus PCI y un bus ISA. El módem y la tarjeta de sonido son dispositivos ISA; el controlador SCSI es un dispositivo PCI.

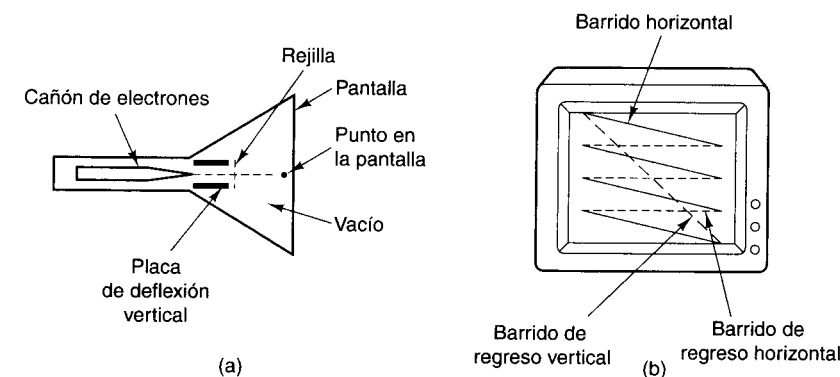
## Teclados

Los teclados son de varios tipos. La IBM PC original se vendía con un teclado que tenía un interruptor de acción brusca bajo cada tecla que proporcionaba retroalimentación táctil y emitía un chasquido cuando la tecla se oprimía lo suficiente. Hoy en día los teclados más baratos tienen teclas que se limitan a hacer contacto mecánico cuando se oprimen. Los mejores tienen una lámina de material elastomérico (una especie de caucho) entre las teclas y la tarjeta de circuitos impresos subyacente. Bajo cada tecla hay un pequeño domo que se aplasta cuando se le oprime con la fuerza suficiente. Un pequeño punto de material conductor dentro del domo cierra el circuito. Algunos teclados tienen un imán debajo de cada tecla que pasa por el interior de una bobina cuando la tecla se oprime e induce una corriente que puede detectarse. Además de éstos, se usan otros métodos, tanto mecánicos como electromagnéticos.

En las computadoras personales, cuando se oprime una tecla, se genera una interrupción que hace que se inicie el manejador de interrupciones del teclado (un programa que forma parte del sistema operativo). El manejador de interrupciones lee un registro de hardware dentro del controlador del teclado para obtener el número de la tecla (1 a 102) que recién se ha oprimido. Cuando se suelta una tecla, se causa una segunda interrupción. De este modo, cuando un usuario oprime la tecla SHIFT (mayúsculas), luego oprime y suelta la tecla M, y luego suelta la tecla SHIFT, el sistema operativo interpreta que el usuario quiere una "M" mayúscula y no una "m" minúscula. El manejo de secuencias de múltiples teclas en las que intervienen SHIFT, CTRL y ALT se realiza totalmente en software (incluida la secuencia CTRL-ALT-DEL de triste fama que sirve para reiniciar todas las IBM PC y sus clones).

## Monitores de CRT

Un monitor es una caja que contiene un **CRT (tubo de rayos catódicos, Cathode Ray Tube)** y sus fuentes de potencia. El CRT contiene un cañón que puede disparar un haz de electrones contra una pantalla fosforescente cerca del frente del tubo, como se muestra en la figura 2-31(a). (Los monitores a color tienen tres cañones de electrones, uno para cada uno de los colores rojo, verde y azul.) Durante el barrido horizontal, el haz cruza la pantalla en unos 50  $\mu$ s, describiendo una línea casi horizontal en ella. Luego ejecuta un retrazo horizontal para regresar al borde izquierdo e iniciar el siguiente barrido. Un dispositivo como éste que produce una imagen línea por línea se denomina dispositivo de **barrido por cuadro**.



**Figura 2-31.** (a) Sección transversal de un CRT. (b) Patrón de barrido de un CRT.

El barrido horizontal se controla con un voltaje que aumenta linealmente y se aplica a las placas de desviación horizontal colocadas a la izquierda y la derecha del cañón de electrones. El movimiento vertical se controla con un voltaje cuyo incremento lineal es mucho más lento y se aplica a las placas de desviación vertical colocadas arriba y abajo del cañón. Después de unos 400 a 1000 barridos, los voltajes de las placas de desviación horizontal y vertical se invierten juntos rápidamente para colocar el haz otra vez en la esquina superior izquierda. Una imagen de pantalla completa normalmente se redibuja entre 30 y 60 veces cada segundo. Los movimientos del haz se muestran en la figura 2-31(b). Aunque según nuestra descripción se usan campos eléctricos para guiar el haz en su barrido de la pantalla, muchos modelos usan campos magnéticos en lugar de eléctricos, sobre todo los monitores más caros.

Para producir un patrón de puntos en la pantalla, hay una rejilla dentro del CRT. Cuando se aplica un voltaje positivo a la rejilla, los electrones se aceleran y hacen que el haz choque con la pantalla y la haga brillar efímeramente. Cuando se usa un voltaje negativo, los electrones se repelen, de modo que no pasan por la rejilla y la pantalla no brilla. Así, el voltaje aplicado a la rejilla hace que el patrón de bits correspondiente aparezca en la pantalla. Este mecanismo permite convertir una señal eléctrica binaria en una imagen formada por puntos brillantes y oscuros.



## Pantallas planas

Los CRT son demasiado voluminosos y pesados para usarse en computadoras portátiles, y es por ello que se emplea una tecnología totalmente distinta en sus pantallas. La más común es la tecnología LCD (**presentación de cristal líquido**, *Liquid Crystal Display*). Esta tecnología es muy compleja, tiene muchas variaciones y está cambiando rápidamente, así que por fuerza la presente descripción será breve y muy simplificada.

Los cristales líquidos son moléculas orgánicas viscosas que fluyen como un líquido pero también tienen una estructura espacial, como un cristal. Un botánico austriaco, Rheinitzer, los descubrió en 1888, y se utilizaron en pantallas (de calculadoras y relojes de pulsera, por ejemplo) por primera vez en la década de los sesenta. Cuando todas las moléculas están alineadas en la misma dirección, las propiedades ópticas del cristal dependen de la dirección y la polarización de la luz incidente. Con la ayuda de un campo eléctrico es posible modificar la alineación molecular y por ende las propiedades ópticas del cristal. En particular, si se coloca una lámpara detrás de un cristal líquido, es posible controlar eléctricamente la intensidad de la luz que se percibe a través del cristal. Esta propiedad puede aprovecharse en la construcción de pantallas planas.

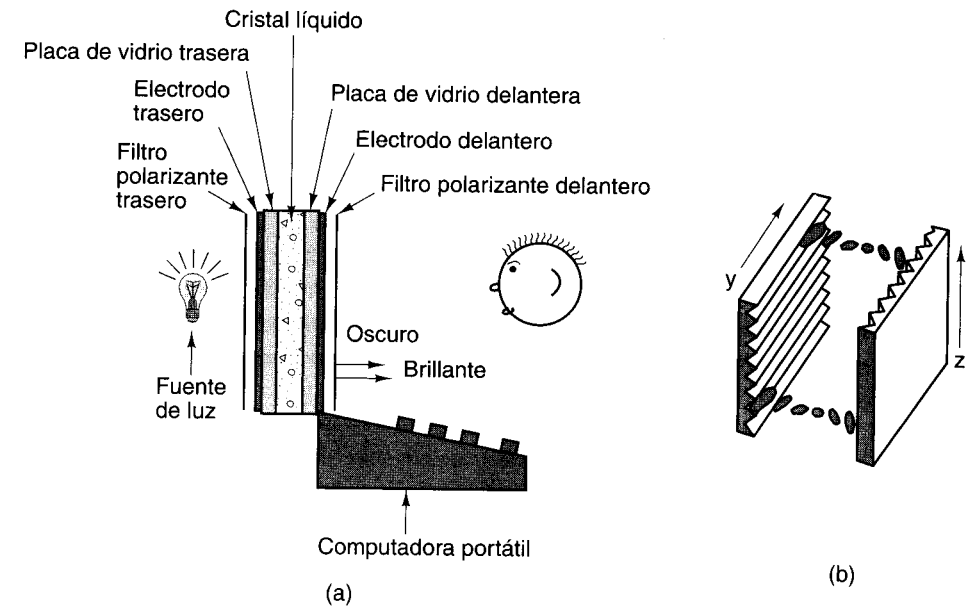
Una pantalla LCD consiste en dos placas de vidrio paralelas entre las que hay un volumen sellado que contiene un cristal líquido. Cada placa tiene conectados electrodos transparentes. Una luz detrás de la placa trasera (natural o artificial) ilumina la pantalla desde atrás. Los electrodos transparentes unidos a cada placa sirven para crear campos eléctricos en el cristal líquido. Diferentes partes de la pantalla reciben diferentes voltajes, y con esto se controla la imagen que se exhibe. Pegados a la parte de adelante y de atrás de la pantalla hay filtros polarizantes porque la tecnología de la pantalla requiere luz polarizada. La configuración general se muestra en la figura 2-32(a).

Aunque se usan muchos tipos de pantallas LCD, ahora consideraremos un tipo específico, la pantalla **TN (nemática torcida, Twisted Nematic)** como ejemplo. En esta pantalla, la placa trasera contiene diminutos surcos horizontales, y la delantera, verticales, como se muestra en la figura 2-32(b). En ausencia de un campo eléctrico, las moléculas de cristal líquido tienden a alinearse en la dirección de los surcos. Puesto que las alineaciones delantera y trasera difieren en 90 grados, las moléculas (y por tanto la estructura del cristal) se tuercen de atrás hacia adelante.

En la parte de atrás de la pantalla hay un filtro polarizante horizontal que únicamente deja pasar luz polarizada horizontalmente. En la parte delantera hay un filtro polarizante vertical que sólo permite pasar luz polarizada verticalmente. Si no hubiera líquido entre las placas, la luz polarizada horizontalmente que atraviesa el filtro polarizante trasero sería bloqueada por el filtro polarizante delantero, y la pantalla sería de color negro uniforme.

Sin embargo, la estructura cristalina torcida de las moléculas de cristal líquido guía la luz a medida que pasa y hace girar su polarización, de modo que sale polarizada horizontalmente. Así, en ausencia de un campo eléctrico, la pantalla LCD es uniformemente brillante. Si se aplica un voltaje a partes selectas de la placa, la estructura torcida se destruye, y la luz no atraviesa esas partes.

Se utilizan dos sistemas para aplicar el voltaje. En una **pantalla de matriz pasiva** (de bajo costo), ambos electrodos contienen alambres paralelos. En una pantalla de  $640 \times 480$ ,



**Figura 2-32.** (a) Construcción de una pantalla LCD. (b) Los surcos de las placas delantera y trasera son mutuamente perpendiculares.

por ejemplo, el electrodo trasero podría tener 640 alambres verticales; y el delantero, 480 alambres horizontales. Si se aplica un voltaje a uno de los alambres verticales y luego se hace pulsar uno de los horizontales, se puede alterar el voltaje en una posición definida de un pixel, haciendo que se oscurezca por un momento. Si se repite este pulso con el siguiente pixel y luego con el siguiente, se puede pintar una línea de barrido oscura, de forma análoga a como se hace en un CRT. Normalmente, toda la pantalla se pinta 60 veces por segundo para engañar el ojo y hacerlo creer que hay una imagen constante ahí, como se hace también en un CRT.

El otro esquema de uso muy difundido es la **pantalla de matriz activa**. Su costo es considerablemente mayor, pero da una mejor imagen y por ello está ganando terreno. En lugar de tener sólo dos juegos de alambres perpendiculares, tiene un diminuto elemento conmutador en cada posición de pixel en uno de los electrodos. Al encender y apagar estos elementos, puede crearse un patrón de voltaje arbitrario en toda la pantalla, y así dibujar un patrón de bits arbitrario.

Hasta ahora hemos descrito cómo funciona una pantalla monocromática. Baste decir que las pantallas a color se basan en los mismos principios generales que las monocromáticas, pero que los detalles son mucho más complicados. Se utilizan filtros ópticos para separar la luz blanca en componentes rojo, verde y azul en cada posición de pixel, de modo que puedan exhibirse de forma independiente. Cualquier color puede crearse con una superposición lineal de estos tres colores primarios.

### Terminales de mapa de caracteres

Se usan comúnmente tres tipos de terminales: de mapa de caracteres, de mapa de bits y RS-232-C. Todas ellas pueden usar cualquier tipo de teclado, pero difieren en la forma en que la computadora se comunica con ellas y en el manejo de las salidas. A continuación describiremos brevemente cada clase.

En una computadora personal hay dos formas de organizar las salidas que se envían a la pantalla: un mapa de caracteres y un mapa de bits. La figura 2-33 muestra cómo se usa un mapa de caracteres para exhibir salidas en el monitor. (El teclado se trata como un dispositivo totalmente independiente.) En la tarjeta de comunicaciones en serie hay una porción de memoria, llamada **memoria de video**, además de algunos circuitos para acceder al bus y generar señales de video.

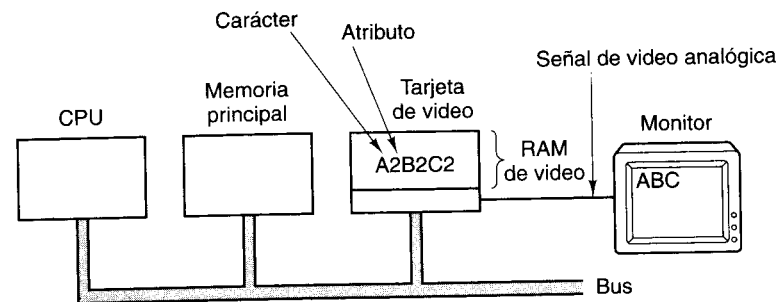


Figura 2-33. Salida en una terminal de una computadora personal.

Para exhibir caracteres, la CPU los copia en la memoria de video en bytes alternados. Cada carácter tiene asociado un **byte de atributo** que describe la forma en que debe exhibirse el carácter. Los atributos pueden incluir su color, intensidad, si está parpadeando o no, etc. Así, una imagen de pantalla de  $25 \times 80$  caracteres requiere 4000 bytes de memoria de video, 2000 para los caracteres y 2000 para los atributos. Casi todas las tarjetas tienen más memoria, a fin de mantener varias imágenes de pantalla.

La tarea de la tarjeta de video consiste en traer una y otra vez caracteres de la RAM de video y generar la señal necesaria que se alimenta al monitor. Se trae toda una línea de caracteres a la vez para poder calcular las líneas de barrido individuales. Esta señal es del tipo analógica de alta frecuencia que controla el barrido del haz de electrones que pinta los caracteres en la pantalla. Puesto que la tarjeta genera una señal de video, el monitor debe estar a unos cuantos metros de la computadora a fin de evitar distorsiones.

### Terminales de mapa de bits

Una variación de esta idea es ver la pantalla no como una matriz de caracteres  $25 \times 80$ , sino como una formación de elementos de imagen llamados **píxeles**. Cada píxel está encendido o

apagado, y representa un bit de información. En las computadoras personales la pantalla podría contener  $640 \times 480$  píxeles, pero es más común encontrar  $800 \times 600$  o más. En las estaciones de trabajo de ingeniería la pantalla normalmente tiene  $1280 \times 960$  píxeles o más. Las terminales que usan un mapa de bits en lugar de un mapa de caracteres se llaman **terminales de mapa de bits**. Todas las tarjetas de video modernas pueden operar como terminales de mapa de caracteres o de mapa de bits, bajo el control de software.

Se utiliza la misma idea general que en la figura 2-33, sólo que la RAM de video se ve como un gran arreglo de bits. El software puede crear ahí cualquier patrón que desee, y se exhibirá instantáneamente. Para dibujar caracteres, el software podría decidir asignar, por ejemplo, un rectángulo de 9 por 14 a cada carácter y "encender" los bits necesarios para que aparezca el carácter. Esta técnica permite al software crear diversos tipos de letra y entremezclarlos a voluntad. Lo único que hace el hardware es exhibir el arreglo de bits. En el caso de pantallas a color, cada píxel requiere 8, 16 o 24 bits.

Las terminales de mapa de bits suelen usarse para presentar imágenes que contienen varias **ventanas**. Una ventana es un área de la pantalla utilizada por un programa. Con múltiples ventanas es posible tener varios programas ejecutándose al mismo tiempo, y que cada uno exhiba sus resultados con independencia de los demás.

Aunque las terminales de mapa de bits son muy flexibles, tienen dos desventajas importantes. Primera, requieren una cantidad considerable de RAM de video. Los tamaños más comunes en estos días son  $640 \times 480$  (VGA),  $800 \times 600$  (SVGA),  $1024 \times 768$  (XVGA) y  $1280 \times 960$ . Cabe señalar que todas éstas tienen una relación de aspecto (anchura:altura) de 4:3, a fin de ajustarse a la relación vigente en los televisores. Para obtener color fiel, se requieren 8 bits para cada uno de los colores primarios, o sea 3 bytes/píxel. Así, una pantalla de  $1024 \times 768$  requiere 2.3 MB de RAM de video.

En vista del tamaño de la memoria necesaria, algunas computadoras hacen concesiones y utilizan un número de 8 bits para indicar el color deseado. Este número se maneja como índice de una tabla en hardware, llamada **paleta de colores**, que contiene 256 entradas, cada una con un valor RGB de 24 bits. Un diseño así, llamado **color indizado**, reduce las necesidades de memoria RAM de video en 2/3 partes, pero sólo permite 256 colores en la pantalla a la vez. Por lo regular, cada ventana de la pantalla tiene su propio mapeo, pero al haber sólo una paleta de colores en hardware es común que si hay varias ventanas en la pantalla sólo la ventana activa aparezca con sus colores correctos.

La segunda desventaja de una pantalla de mapa de bits es su desempeño. Una vez que los programadores de aplicaciones se enteran de que pueden controlar cada píxel tanto en el espacio como en el tiempo, quieren hacerlo. Aunque es posible copiar datos de la RAM de video al monitor sin pasar por el bus de sistema principal, la transferencia de datos a la RAM de video sí emplea el bus de sistema. Para exhibir multimedia de pantalla completa a todo color en una pantalla de  $1024 \times 768$  es preciso copiar 2.3 MB de datos en la RAM de video para cada cuadro. En el caso de video con pleno movimiento, se requiere una tasa de por lo menos 25 cuadros/s, para una tasa de datos total de 57.6 MB/s. Esta carga es mucho más que lo que el bus (E)ISA puede manejar, de modo que las tarjetas de video de alto desempeño para IBM PC necesitan ser tarjetas PCI, y aun así se requieren importantes concesiones.

Un problema de desempeño relacionado es cómo desplazar la imagen en la pantalla (*scroll*). Un método sería copiar todos los bits en software, pero esto impone una carga gigantesca a la CPU. Es natural que muchas tarjetas de video estén equipadas con hardware especial para desplazar partes de la pantalla modificando registros base en vez de por copiado.

### Terminales RS-232-C

Docenas de compañías fabrican computadoras y muchas más fabrican terminales (sobre todo para mainframes). Con el fin de lograr que (casi) cualquier terminal pueda usarse con (casi) cualquier computadora, la Electronics Industries Association (EIA) creó una interfaz estándar computadora-terminal llamada **RS-232-C**. Cualquier terminal que reconozca la interfaz RS-232-C se puede conectar a cualquier computadora que también reconozca esa interfaz.

Las terminales RS-232-C tienen un conector estandarizado de 25 terminales. La norma RS-232-C define el tamaño y la forma físicos del conector, los niveles de voltaje y el significado de cada una de las señales de las agujas.

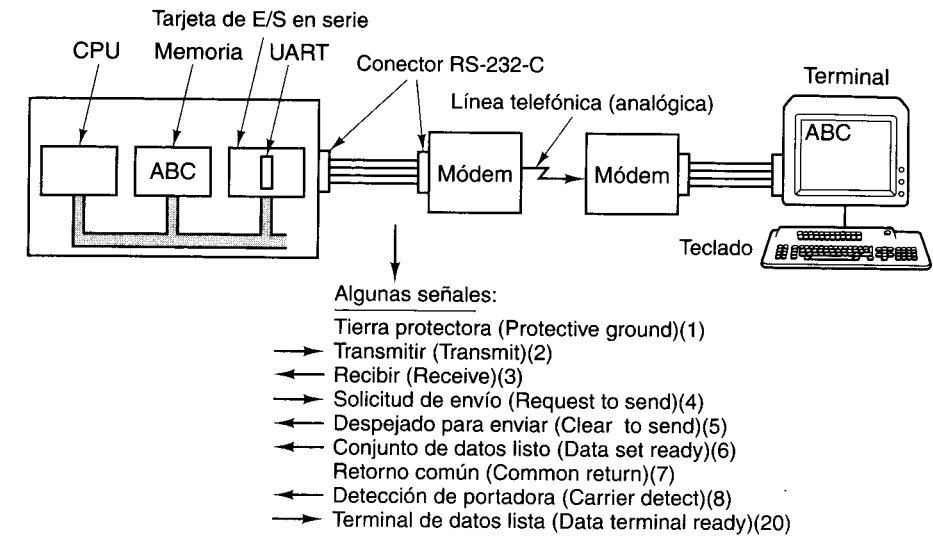
Cuando la computadora y la terminal están muy separadas, en muchos casos la única forma práctica de conectarlos es a través del sistema telefónico. Desafortunadamente, el sistema telefónico no puede transmitir las señales requeridas por la norma RS-232-C, y es necesario insertar un dispositivo llamado **módem** (**modulador-demodulador**) entre la computadora y el teléfono y también entre la terminal y el teléfono, a fin de efectuar la conversión de las señales. Estudiaremos los módems en breve.

La figura 2-34 muestra la colocación de la computadora, los módems y la terminal cuando se usa una línea telefónica. Si la terminal está lo bastante cerca de la computadora como para usar cableado directo, no se usan módems, pero sí los mismos conectores y cables RS-232-C, aunque no se necesiten las terminales relacionadas con el control del módem.

Para comunicarse, la computadora y la terminal contienen un chip llamado **UART** (**transmisor-receptor universal asincrónico**, *Universal Asynchronous Receiver Transmitter*), además de lógica para acceder al bus. Para exhibir un carácter, la computadora lo trae de su memoria principal y lo presenta al UART, que entonces lo mete en el cable RS-232-C bit por bit. De hecho, el UART es un convertidor de paralelo a serie, ya que recibe un carácter completo (1 byte) en una sola operación y produce los bits uno por uno con cierta rapidez. Además, el UART añade un bit de inicio y un bit de paro a cada carácter para delimitar el principio y el final del carácter (a 110 bps, se usan dos bits de paro).

En la terminal, otro UART recibe los bits y reconstruye el carácter, que luego se exhibe en la pantalla. Las entradas del teclado de la terminal se someten a una conversión paralelo-serie en la terminal y el UART las reensambla en la computadora.

La norma RS-232-C define casi 25 señales, pero en la práctica sólo se usan unas cuantas (y casi todas esas pueden omitirse cuando la terminal se conecta directamente a la computadora sin módems). Las terminales 2 y 3 son para transmitir y recibir datos, respectivamente. Cada terminal maneja un flujo de bits en un sentido. Cuando se enciende la terminal o computadora, habilita (es decir, pone en 1) la señal de (Lista Terminal de Datos) para informar al módem que está encendida. Por su parte, el módem habilita la señal de Data Set Ready para



**Figura 2-34.** Conexión de una terminal RS-232-C con una computadora. Los números entre paréntesis en la lista de señales son los números de terminal.

indicar su presencia. Cuando la terminal o computadora quiere enviar datos, habilita la señal Request to Send para pedir permiso. Si el módem está dispuesto a conceder ese permiso, habilita la señal Clear to Send como respuesta. Otras terminales tienen diversas funciones de estado, prueba y temporización.

### 2.4.3 Ratones

Conforme pasa el tiempo, personas con cada vez menos conocimientos acerca de cómo funcionan las computadoras están usando estas máquinas. Las computadoras de la generación ENIAC sólo eran utilizadas por quienes las habían construido. En los cincuenta, sólo programadores profesionales con alto nivel de capacitación usaban computadoras. Hoy día las computadoras son utilizadas ampliamente por personas que necesitan realizar alguna tarea y no saben (ni quieren saber) mucho acerca de cómo funcionan o cómo se programan las computadoras.

En el pasado, casi todas las computadoras tenían interfaces de línea de comandos, en las que los usuarios tecleaban comandos. Puesto que muchas personas no eran especialistas en computación percibían las interfaces de línea de comandos como poco amables hacia el usuario, por no decir hostiles, algunos fabricantes de computadoras crearon interfaces de apuntar y hacer clic, como las de Macintosh y Windows. Para usar este modelo se requiere una forma de apuntar a la pantalla, y la forma más común de hacerlo es con un ratón.

Un **ratón** (*mouse*) es una pequeña caja de plástico que descansa sobre la mesa junto al teclado. Cuando el ratón se mueve sobre la mesa, también se mueve un pequeño puntero en la

pantalla, y ello permite a los usuarios apuntar a elementos de la pantalla. El ratón tiene uno, dos o tres botones en la parte superior, que permiten a los usuarios seleccionar opciones de menús. Se ha derramado mucha sangre como resultado de discusiones acerca de cuántos botones debe tener un ratón. Los usuarios poco sofisticados prefieren uno (es difícil oprimir el botón equivocado si sólo hay uno), pero a los sofisticados les gusta la potencia de múltiples botones para hacer infinidad de cosas.

Se han producido tres tipos de ratones: mecánicos, ópticos y optomecánicos. Los primeros ratones tenían dos ruedas de caucho que sobresalían por abajo y tenían sus ejes perpendiculares entre sí. Cuando el ratón se movía paralelo a su eje principal, una rueda giraba. Cuando se movía perpendicularmente a su eje principal, giraba la otra rueda. Cada rueda impulsaba un resistor variable (potenciometro). Si se medían los cambios en la resistencia, era posible ver qué tanto había girado cada rueda y así calcular qué tanto se había movido el ratón en cada dirección. En años recientes este diseño ha sido reemplazado en buena medida por uno en el que se usa una esfera que sobresale ligeramente de la base, en lugar de ruedas. Éste se muestra en la figura 2-35.

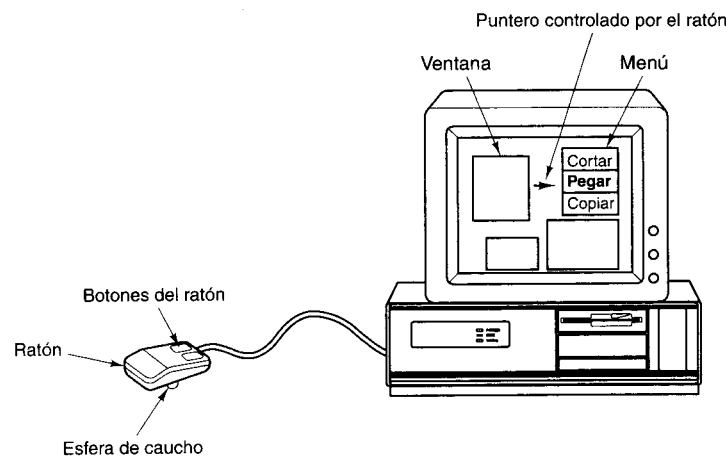


Figura 2-35. Uso del ratón para apuntar a opciones de un menú.

El segundo tipo de ratón es el óptico. Este tipo no tiene ruedas ni esfera. En vez de ello, tiene un **LED (diodo emisor de luz, Light Emitting Diode)** y un fotodetector en la base. El ratón óptico se utiliza sobre una base de plástico especial que contiene una cuadrícula de líneas muy juntas. A medida que el ratón se mueve sobre la cuadrícula, el fotodetector percibe el paso de las líneas por los cambios en la cantidad de luz del LED que se refleja. Unos circuitos internos del ratón cuentan el número de líneas de cuadrícula que se cruzan en cada dirección.

El tercer tipo de ratón es optomecánico. Al igual que el ratón mecánico más nuevo, tiene una esfera rodante que hace girar dos ejes perpendiculares entre sí. Los ejes están conectados

a codificadores provistos de ranuras a través de las cuales puede pasar la luz. A medida que el ratón se mueve, los ejes giran y pulsos de luz inciden sobre los detectores cada vez que una ranura pasa entre un LED y su detector. El número de pulsos detectados es proporcional a la cantidad de movimiento.

Aunque los ratones se pueden configurar de varias maneras, un esquema común es hacer que el ratón envíe una secuencia de 3 bytes a la computadora cada vez que se mueva cierta distancia mínima (por ejemplo, 0.01 de pulgada), llamado en ocasiones **mickey**. Por lo regular, dichos caracteres llegan por una línea serial, bit por bit. El primer byte contiene un entero con signo que indica cuántas unidades se movió el ratón en la dirección *x* en los últimos 100 ms. El segundo byte da la misma información para la dirección *y*. El tercer byte contiene la situación actual de los botones del ratón. A veces se usan dos bytes para cada coordenada.

Software de bajo nivel de la computadora acepta esta información conforme llega y convierte los movimientos relativos enviados por el ratón en una posición absoluta. Luego, el software exhibe una flecha en la pantalla en la posición correspondiente al ratón. Cuando la flecha apunta al elemento correcto, el usuario hace clic con un botón del ratón y la computadora puede determinar cuál elemento se seleccionó, porque sabe en qué parte de la pantalla está la flecha.

## 2.4.4 Impresoras

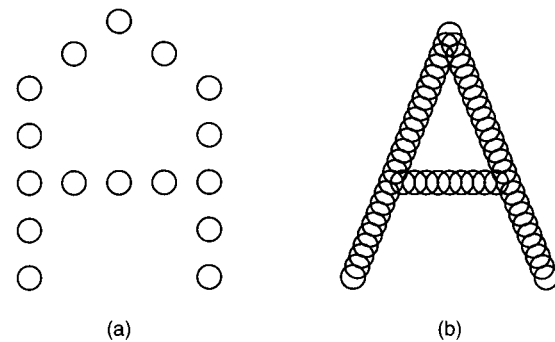
Después de preparar un documento o de bajar una página de la World Wide Web, es común que los usuarios quieran imprimirlos, así que todas las computadoras se pueden equipar con una impresora. En esta sección describiremos algunos de los tipos más comunes de impresoras monocromáticas (es decir, blanco y negro) y a color.

### Impresoras monocromáticas

El tipo de impresora más económico es la **impresora de matriz**, en la que una cabeza de impresión que contiene entre 7 y 24 agujas activables electromagnéticamente se mueve a lo largo de cada línea de impresión. Las impresoras de bajo costo tienen siete agujas para imprimir, digamos, 80 caracteres en una matriz de  $5 \times 7$  a lo largo de la línea. De hecho, la línea de impresión consiste en siete líneas horizontales, cada una de las cuales consta de  $5 \times 80 = 400$  puntos. Cada punto puede imprimirse o no, dependiendo de los caracteres que han sido seleccionados para tal efecto. En la figura 2-36(a) se ilustra la letra "A" impresa en una matriz de  $5 \times 7$ .

La calidad de impresión se puede incrementar empleando dos técnicas: usar más agujas, y hacer que los puntos se traslapen. En la figura 2-36(b) se muestra una "A" impresa con 24 agujas que producen puntos que se traslapan. Por lo regular se requieren varias pasadas sobre cada línea de barrido para producir puntos traslapados, por lo que un aumento en la calidad generalmente implica una tasa de impresión más lenta. Casi todas las impresoras de matriz pueden operar de varios modos, que ofrecen diferentes equilibrios entre calidad de impresión y rapidez.

Las impresoras de matriz son económicas (sobre todo en términos de consumibles) y muy confiables, pero son lentas, ruidosas y malas para imprimir gráficos. Estas impresoras tienen tres usos principales en los sistemas modernos. Primero, se usan mucho para imprimir



**Figura 2-36.** (a) La letra "A" en una matriz de  $5 \times 7$ . (b) La letra "A" impresa con 24 agujas que se traslapan.

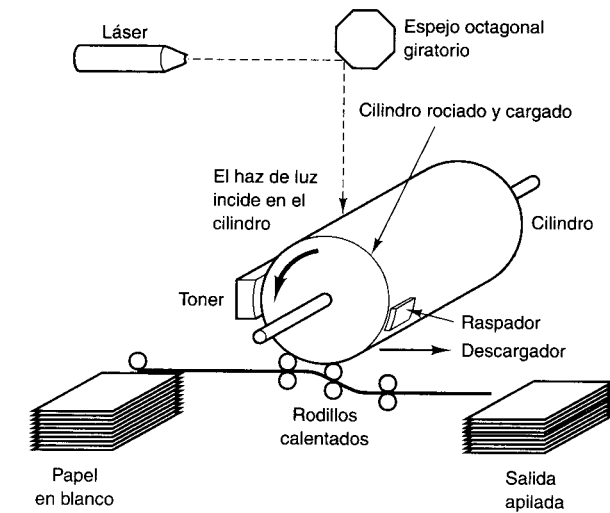
en formatos grandes ( $> 30$  cm) preimpresos. Segundo, son buenas para imprimir en pequeños trozos de papel, como los recibos de caja registradora, boletas de transacción de cajero automático o tarjeta de crédito, pases de abordar de líneas aéreas, etc. Tercero, generalmente son la tecnología más económica para imprimir en formas continuas de varias copias que tienen papel carbón intercalado.

Para la impresión casera de bajo costo, las **impresoras de inyección de tinta** son las favoritas. La cabeza de impresión móvil, que lleva un cartucho de tinta, se mueve horizontalmente a lo ancho del papel mientras rocía tinta con sus diminutas boquillas. Dentro de cada boquilla, una pequeña gota de tinta se calienta eléctricamente más allá de su punto de ebullición, hasta que hace explosión. La única dirección en que la tinta puede moverse es hacia la salida de la boquilla para chocar contra el papel. Luego la boquilla se enfría y el vacío que se produce succiona otra gotita de tinta. La rapidez de la impresora está limitada por la rapidez con que puede repetirse el ciclo de ebullición/enfriamiento. Las impresoras de inyección de tinta suelen tener definiciones de 300 **dpi** (**puntos por pulgada, dots per inch**) a 720 dpi, aunque también las hay de 1440 dpi. Estas impresoras son económicas, silenciosas y tienen buena calidad, pero también son lentas, usan cartuchos de tinta caros y producen impresiones saturadas de tinta.

Tal vez el avance más interesante en cuanto a impresión desde que Johann Gutenberg inventó los tipos móviles en el siglo xv, es la **impresora láser**. Este dispositivo combina una imagen de alta calidad, excelente flexibilidad, buena velocidad y costo moderado en un mismo periférico. Las impresoras láser emplean casi la misma tecnología que las fotocopadoras; de hecho, muchas compañías producen aparatos que combinan el copiado y la impresión (y a veces también un faccímil).

La tecnología básica se ilustra en la figura 2-37. El corazón de la impresora es un cilindro de precisión giratorio (o, en algunos sistemas del extremo superior, una banda). Al principio de cada ciclo de página, el cilindro se carga hasta cerca de 1000 volts y se recubre con un material fotosensible. Luego la luz de un láser se mueve a lo largo del cilindro de forma muy similar a como un haz de electrones se mueve en un CRT, sólo que en lugar de efectuar la desviación horizontal con la aplicación de un voltaje se usa un espejo octagonal giratorio

para barrer el cilindro a lo largo. El haz de luz se modula para producir un patrón de puntos claros y oscuros. Los puntos en los que el haz incide pierden su carga eléctrica.



**Figura 2-37.** Funcionamiento de una impresora láser.

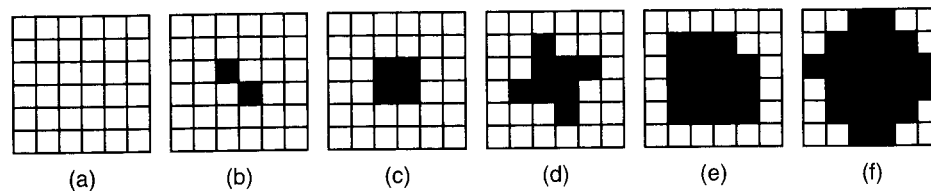
Una vez que se ha pintado una línea de puntos, el cilindro gira una fracción de grado para que pueda pintarse la siguiente línea. En algún momento, la primera línea de puntos llega al tóner, un depósito de polvo negro sensible a las cargas electrostáticas. El tóner es atraído hacia los puntos que todavía conservan su carga, y así forman una imagen visual de esa línea. Un poco más adelante en el trayecto de transporte, el cilindro recubierto con tóner se oprime contra el papel y transfiere a éste el polvo negro. Luego el papel pasa entre rodillos calientes que fusionan el tóner con el papel de forma permanente, lo que fija la imagen. Al continuar su rotación, el cilindro pierde su carga y un raspador elimina cualquier residuo de tóner, antes de que el cilindro se vuelva a cargar y recubrir para la siguiente página.

Sobra decir que este proceso es una combinación en extremo compleja de ciencia física, ciencia química, ingeniería mecánica e ingeniería óptica. No obstante, varios proveedores producen ensambles completos, llamados **máquinas de impresión**, que los fabricantes de impresoras láser combinan con sus propios circuitos electrónicos y software para crear una impresora completa. Los circuitos constan de una CPU y varios megabytes de memoria para contener el mapa de bits de una página completa y numerosos tipos de letra, algunos de los cuales son proporcionados con la impresora, mientras que otros se bajan de una fuente externa, como la computadora. Casi todas las impresoras aceptan comandos que describen las páginas a imprimir (en lugar de aceptar simplemente mapas de bits preparados por la CPU principal). Estos comandos se dan en lenguajes como PCL de HP y PostScript de Adobe.

Las impresoras láser de 600 dpi o mejores pueden imprimir fotografías en blanco y negro con una calidad razonable, pero la tecnología tiene sus bemoles. Considere una fotografía digitalizada a 600 dpi que se va a imprimir en una impresora de 600 dpi. La imagen digitalizada contiene  $600 \times 600$  píxeles/pulg<sup>2</sup>, cada uno de los cuales consiste en un valor de gris que va de 0

(blanco) a 255 (negro). La impresora también puede imprimir 600 dpi, pero cada pixel impreso es negro (tónér presente) o bien blanco (sin tónér). No es posible imprimir tonos de gris.

La solución usual para imprimir imágenes que tienen tonos de gris es utilizar **medios tonos**, como se hace en los carteles que se imprimen comercialmente. La imagen se divide en celdas de medio tono, cada una de las cuales contiene por lo regular  $6 \times 6$  píxeles. Cada celda puede contener entre 0 y 36 píxeles negros. El ojo percibe una celda con muchos píxeles como más oscura que una que tiene menos píxeles. Los valores de gris dentro del intervalo de 0 a 255 se representan dividiendo este intervalo en 37 zonas. Los valores del 0 al 6 están en la zona 0, los valores del 7 al 13 están en la zona 1, y así (la zona 36 es un poco más pequeña que las demás porque 256 no es divisible exactamente entre 37). Cada vez que se encuentra un valor de gris en la zona 0, su celda de medio tono en el papel se deja en blanco, como se ilustra en la figura 2-38(a). Un valor de zona 1 se imprime como un pixel negro. Un valor de zona 2 se imprime como dos píxeles negros, como se muestra en la figura 2-38(b). En la figura 2-38(c)-(f) se muestran otros valores de zona. Desde luego, si tomamos una fotografía digitalizada a 600 dpi y la convertimos en medios tonos con esta técnica, reduciremos la definición efectiva a 100 celdas/pulgada. Ésta se denomina **frecuencia de pantalla de medio tono** y por convención se mide en **lpi (líneas por pulgada)**.



**Figura 2-38.** Puntos de medio tono para diversos intervalos de escala de grises. (a) 0-6. (b) 14-20. (c) 28-34. (d) 56-62. (e) 105-111. (f) 161-167.

### Impresoras a color

Las imágenes a color pueden verse de dos maneras: como luz transmitida o como luz reflejada. Las imágenes de luz transmitida, como las que se producen en los monitores de CRT, se forman a partir de la superposición lineal de los tres colores primarios aditivos, rojo, verde y azul. Las imágenes de luz reflejada, como las fotografías a color y las ilustraciones de las revistas, absorben ciertas longitudes de onda de la luz y reflejan el resto. Estas imágenes se forman por la superposición lineal de los tres colores primarios sustractivos, turquesa o cian (se absorbe todo el rojo), amarillo (se absorbe todo el azul) y magenta (se absorbe todo el verde). En teoría, podemos producir cualquier color mezclando tinta turquesa, amarilla y magenta, pero en la práctica es difícil obtener tintas lo bastante puras como para que absorban toda la luz y produzcan un verdadero negro. Por esta razón, casi todos los sistemas de impresión a color emplean cuatro tintas: turquesa, amarilla, magenta y negra. Estos sistemas se llaman **impresoras CYMK** por las iniciales de los colores en inglés: *Cyan, Yellow, Magenta* y *black* (se usa la *K* y no la *B* para no confundir el *black* (negro) con *Blue*, azul). Los monitores, en cambio, usan luz transmitida y el sistema RGB para producir colores.

El conjunto total de colores que una pantalla o impresora puede producir es su **gama**. Ningún dispositivo tiene una gama que coincida con el mundo real, puesto que en el mejor de los casos cada color viene en 256 intensidades, lo que da sólo 16,777,216 colores discretos. Imperfecciones de la tecnología reducen aun más el total, y los colores restantes no siempre están distribuidos de manera uniforme en el espectro de color. Además, la percepción del color tiene mucho que ver con el funcionamiento de los bastones y los conos de la retina humana, no sólo con los aspectos físicos de la luz.

Una consecuencia de las observaciones anteriores es que convertir una imagen a color que se ve bien en la pantalla, en una imagen impresa idéntica, no es nada trivial. Algunos de los problemas son:

1. Los monitores a color usan luz transmitida; las impresoras a color usan luz reflejada.
2. Los CRT producen 256 intensidades por color; las impresoras a color deben emplear medios tonos.
3. Los monitores tienen un fondo oscuro; el papel tiene un fondo claro.
4. Las gamas RGB y CYMK son diferentes.

Lograr que las imágenes a color impresas coincidan con el mundo real (o siquiera con las imágenes en pantalla) requiere calibración del dispositivo, software avanzado y considerable experiencia por parte del usuario.

Existen cinco tecnologías de uso común para la impresión a color, todas ellas basadas en el sistema CYMK. En el extremo inferior están las impresoras de inyección de tinta a color. Éstas funcionan igual que las monocromáticas, pero con cuatro cartuchos de tinta (para C, Y, M y K) en lugar de uno. Estas impresoras producen buenos resultados con gráficos a color y resultados aceptables con fotografías, a un costo moderado (las impresoras no cuestan mucho, pero los cartuchos de tinta sí).

Para obtener resultados óptimos hay que usar tinta y papel especiales. Existen dos tipos de tinta. Las **tintas basadas en colorantes** que consisten en sustancias de coloración disueltas en un fluido portador. Éstas producen colores brillantes y fluyen fácilmente. Su desventaja principal es que se destiñen cuando se exponen a luz ultravioleta, como la contenida en la luz solar. Las **tintas basadas en pigmentos** contienen partículas sólidas de pigmento suspendidas en un fluido portador que se evapora del papel y deja atrás el pigmento. Éstas no se destiñen con el tiempo pero no son tan brillantes como las tintas basadas en colorantes y las partículas de pigmento tienden a taponar las boquillas, que requieren una limpieza periódica. Se necesita papel recubierto o satinado para imprimir fotografías. Estos tipos de papel están diseñados especialmente para retener las pequeñas gotas de tinta e impedir que se extiendan.

En el siguiente escalón arriba de las impresoras de inyección de tinta están las **impresoras de tinta sólida**. Éstas aceptan cuatro bloques sólidos de una tinta cerosa especial que se funden para introducirlos en depósitos de tinta caliente. Los tiempos de preparación de estas impresoras pueden llegar a los 10 minutos, mientras se funden los bloques de tinta. La tinta caliente se rocía sobre el papel, donde se solidifica y fusiona con el papel cuando éste se comprime entre dos rodillos duros.

El tercer tipo de impresora a color es la impresora láser a color. Ésta funciona como su pariente monocromático, excepto que se depositan imágenes C, Y, M y K individuales que se

transfieren a un rodillo utilizando cuatro tóners distintos. Puesto que generalmente se produce con anticipación el mapa de bits completo, una imagen de  $1200 \times 1200$  dpi para una página que contiene 80 pulgadas cuadradas requiere 115 millones de píxeles. Con 4 bits/píxel, la impresora necesita 55 MB sólo para el mapa de bits, sin contar la memoria para los procesadores internos, tipos de letra, etc. Este requisito hace que las impresoras láser a color sean costosas, pero la impresión es rápida, la calidad es alta y las imágenes son duraderas.

La cuarta clase de impresora a color es la **impresora de cera**. Ésta tiene una cinta ancha de cera de cuatro colores segmentada en bandas del tamaño de una página. Miles de elementos calefactores funden la cera a medida que el papel pasa debajo de ella. La cera se fusiona con el papel en forma de píxeles empleando el sistema CYMK. Las impresoras de cera solían ser la principal tecnología de impresión a color, pero están siendo sustituidas por las otras clases, cuyos consumibles son más económicos.

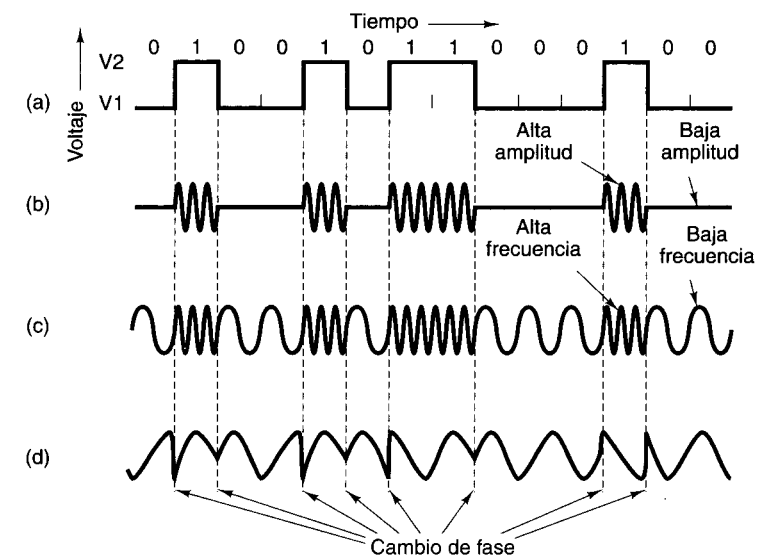
El quinto tipo de impresora a color es la **impresora de sublimación de colorante**. Aunque la palabra sublimación tiene connotaciones freudianas, es el término científico que describe la conversión de un sólido en gas sin pasar por el estado líquido. El hielo seco (dióxido de carbono congelado) es un material conocido que se sublima. En una impresora de sublimación de colorante, un portador que contiene los colorantes CYMK pasa por una cabeza de impresión térmica que contiene cientos de elementos calefactores programables. Los colorantes se vaporizan instantáneamente y son absorbidos por un papel especial que está cerca. Cada elemento calefactor puede producir 256 temperaturas distintas. Cuanto más alta es la temperatura, más colorante se deposita y más intenso es el color. A diferencia de las demás impresoras a color, es posible obtener colores casi continuos para cada píxel, de modo que no se requieren medios tonos. Casi todas las impresoras de fotografías instantáneas emplean el proceso de sublimación de colorantes para producir imágenes fotográficas altamente realistas en papel especial (y caro).

### 2.4.5 Módems

Con el crecimiento del uso de las computadoras en los últimos años, es común que una computadora necesite comunicarse con otra. Por ejemplo, mucha gente tiene una computadora personal en casa que usa para comunicarse con su computadora en el trabajo, con un proveedor de servicio de Internet o con un sistema de "banco en su casa". Todas estas aplicaciones utilizan el teléfono para efectuar la comunicación.

Sin embargo, una línea telefónica ordinaria no es apropiada para transmitir señales de computadora, que generalmente representan un 0 como 0 volts y un 1 como 3 a 5 volts, como se muestra en la figura 2-39(a). Las señales de dos niveles sufren una distorsión considerable cuando se transmiten por una línea telefónica de grado de voz, y esto da pie a errores de transmisión. En cambio, una señal de onda senoidal pura con una frecuencia de 1000 a 2000 Hz, llamada **portadora**, se puede transmitir con relativamente poca distorsión, y este hecho se aprovecha en casi todos los sistemas de telecomunicaciones.

Puesto que las pulsaciones de una onda senoidal son totalmente predecibles, una onda senoidal pura no transmite información. Sin embargo, si se varía la amplitud, la frecuencia o la fase, se puede transmitir una secuencia de unos y ceros, como se muestra en la figura 2-39.



**Figura 2-39.** Transmisión del número binario 01001011000100 por una línea telefónica bit por bit. (a) Señal de dos niveles. (b) Modulación de amplitud. (c) Modulación de frecuencia. (d) Modulación de fase.

Este proceso se llama **modulación**. En la **modulación de amplitud** [vea la figura 2-39(b)] se usan dos niveles de voltaje distintos para el 0 y para el 1. Una persona que escuchara datos digitales transmitidos con una tasa de datos muy baja escucharía un ruido fuerte cuando se transmite un 1 y silencio cuando se transmite un 0.

En la **modulación de frecuencia** [vea la figura 2-39(c)] el nivel de voltaje es constante pero la frecuencia de la portadora es diferente para el 1 y para el 0. Una persona que escuchara datos digitales de frecuencia modulada escucharía dos tonos, uno para el 0 y otro para el 1. Esta técnica también se conoce como **modulación por desplazamiento de frecuencia** (FSK, *Frequency Shift Keying*).

En la **modulación de fase** simple [vea la figura 2-39(d)], la amplitud y la frecuencia no cambian, pero la fase de la portadora se invierte 180 grados cuando los datos cambian de 0 a 1 o de 1 a 0. En los sistemas de modulación de fase más avanzados, al principio de cada intervalo de tiempo indivisible la fase de la portadora se desplaza abruptamente 45, 135, 225 o 315 grados, lo que permite transmitir dos bits en cada intervalo de tiempo. Esto se conoce como codificación por fase **dibit**. Por ejemplo, un cambio de fase de 45 grados podría representar 00 y uno de 135 grados, 01. También existen otros esquemas para transmitir tres o más bits por intervalo de tiempo. El número de intervalos (es decir, el número de posibles cambios de señal por segundo) se llama tasa de **bauds** (*baud rate*). Con dos o más bits por intervalo, la tasa de bits excederá la tasa de bauds. Muchas personas confunden estos dos términos.

Si los datos por transmitir consisten en una serie de caracteres de ocho bits, sería deseable tener una conexión capaz de transmitir ocho bits simultáneamente, es decir, ocho pares de hilos. Puesto que las líneas telefónicas de grado de voz sólo ofrecen un canal, los bits se



deben enviar en serie, uno tras otro (o en grupos de dos si se está empleando codificación dibit). El dispositivo que acepta caracteres de una computadora en forma de señales de dos niveles, bit por bit, y transmite los bits en grupos de uno o dos, en forma de amplitud modulada, frecuencia modulada o fase modulada, es el módem. Para marcar el principio y el final de cada carácter, un carácter de ocho bits normalmente se envía precedido por un bit de inicio y seguido de un bit de paro, para un total de 10 bits.

El módem transmisor envía los bits individuales de un carácter a intervalos de tiempo equidistantes. Por ejemplo, 9600 bauds implica un cambio de señal cada 104  $\mu$ s. Un segundo módem en el extremo receptor convierte una portadora modulada en un número binario. Puesto que los bits llegan al receptor a intervalos regulares, una vez que el módem receptor ha determinado el inicio del carácter su reloj le dice cuándo debe muestrear la línea para leer los bits que llegan.

Los módems modernos operan con tasas de datos entre 28,800 bits/s (bps) y 57,600 bits/s, y normalmente a tasas de bauds mucho más bajas. Emplean una combinación de técnicas para enviar varios bits por baud, modulando la amplitud, la frecuencia y la fase. Casi todos ellos son **dúplex (full duplex)**, lo que implica que pueden transmitir en ambas direcciones al mismo tiempo (empleando diferentes frecuencias). Los módems o líneas de transmisión que sólo pueden transmitir en una dirección a la vez (como una vía férrea sencilla que puede manejar trenes que van al norte o trenes que van al sur, pero no al mismo tiempo) se llaman **semidúplex (half duplex)**. Las líneas que sólo pueden transmitir en una dirección son **simplex**.

## ISDN

A principios de la década de los ochenta las PTT europeas (administraciones de correo, teléfono y telégrafo) crearon un estándar para telefonía digital llamado **ISDN (red digital de servicios integrados, Integrated Services Digital Network)**. La intención original era que las personas de edad avanzada pudieran tener alarmas en sus casas conectadas a oficinas de monitoreo centrales, y muchas otras aplicaciones extrañas que nunca se hicieron realidad. Ellas promovieron intensamente la idea, pero sin mucho éxito. Luego, la World Wide Web apareció de la noche a la mañana y la gente estaba exigiendo a gritos acceso digital con gran ancho de banda a Internet. Así, de repente, ISDN había descubierto una superaplicación (aunque no había sido contemplada por sus diseñadores). La ISDN también se popularizó en Estados Unidos y otros países.

Cuando un cliente de una compañía telefónica se suscribe a ISDN, se sustituye la antigua línea analógica por una digital. (De hecho, no se cambia la línea misma, sólo el equipo en sus extremos.) La nueva línea comprende dos canales digitales independientes de 64,000 bits/s cada uno, más un canal de señalización de 16,000 bits/s. Existen equipos que combinan los tres canales en un solo canal digital de 144,000 bps. Para las empresas está disponible una línea ISDN de 30 canales.

ISDN no sólo es más rápida que un canal analógico, sino que también permite establecer conexiones en un tiempo que normalmente no es mayor que un segundo, ya no requiere un módem analógico, y es mucho más confiable (menos errores) que una línea analógica. Además, cuenta con varias funciones y opciones adicionales que las líneas analógicas no siempre ofrecen.

En la figura 2-40 se muestra la estructura de una conexión ISDN. Lo que la compañía portadora proporciona es un conducto de bits digital que se limita a transportar bits. El significado de los bits es cuestión del transmisor y del receptor. La interfaz entre el equipo del cliente y el equipo de la portadora es el dispositivo NT1, con la interfaz T en un lado y la interfaz U en el otro. En Estados Unidos los clientes deben comprar su propio dispositivo NT1. En muchos países europeos deben rentarlo a la compañía portadora.

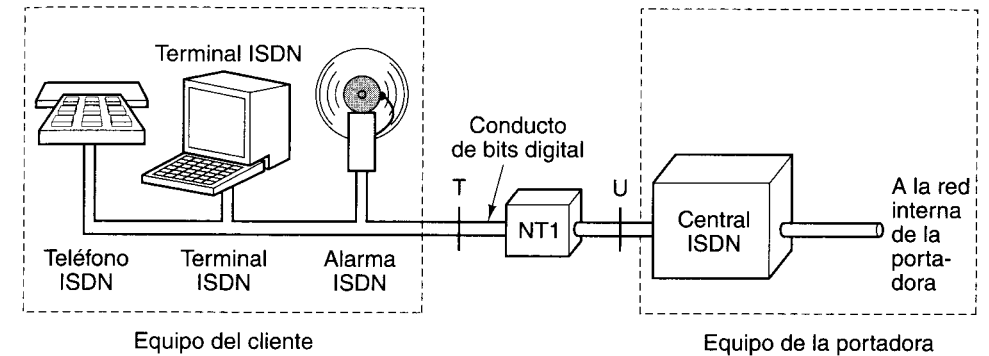


Figura 2-40. ISDN para uso casero.

## 2.4.6 Códigos de caracteres

Cada computadora usa un conjunto de caracteres. Como mínimo, este conjunto incluye las 26 letras mayúsculas del inglés, las 26 letras minúsculas, los dígitos del 0 al 9 y un conjunto de símbolos especiales, como espacio, punto, signo menos, coma y retorno de carro.

Para transferir estos caracteres a la computadora, se asigna un número a cada uno: por ejemplo, a = 1, b = 2, ..., z = 26, + = 27, - = 28. La correspondencia entre caracteres y enteros se llama **código de caracteres**. Es indispensable que dos computadoras que se comunican empleen el mismo código, pues de lo contrario no podrán entenderse. Para ello se han creado normas, y a continuación examinaremos dos de las más importantes.

## ASCII

Un código de amplio uso se llama **ASCII (Código Estándar Americano para Intercambio de Información, American Standard Code for Information Interchange)**. Cada carácter ASCII tiene 7 bits, lo que permite representar un total de 128 caracteres. La figura 2-41 muestra el código ASCII. Los códigos 0 a 1F (hexadecimal) son caracteres de control y no se imprimen.

Muchos de los caracteres de control ASCII desempeñan funciones durante la transmisión de datos. Por ejemplo, un mensaje podría consistir en un carácter SOH (*Start of Header*, inicio de cabecera), una cabecera, un carácter STX (*Start of Text*, inicio de texto), el texto

| Hex | Nombre | Significado           | Hex | Nombre | Significado                   |
|-----|--------|-----------------------|-----|--------|-------------------------------|
| 0   | NUL    | Nulo                  | 10  | DLE    | Escape de enlace de datos     |
| 1   | SON    | Inicio de cabecera    | 11  | DC1    | Control de dispositivo 1      |
| 2   | STX    | Inicio de texto       | 12  | DC2    | Control de dispositivo 2      |
| 3   | ETX    | Fin de texto          | 13  | DC3    | Control de dispositivo 3      |
| 4   | EOT    | Fin de transmisión    | 14  | DC4    | Control de dispositivo 4      |
| 5   | ENQ    | Pregunta              | 15  | NAK    | Reconocimiento negativo       |
| 6   | ACK    | Reconocimiento        | 16  | SYN    | Tiempo muerto sincrónico      |
| 7   | BEL    | Alarma                | 17  | ETB    | Fin de bloqueo de transmisión |
| 8   | BS     | Retroceso             | 18  | CAN    | Cancelar                      |
| 9   | HT     | Tabulación horizontal | 19  | EM     | Fin de medio                  |
| A   | LF     | Salto de línea        | 1A  | SUB    | Sustituto                     |
| B   | VT     | Tabulación vertical   | 1B  | ESC    | Escape                        |
| C   | FF     | Salto de página       | 1C  | FS     | Separador de archivos         |
| D   | CR     | Retorno de carro      | 1D  | GS     | Separador de grupos           |
| E   | SO     | Desactivar cambio     | 1E  | RS     | Separador de registros        |
| F   | SI     | Activar cambio        | 1F  | US     | Separador de unidades         |

| Hex | Car       | Hex | Car | Hex | Car | Hex | Car | Hex | Car | Hex | Car |
|-----|-----------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 20  | (Espacio) | 30  | 0   | 40  | @   | 50  | P   | 60  | .   | 70  | p   |
| 21  | !         | 31  | 1   | 41  | A   | 51  | Q   | 61  | a   | 71  | q   |
| 22  | "         | 32  | 2   | 42  | B   | 52  | R   | 62  | b   | 72  | r   |
| 23  | #         | 33  | 3   | 43  | C   | 53  | S   | 63  | c   | 73  | s   |
| 24  | \$        | 34  | 4   | 44  | D   | 54  | T   | 64  | d   | 74  | t   |
| 25  | %         | 35  | 5   | 45  | E   | 55  | U   | 65  | e   | 75  | u   |
| 26  | &         | 36  | 6   | 46  | F   | 56  | V   | 66  | f   | 76  | v   |
| 27  | '         | 37  | 7   | 47  | G   | 57  | W   | 67  | g   | 77  | w   |
| 28  | (         | 38  | 8   | 48  | H   | 58  | X   | 68  | h   | 78  | x   |
| 29  | )         | 39  | 9   | 49  | I   | 59  | Y   | 69  | i   | 79  | y   |
| 2A  | *         | 3A  | :   | 4A  | J   | 5A  | Z   | 6A  | j   | 7A  | z   |
| 2B  | +         | 3B  | ;   | 4B  | K   | 5B  | [   | 6B  | k   | 7B  | {   |
| 2C  | ,         | 3C  | <   | 4C  | L   | 5C  | \   | 6C  | l   | 7C  |     |
| 2D  | -         | 3D  | =   | 4D  | M   | 5D  | ]   | 6D  | m   | 7D  | }   |
| 2E  | .         | 3E  | >   | 4E  | N   | 5E  | ^   | 6E  | n   | 7E  | ~   |
| 2F  | /         | 3F  | ?   | 4F  | O   | 5F  | -   | 6F  | o   | 7F  | DEL |

Figura 2-41. El conjunto de caracteres ASCII.

mismo, un carácter ETX (*End of Text*, fin de texto) y luego un carácter EOT (*End of Transmission*, fin de transmisión). En la práctica los mensajes que se envían por líneas telefónicas y redes tienen un formato muy diferente, y es por ello que ya casi no se usan los códigos de control de transmisión ASCII.

Los caracteres imprimibles ASCII no tienen mayor ciencia. Incluyen las letras mayúsculas y minúsculas, los dígitos, signos de puntuación y unos cuantos símbolos matemáticos.

## UNICODE

La industria de las computadoras creció principalmente en Estados Unidos, y esto dio lugar al conjunto de caracteres ASCII. El ASCII está bien para el inglés pero no tanto para otros idiomas. El francés necesita acentos (por ejemplo, *systeme*); el alemán necesita signos diacríticos (por ejemplo, *für*), etc. Algunos lenguajes europeos tienen unas cuantas letras que no están incluidas en ASCII, como la ß alemana y la ø danesa. Algunos idiomas tienen alfabetos totalmente distintos (como el ruso y el árabe), y unos cuantos idiomas ni siquiera tienen un alfabeto (como el chino). A medida que las computadoras se propagaron a todos los rincones del mundo y los proveedores de software comenzaron a querer vender productos en países en los que la mayoría de los usuarios no habla inglés, se hizo necesario un conjunto de caracteres distinto.

El primer intento de extender ASCII fue IS 646, que añadió otros 128 caracteres a ASCII para convertirlo en un código de ocho bits llamado **Latin-1**. Los caracteres adicionales eran en su mayor parte letras latinas con acentos y signos diacríticos. El siguiente intento fue IS 8859, que introdujo el concepto de **página de código**, un conjunto de 256 caracteres para un idioma o grupo de idiomas específico. IS 8859-1 es Latin-1. IS 8859-2 maneja los idiomas eslavos basados en el latín (como el checo, el polaco y el húngaro). IS 8859-3 contiene los caracteres necesarios para el turco, maltés, esperanto y gallego, etc. El problema con el enfoque de página de código es que el software tiene que saber en qué página está, es imposible combinar idiomas de diferentes páginas, y el esquema no cubre el chino ni el japonés.

Un grupo de compañías de computación decidió resolver este problema formando un consorcio para crear un sistema nuevo, llamado **UNICODE**, y hacer que se proclamara como Norma Internacional (IS 10646). Actualmente varios lenguajes de programación (como Java), algunos sistemas operativos (como Windows NT) y muchas aplicaciones reconocen UNICODE. Es probable que la aceptación de este sistema aumente cada vez más a medida que la industria de la computación se vuelve global.

La idea básica en que se basa UNICODE es asignar a cada carácter y símbolo un valor único y permanente de 16 bits, llamado **punto de código**. No se usan caracteres de múltiples bytes ni secuencias de escape. Al hacer que cada símbolo tenga 16 bits la escritura de software se simplifica mucho.

Con símbolos de 16 bits, UNICODE tiene 65,356 puntos de código. Puesto que los idiomas del mundo emplean colectivamente unos 200,000 símbolos, los puntos de código son un recurso escaso que debe asignarse con mucho cuidado. Ya se ha asignado cerca de la mitad de los puntos de código, y el consorcio UNICODE examina continuamente propuestas para asignar el resto. A fin de acelerar la aceptación de UNICODE, el consorcio utilizó sagazmente Latin-1 para los puntos de código del 0 al 255, lo que facilita la conversión entre ASCII y UNICODE.

Para evitar el desperdicio de puntos de código, cada signo diacrítico tiene su propio punto de código, y corresponde al software combinar los signos diacríticos con sus vecinas para formar nuevos caracteres.

El espacio de puntos de código se divide en bloques, cada uno de los cuales es un múltiplo de 16 puntos de código. Cada alfabeto principal en UNICODE tiene una secuencia de zonas consecutivas. Algunos ejemplos (y el número de códigos de punto asignados) son latín (336), griego (144), cirílico (256), armenio (96), hebreo (112), devanagari (128), gurmukhi (128), oriya (128), telugu (128) y kannada (128). Cabe señalar que a cada uno de estos idiomas se les ha asignado más puntos de códigos que las letras que tienen. Esta decisión se tomó en parte porque muchos idiomas tienen varias formas para cada letra. Por ejemplo, cada letra del español tiene dos formas, minúscula y MAYÚSCULA. Algunos idiomas tienen tres o más formas, posiblemente dependiendo de si la letra está al principio, en medio o al final de una palabra.

Además de estos alfabetos, se han asignado puntos de código a signos diacríticos (112) y de puntuación (112), subíndices y superíndices (48), símbolos de divisas (48) y matemáticos (256), formas geométricas (96) y ornamentos tipográficos (192).

Después de éstos vienen los símbolos requeridos para el chino, japonés y coreano. Primero hay 1024 símbolos fonéticos (como katakana y bopomofo) y luego los ideogramas Han unificados (20,992) empleados en chino y japonés, y las sílabas Hangul coreanas (11,156).

A fin de que los usuarios puedan inventar caracteres especiales para propósitos específicos, se han asignado 6400 puntos de código para uso local.

Si bien UNICODE resuelve muchos problemas relativos a la internacionalización, no resuelve todos los problemas del mundo (ni intenta hacerlo). Por ejemplo, si bien el alfabeto latino está en orden, los ideogramas Han no están en el orden del diccionario; por consiguiente, un programa en español puede examinar “gato” y “perro” y ordenarlos alfabéticamente con sólo comparar el valor UNICODE de su primer carácter. Un programa japonés necesita tablas externas para determinar cuál de dos símbolos precede al otro en el diccionario.

Otro problema es que continuamente aparecen palabras nuevas. Hace 50 años nadie hablaba de applets, ciberespacio, gigabytes, láseres, módems o videocintas. La adición de palabras nuevas al español no requiere nuevos puntos de código, pero añadirlas al japonés sí. Además de nuevas palabras técnicas, existe demanda para añadir al menos 20,000 nombres personales y de lugares nuevos (casi todos chinos). Los ciegos piensan que el Braille debe estar incluido, y todo tipo de grupos con intereses especiales quieren los puntos de código que, según ellos, por derecho les corresponden. El consorcio UNICODE estudia todas las nuevas propuestas y emite un dictamen.

UNICODE emplea el mismo punto de código para caracteres que son casi idénticos pero tienen diferente significado, o que chinos y japoneses escriben de forma ligeramente distinta (como si un procesador de textos en español siempre escribiera “caza” como “casa”, porque suenan igual). Algunas personas ven esto como una optimización para ahorrar valiosos puntos de código; otras creen que es una manifestación del imperialismo cultural anglosajón (y usted que creía que asignar valores de 16 bits a los caracteres nada tenía que ver con la política). Para empeorar las cosas, un diccionario japonés completo tiene 50,000 kanji (sin incluir nombres), así que con sólo 20,992 puntos de código disponibles para los ideogramas Han, fue preciso tomar decisiones. No todos los japoneses creen que un consorcio de compañías de computación, aunque algunas de ellas sean japonesas, sea el foro ideal para tomar esas decisiones.

## 2.5 RESUMEN

Los sistemas de computación se construyen con tres tipos de componentes: procesadores, memorias y dispositivos de E/S. La tarea de un procesador es buscar instrucciones una por una en una memoria, decodificarlas y ejecutarlas. El ciclo de búsqueda-decodificación-ejecución siempre puede describirse como un algoritmo y, de hecho, a veces es ejecutado por un intérprete en software que se ejecuta en un nivel más bajo. A fin de mejorar la velocidad, muchas computadoras actuales tienen uno o más filas de procesamiento, o tienen un diseño superescalar con varias unidades funcionales que operan en paralelo.

Cada vez son más comunes los sistemas con varios procesadores. Las computadoras paralelas incluyen los procesadores de arreglos, en los que la misma operación se ejecuta con varios conjuntos de datos al mismo tiempo, los multiprocesadores, en los que varias CPU comparten una misma memoria, y las multicomputadoras, en las que varias computadoras, cada una con su propia memoria, se comunican mediante el envío de mensajes.

Las memorias pueden clasificarse como primarias o secundarias. La memoria primaria se usa para contener el programa que se está ejecutando. Su tiempo de acceso es corto –unas cuantas decenas de nanosegundos como máximo– e independiente de la dirección a la que se está accediendo. Las cachés reducen este tiempo de acceso aún más. Algunas memorias están equipadas con códigos de corrección de errores para mejorar la confiabilidad.

Las memorias secundarias, en contraste, tienen tiempos de acceso mucho más largos (milisegundos o más) y que dependen de la ubicación de los datos que se están leyendo o escribiendo. Las cintas, discos magnéticos y discos ópticos son las memorias secundarias más comunes. Hay muchas variedades de discos magnéticos: discos flexibles, discos Winchester, discos IDE, discos SCSI y RAID. Los discos ópticos incluyen los CD-ROM, los CD-R y los DVD.

Los dispositivos de E/S sirven para transferir información de y hacia la computadora, y están conectados al procesador y a la memoria con uno o más buses. Como ejemplos podemos citar las terminales, los ratones, las impresoras y los módems. Casi todos los dispositivos de E/S utilizan el código de caracteres ASCII, aunque UNICODE está adquiriendo cada vez más aceptación a medida que la industria de las computadoras se vuelve global.

## PROBLEMAS

1. Considere el funcionamiento de una máquina con el camino de datos de la figura 2-2. Suponga que cargar los registros de entrada de la ALU toma 5 ns, para ejecutar la operación la ALU toma 10 ns y almacenar el resultado en un registro toma 5 ns. ¿Qué número máximo de MIPS podrá ejecutar esta máquina si no hay fila de procesamiento?
2. ¿Para qué sirve el paso 2 de la lista de la sección 2.1.2? ¿Qué sucedería si se omitiera este paso?
3. En la computadora 1, todas las instrucciones tardan 10 ns en ejecutarse. En la computadora 2, todas tardan 5 ns. ¿Puede decir con certeza que la computadora 2 es más rápida? Explique.

4. Suponga que está diseñando una computadora de un solo chip que se usará en sistemas incorporados. El chip tendrá toda su memoria en el chip y ésta operará a la misma velocidad que la CPU sin retraso por acceso. Examine cada uno de los principios vistos en la sección 2.1.4 y diga si siguen siendo tan importantes (suponiendo que se desea un buen desempeño).
5. ¿Sería posible añadir cachés a los procesadores de la figura 2-8(b)? Si es posible, ¿qué problema habría que resolver primero?
6. Cierta operación es altamente secuencial; es decir, cada paso depende del que lo precede. ¿Qué sería más apropiado para este cálculo, un arreglo de procesadores o un procesador con fila de procesamiento? Explique.
7. Para competir con el reciente invento de la imprenta, cierto monasterio medieval decidió producir en masa libros de bolsillo escritos a mano juntando un gran número de escribanos en un gran salón. El monje director gritaba la primera palabra del libro a producir y todos los escribanos la copiaban. Luego el monje director gritaba la segunda palabra y todos los escribanos la copiaban. Este proceso se repetía hasta que se había leído en voz alta y copiado todo el libro. ¿A cuál de los sistemas de procesador paralelo que vimos en la sección 2.1.6 se parece más este sistema?
8. Conforme bajamos por la jerarquía de memoria de cinco niveles que vimos en el texto, el tiempo de acceso aumenta. Haga una conjetura razonable de la relación entre el tiempo de acceso de un disco óptico y el de la memoria de registros. Suponga que el disco ya está en línea.
9. Calcule la tasa de datos del ojo humano con base en la información siguiente: El campo visual consiste en unos  $10^6$  elementos (píxeles). Cada píxel puede reducirse a una superposición de los tres colores primarios, cada uno de los cuales tiene 64 intensidades. La definición temporal es de 100 ms.
10. La información genética de todos los seres vivos se codifica en moléculas de DNA. Una molécula de DNA es una secuencia lineal de los cuatro nucleótidos básicos: A, C, G y T. El genoma humano contiene aproximadamente  $3 \times 10^9$  nucleótidos en forma de unos 100,000 genes. ¿Qué capacidad de información total (en bits) tiene el genoma humano? ¿Qué capacidad de información (en bits) tiene un gen típico?
11. ¿Cuáles de las siguientes memorias son posibles? ¿Cuáles son razonables? explique.
  - a. Dirección de 10 bits, 1024 celdas, tamaño de celda de 8 bits
  - b. Dirección de 10 bits, 1024 celdas, tamaño de celda de 12 bits
  - c. Dirección de 9 bits, 1024 celdas, tamaño de celda de 10 bits
  - d. Dirección de 11 bits, 1024 celdas, tamaño de celda de 10 bits
  - e. Dirección de 10 bits, 10 celdas, tamaño de celda de 1024 bits
  - f. Dirección de 1024 bits, 10 celdas, tamaño de celda de 10 bits
12. Los sociólogos pueden obtener tres posibles respuestas a una pregunta de encuesta típica como “¿Cree usted en el Ratoncito Pérez?”: sí, no, y no tengo opinión al respecto. Con esto en mente, la Compañía de Computadoras Sociomagnéticas decidió construir una computadora para procesar datos de encuestas. Esta computadora tiene una memoria trinaría; es decir, cada byte (¿tryte?) consiste en 8 trits, y un trit contiene un 0, un 1 o un 2. ¿Cuántos trits se necesitan para almacenar un número de 6 bits? Dé una expresión para el número de trits necesarios para almacenar  $n$  bits.

13. Cierta computadora puede equiparse con 268,435,456 bytes de memoria. ¿Por qué habría de escoger un fabricante un número tan extraño, en lugar de un número fácil de recordar como 250,000,000?
14. Derive un código Hamming de paridad par para los dígitos del 0 al 9.
15. Derive un código para los dígitos 0 al 9 cuya distancia de Hamming sea 2.
16. En un código de Hamming, algunos bits se “desperdician” en el sentido de que se usan para verificación y no para información. ¿Cuál es el porcentaje de bits desperdiciados en mensajes cuya longitud total (datos + bits de verificación) es  $2^n - 1$ ? Evalúe esta expresión numéricamente para valores de  $n$  de 3 a 10.
17. Los errores de transmisión en las líneas telefónicas a menudo ocurren en ráfagas (muchos bits consecutivos con error) en lugar de individualmente. Puesto que el código de Hamming básico sólo puede corregir errores individuales dentro de un carácter, no sirve de nada si una ráfaga de ruido hace que  $n$  bits consecutivos sean erróneos. Diseñe un método para transmitir texto ASCII por una línea en la que las ráfagas de ruido pudieran hacer que hasta 100 bits consecutivos fueran erróneos. Suponga que el intervalo mínimo entre dos ráfagas de ruido es de miles de caracteres. *Sugerencia:* piense cuidadosamente en el orden de transmisión de los bits.
18. ¿Cuánto tiempo toma leer un disco que tiene 800 cilindros, cada uno de los cuales contiene cinco pistas de 32 sectores? Primero deben leerse todos los sectores de la pista 0, comenzando con el sector 0, luego todos los sectores de la pista 1 comenzando con el sector 0, y así sucesivamente. El tiempo de rotación es de 20 ms, y una búsqueda tarda 10 ms entre cilindros adyacentes y 50 ms en el peor de los casos. La conmutación entre las pistas de un cilindro puede ser instantánea.
19. El disco que se ilustra en la figura 2-19 tiene 64 sectores/pista y una tasa de rotación de 7200 rpm. Calcule la tasa de transferencia sostenida del disco a lo largo de una pista.
20. Una computadora tiene un bus con un tiempo de ciclo de 25 ns, durante el cual puede leer o escribir una palabra de 32 bits de la memoria. La computadora tiene un disco Ultra-SCSI que utiliza el bus y opera a 40 MB/s. La CPU normalmente busca y ejecuta una instrucción de 32 bits cada 25 ns. ¿Qué tanto retrasa el disco a la CPU?
21. Imagine que está escribiendo la parte de administración de disco de un sistema operativo. Lógicamente, usted representa el disco como una secuencia de bloques, del 0 en la parte interior hasta algún valor máximo en la exterior. A medida que se crean archivos, usted tiene que asignar sectores libres, y podría hacerlo desde el interior hacia afuera o desde el exterior hacia adentro. ¿Importa cuál estrategia escoja? Explique su respuesta.
22. El direccionamiento LBA emplea 24 bits para direccionar un sector. ¿Cuál es el disco más grande que puede manejar?
23. RAID nivel 3 puede corregir errores de un solo bit utilizando sólo una unidad de disco de paridad. ¿De qué sirve RAID nivel 2? Después de todo, sólo puede corregir un error también, y necesita más unidades de disco para hacerlo.
24. ¿Qué capacidad exacta (en bytes) tiene un CD-ROM modo 2 que contiene los 74 minutos de datos estándar?
25. Para quemar un CD-R, el láser debe emitir pulsos a alta velocidad. Al operar a velocidad 4x en el modo 1, ¿qué duración tiene un pulso en nanosegundos?
26. Para almacenar 133 minutos de video en un DVD de un solo lado y una sola capa se requiere una cantidad apreciable de compresión. Calcule el factor de compresión requerido. Suponga que se

dispone de 3.5 GB de espacio para la pista de video, que la definición de imagen es de  $720 \times 480$  pixeles con color de 24 bits, y que las imágenes se exhiben a razón de 30 cuadros/s.

27. La tasa de transferencia entre una CPU y su memoria es varios órdenes de magnitud mayor que la tasa de transferencia de E/S mecánica. ¿Cómo puede esta diferencia causar ineficiencias? ¿Cómo puede aliviarse?
28. Una terminal de mapa de bits tiene una pantalla de  $1024 \times 768$ . La pantalla se redibuja 75 veces cada segundo. ¿Qué duración tiene el pulso correspondiente a un pixel?
29. Un fabricante anuncia que su terminal a color de mapa de bits puede exhibir  $2^{24}$  colores distintos. Sin embargo, el hardware sólo tiene un byte para cada pixel. ¿Cómo puede hacerse esto?
30. Con cierto tipo de letra, una impresora láser monocromática puede imprimir 50 líneas de 80 caracteres por página. Un carácter típico ocupa un cuadro de  $2 \text{ mm} \times 2 \text{ mm}$ , y cerca del 25% de esta área es tóner. El resto está en blanco. La capa de tóner tiene un espesor de 25 micras. El cartucho de tóner de la impresora mide  $25 \times 8 \times 2 \text{ cm}$ . ¿Cuántas páginas pueden imprimirse con un cartucho de tóner?
31. Cuando texto ASCII con paridad par se transmite asincrónicamente a razón de 2880 caracteres/s por un módem de 28,800 bps, ¿qué porcentaje de los bits recibidos contienen realmente datos (no gasto extra)?
32. La Hi-Fi Modem Company acaba de diseñar un nuevo módem con modulación de frecuencia que utiliza 16 frecuencias en lugar de sólo 2. Cada segundo se divide en  $n$  intervalos de tiempo iguales, cada uno de los cuales contiene uno de los 16 tonos posibles. ¿Cuántos bits por segundo puede transmitir este módem si emplea transmisión sincrónica?
33. Estime cuántos caracteres, contando los espacios, contiene un libro de texto de computación típico. ¿Cuántos bits se necesitan para codificar un libro en ASCII con paridad? ¿Cuántos CD-ROM se necesitan para almacenar una biblioteca de ciencias de la computación que tiene 10,000 libros? ¿Cuántos DVD de dos lados y capa dual se necesitan para la misma biblioteca?
34. Decodifique el siguiente texto binario en ASCII: 1001001 0100000 1001100 1001111 1010110 1000101 0100000 1011001 1001111 1010101 0101110.
35. Escriba un procedimiento *hamming(ascii, codificado)* que convierta los siete bits de orden bajo de *ascii* en una palabra de código entera de 11 bits almacenada en *codificado*.
36. Escriba una función *distancia(código, n, k)* que acepte como entrada un arreglo *código* de  $n$  caracteres de  $k$  bits cada uno y devuelva como salida la distancia del conjunto de caracteres.

# 3

## EL NIVEL DE LÓGICA DIGITAL

En la base de la jerarquía de la figura 1-2 encontramos el nivel de lógica digital, el hardware real de la computadora. En este capítulo examinaremos muchos aspectos de la lógica digital como cimientos para el estudio de niveles más altos en capítulos subsecuentes. Este tema está en la frontera entre las ciencias de la computación y la ingeniería eléctrica, pero el material está explicado de manera clara, por lo que no se necesita experiencia previa en hardware ni en ingeniería para entenderlo.

Los elementos básicos con que se construyen todas las computadoras digitales son asombrosamente sencillos. Iniciaremos nuestro estudio examinando estos elementos básicos y también el álgebra especial de dos valores (álgebra booleana) que se usa para analizarlos. Luego examinaremos algunos circuitos fundamentales que pueden construirse a base de compuertas en combinaciones sencillas, incluidos circuitos que realizan operaciones aritméticas. El siguiente tema es la forma de combinar compuertas para almacenar información, es decir, cómo se organizan las memorias. Después pasaremos al tema de las CPU, y en especial a la forma en que se comunican con la memoria y los dispositivos periféricos. En una sección posterior del capítulo se discutirán muchos ejemplos de aplicación en la industria.

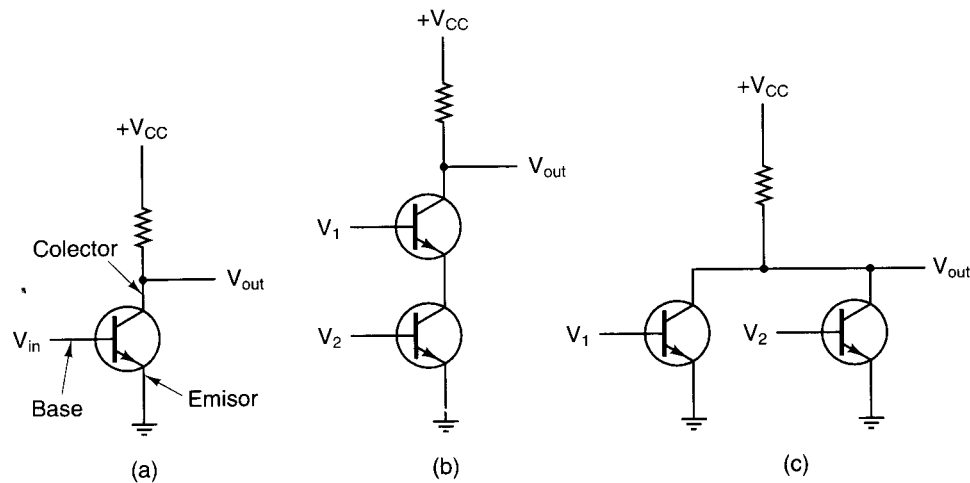
### 3.1 COMPUERTAS Y ÁLGEBRA BOOLEANA

Podemos construir circuitos digitales a partir de un número reducido de elementos primitivos combinándolos de un sinnúmero de maneras. En las secciones que siguen describiremos esos elementos primitivos, mostraremos cómo pueden combinarse, y presentaremos una potente técnica matemática que puede servir para analizar su comportamiento.

### 3.1.1 Compuertas

Un circuito digital es en el que sólo están presentes dos valores lógicos. Por lo regular, una señal entre 0 y 1 volt representa un valor (digamos, el 0 binario) y una señal entre 2 y 5 volts representa el otro valor (digamos, el 1 binario). No se permiten voltajes fuera de estos dos intervalos. Diminutos dispositivos electrónicos, llamados **compuertas**, pueden calcular diversas funciones con estas señales de dos valores. Las compuertas constituyen la base de hardware sobre la que se construyen todas las computadoras digitales.

Los detalles del funcionamiento interno de las compuertas rebasan el alcance de este libro, y pertenecen al **nivel de dispositivo**, que está abajo de nuestro nivel 0. No obstante, haremos aquí una muy breve digresión para examinar la idea básica, que no es difícil. Toda la lógica digital moderna se apoya en última instancia en el hecho de que es posible hacer que un transistor se comporte como un conmutador binario muy rápido. En la figura 3-1(a) se muestra un transistor bipolar (el círculo) incorporado a un circuito muy sencillo. Este transistor tiene tres conexiones con el mundo exterior: el **colector**, la **base** y el **emisor**. Cuando el voltaje de entrada,  $V_{in}$ , es menor que cierto valor crítico, el transistor se apaga y actúa como una resistencia infinita. Esto hace que la salida del circuito,  $V_{out}$ , adquiera un valor cercano a  $V_{CC}$ , un voltaje regulado externamente, que suele ser +5 volts para este tipo de transistor. Cuando  $V_{in}$  excede el valor crítico, el transistor se enciende y actúa como un alambre, lo que hace que  $V_{out}$  se baje a tierra (por convención, 0 volts).



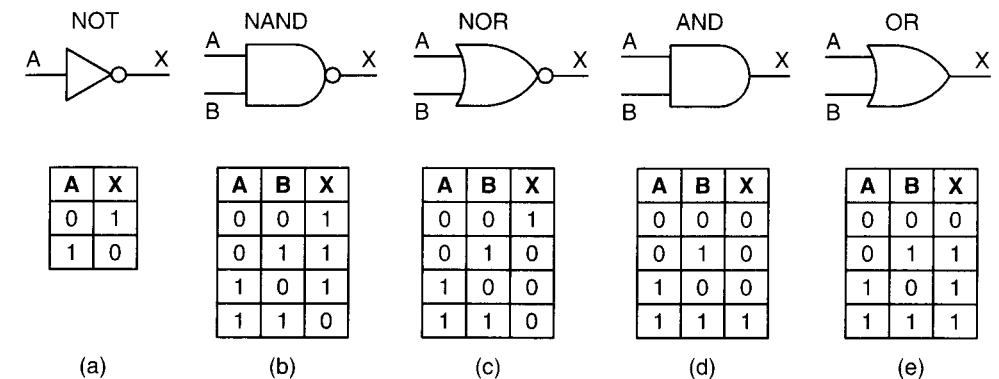
**Figura 3-1.** (a) Transistor inversor. (b) Compuerta NAND. (c) Compuerta NOR.

El aspecto importante es que cuando  $V_{in}$  es bajo,  $V_{out}$  es alto, y viceversa. Así, este circuito es un inversor que convierte un 0 lógico en un 1 lógico, y un 1 lógico en un 0 lógico. El resistor (la línea en zigzag) es necesario para limitar la cantidad de corriente que el transistor absorbe, y evitar que se queme. El tiempo necesario para conmutar de un estado al otro suele ser de unos cuantos nanosegundos.

En la figura 3-1(b) dos transistores se han conectado en serie. Si  $V_1$  y  $V_2$  son altos, ambos transistores conducirán y  $V_{out}$  bajará. Si cualquiera de las entradas es baja, el transistor correspondiente se apagará y la salida será alta. En otras palabras,  $V_{out}$  será bajo si y sólo si tanto  $V_1$  como  $V_2$  son altos.

En la figura 3-1(c) los dos transistores están conectados en paralelo, no en serie. En esta configuración, si cualquiera de las entradas es alta, el transistor correspondiente se encenderá y ocasionará que la salida vaya a tierra. Si ambas entradas son bajas, la salida se mantendrá alta.

Estos tres circuitos, o sus equivalentes, forman las tres compuertas básicas, y se llaman compuertas NOT, NAND y NOR, respectivamente. Las compuertas NOT también se conocen como **inversores**; aquí usaremos los dos términos indistintamente. Si ahora adoptamos la convención de que "alto" ( $V_{CC}$  volts) es un 1 lógico, y que "bajo" (tierra) es un 0 lógico, podremos expresar el valor de la salida en función de los valores de las entradas. Los símbolos que se usan para representar estas tres compuertas se muestran en la figura 3-2(a)-(c), junto con el comportamiento funcional de cada circuito. En estas figuras, A y B son entradas y X es la salida. Cada renglón especifica la salida para una combinación distinta de las entradas.



**Figura 3-2.** Símbolos y comportamiento funcional de las cinco compuertas básicas.

Si la señal de salida de la figura 3-1(b) se alimenta en un circuito inversor, obtenemos otro circuito que es exactamente opuesto a la compuerta NAND, a saber, un circuito cuya salida es 1 si y sólo si ambas entradas son 1. Un circuito así se llama compuerta AND; su símbolo y descripción funcional se dan en la figura 3-2(d). De forma similar, la compuerta NOR puede conectarse a un inversor para dar un circuito cuya salida es 1 si cualquiera de las entradas es 1, pero 0 si ambas entradas son 0. El símbolo y la descripción funcional de este circuito, llamado compuerta OR, se dan en la figura 3-2(e). Los círculos pequeños que se incluyen en los símbolos para el inversor, la compuerta NAND y la compuerta NOR se llaman **burbujas de inversión (compuertas universales)**, y se usan también en otros contextos para indicar una señal invertida.

Las cinco compuertas de la figura 3-2 son los principales bloques de construcción del nivel de lógica digital. Por la explicación anterior debe quedar claro que las compuertas NAND y NOR requieren dos transistores cada una, mientras que las compuertas AND y OR requieren tres.

Por esta razón, muchas computadoras se basan en las compuertas NAND y NOR en lugar de las más conocidas AND y OR. (En la práctica, todas las compuertas se implementan de forma un tanto distinta, pero las NAND y NOR siguen siendo más sencillas que las AND y OR. Vale la pena señalar que las compuertas pueden tener más de dos entradas. En principio, una compuerta NAND, por ejemplo, puede tener un número arbitrario de entradas, pero en la práctica es inusitado que haya más de ocho entradas.

Aunque el tema de la construcción de las compuertas pertenece al nivel de dispositivos, nos gustaría mencionar las principales familias de tecnología de fabricación porque continuamente se hace referencia a ellas. Las dos tecnologías principales son la **bipolar** y la **MOS** (Metal Óxido Semiconductor). Los principales tipos bipolares son **TTL** (lógica transistor-transistor), que ha sido el caballito de batalla de la electrónica digital durante muchos años, y **ECL** (lógica de emisor acoplado, *Emitter-Coupled Logic*), que se usa cuando se requiere una operación a muy alta velocidad.

Las compuertas MOS son más lentas que las TTL y ECL pero requieren mucho menos potencia y ocupan mucho menos espacio, lo que permite encapsular grandes cantidades de compuertas. Existen muchas variedades de MOS, que incluyen PMOS, NMOS y CMOS. Si bien los transistores MOS tienen una construcción diferente de la de los transistores bipolares, su capacidad para funcionar como interruptores electrónicos es la misma. Casi todas las CPU y las memorias modernas emplean tecnología CMOS, que opera a +3.3 volts. Esto es todo lo que trataremos sobre el nivel de dispositivos. Los lectores interesados en ahondar en el estudio de este nivel pueden consultar las lecturas que se sugieren en el capítulo 9.

### 3.1.2 Álgebra booleana

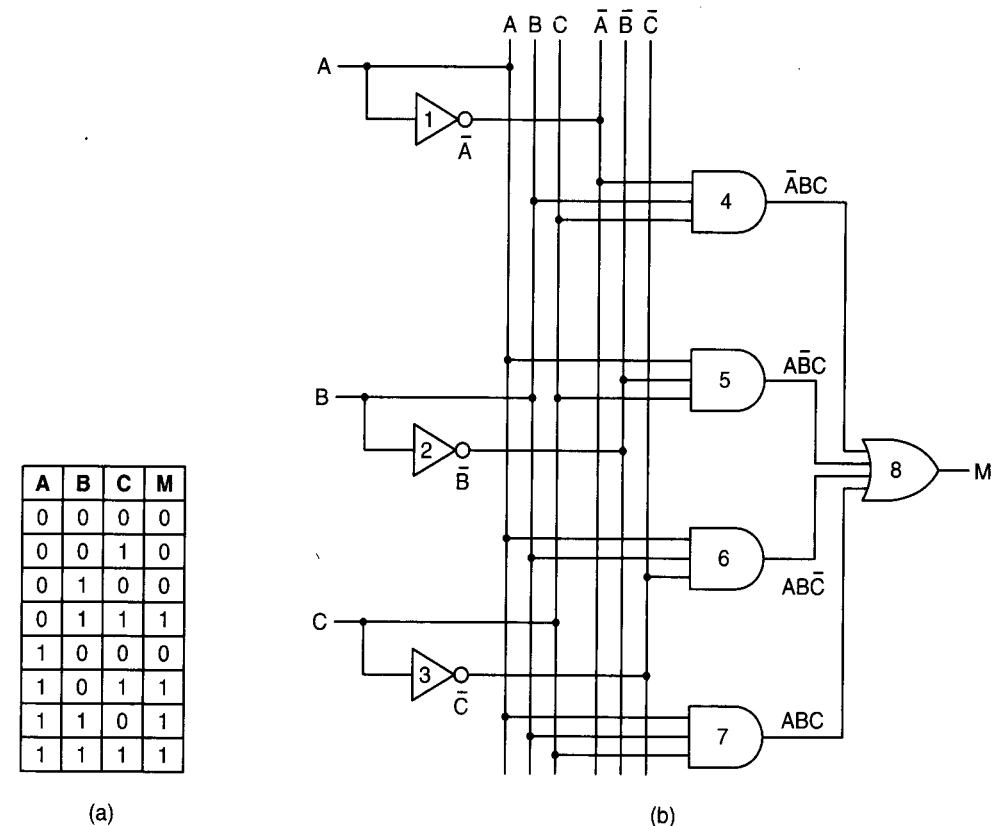
Para describir los circuitos que pueden construirse combinando compuertas se requiere un nuevo tipo de álgebra, una en la que las variables y funciones sólo puedan adoptar los valores 0 y 1. Semejante álgebra se denomina **álgebra booleana**, así llamada por su descubridor, el matemático inglés George Boole (1815-1864). En términos estrictos, nos estamos refiriendo realmente a un tipo específico de álgebra booleana, un **álgebra de conmutación**, pero el término “álgebra booleana” se usa tan ampliamente para referirse al “álgebra de conmutación” que no vamos a hacer la distinción aquí.

Así como hay funciones en el álgebra “ordinaria” (la de bachillerato), también hay funciones en el álgebra booleana. Una función booleana tiene una o más variables de entrada y produce un resultado que depende sólo de los valores de dichas variables. Podemos definir una función sencilla,  $f$ , diciendo que  $f(A)$  es 1 si  $A$  es 0 y es 0 si  $A$  es 1. Ésta es la función NOT de la figura 3-2(a).

Puesto que una función booleana de  $n$  variables sólo tiene  $2^n$  posibles combinaciones de los valores de entrada, la función puede describirse totalmente con una tabla de  $2^n$  renglones, cada uno de los cuales indica el valor de la función para una combinación distinta de los valores de entrada. Semejante tabla se llama **tabla de verdad**. Todas las tablas de la figura 3-2 son ejemplos de tablas de verdad. Si convenimos en enumerar siempre las filas de una tabla de verdad en orden numérico (base 2), es decir, para dos variables en el orden 00, 01, 10 y 11, la función podrá describirse totalmente con el número binario de  $2^n$  bits que se obtiene leyendo

verticalmente la columna del resultado de la tabla de verdad. Así, NAND es 1110, NOR es 1000, AND es 0001 y OR es 0111. Obviamente, sólo existen 16 funciones booleanas de dos variables, que corresponden a las 16 cadenas de resultado de 4 bits posibles. En contraste, el álgebra ordinaria tiene un número infinito de funciones de dos variables, ninguna de las cuales puede describirse dando una tabla de salidas para todas las posibles entradas porque cada variable puede adoptar cualquiera de un número infinito de valores posibles.

En la figura 3-3(a) se muestra la tabla de verdad para una función booleana de tres variables:  $M = f(A, B, C)$ . Esta función es la función de mayoría lógica, es decir, es 0 si la mayor parte de sus entradas son 0, y 1 si la mayor parte de sus entradas son 1. Aunque cualquier función booleana puede especificarse cabalmente dando su tabla de verdad, a medida que el número de variables aumenta esta notación se vuelve cada vez menos manejable. Por ello, se utiliza otra notación.



**Figura 3-3.** (a) Tabla de verdad de la función de mayoría de tres variables. (b) Circuito para (a).

Para entender de dónde sale esta notación, observe que cualquier función booleana puede especificarse indicando cuáles combinaciones de variables de entrada dan una salida con valor 1. Para la función de la figura 3-3(a), hay cuatro combinaciones de variables que hacen



que  $M$  sea 1. Por convención, colocamos una barra o testa sobre una variable de entrada para indicar que su valor se invierte. La ausencia de la barra implica que la variable no se invierte. Además, utilizaremos la multiplicación implícita o un punto para indicar la función AND booleana y + para denotar la función OR booleana. Así, por ejemplo,  $\overline{A}BC$  adopta el valor 1 sólo cuando  $A = 1$ ,  $B = 0$  y  $C = 1$ . También,  $\overline{A}\overline{B} + B\overline{C}$  es 1 sólo cuando ( $A = 1$  y  $B = 0$ ) o ( $B = 1$  y  $C = 0$ ). Las cuatro filas de la figura 3-3(a) que producen bits 1 en la salida son:  $\overline{A}BC$ ,  $\overline{A}B\overline{C}$ ,  $\overline{A}B\overline{C}$  y  $ABC$ . La función,  $M$ , es verdadera (o sea, 1) si es verdadera cualquiera de estas cuatro condiciones; por tanto, podemos escribir

$$M = \overline{A}BC + \overline{A}B\overline{C} + \overline{A}B\overline{C} + ABC$$

y ésta será una forma compacta de dar la tabla de verdad. Así, una función de  $n$  variables puede describirse dando una “suma” de cuando más  $2^n$  términos “producto” de  $n$  variables. Esta formulación tiene gran importancia porque, como veremos en breve, nos lleva directamente a una implementación de la función empleando compuertas estándar.

Es importante tener presente la distinción entre una función booleana abstracta y su implementación con un circuito electrónico. Una función booleana consta de variables, como  $A$ ,  $B$  y  $C$ , y operadores booleanos como AND, OR y NOT. Una función booleana se describe dando una tabla de verdad o una función booleana como

$$F = \overline{A}BC + \overline{A}B\overline{C}$$

Una función booleana puede implementarse con un circuito electrónico (a menudo de muchas formas distintas) utilizando señales que representen las variables de entrada y salida, y compuertas como AND, OR y NOT. Generalmente usaremos la notación AND, OR y NOT para referirnos a los operadores booleanos, y AND, OR y NOT para referirnos a las compuertas, pero en muchos casos esto es ambiguo.

### 3.1.3 Implementación de funciones booleanas

Como ya lo afirmamos, la formulación de una función booleana como una suma de hasta  $2^n$  términos producto nos lleva directamente a una posible implementación. Utilizando la figura 3-3 como ejemplo, vemos cómo se logra esa implementación. En la figura 3-3(b) las entradas  $A$ ,  $B$  y  $C$  aparecen a la izquierda, y la función de salida,  $M$ , aparece a la derecha. Puesto que se necesitan complementos (inversos) de las variables de entrada, se generan sacando una derivación de cada entrada y haciendo pasar estas derivaciones por los inversores rotulados 1, 2 y 3. Para que la figura sea más o menos clara, hemos dibujado seis líneas verticales, tres conectadas a las variables de entrada y tres conectadas a sus complementos. Estas líneas son una fuente cómoda de entradas para las compuertas subsecuentes. Por ejemplo,  $A$  es una entrada de las compuertas 5, 6 y 7. En un circuito real estas compuertas probablemente se conectarían directamente a  $A$  sin usar alambres “verticales” intermedios.

El circuito contiene cuatro compuertas AND, una para cada término de la ecuación de  $M$  (es decir, una para cada renglón de la tabla de verdad que tiene 1 bit en la columna de resultado). Cada compuerta AND calcula un renglón de la tabla de verdad, como se indica. Por último, se obtiene el OR de todos los términos producto para obtener el resultado final.

El circuito de la figura 3-3(b) sigue una convención que usaremos una y otra vez en todo el libro: cuando dos líneas se cruzan, no hay una conexión implícita a menos que haya un punto grueso en la intersección. Por ejemplo, la salida de la compuerta 3 cruza las seis líneas verticales pero sólo está conectada a  $\overline{C}$ . Advertimos al lector que algunos autores emplean otras convenciones.

El ejemplo de la figura 3-3 nos muestra claramente cómo podemos implementar un circuito para cualquier función booleana:

1. Escribir la tabla de verdad para la función.
2. Incluir inversores para generar el complemento de cada entrada.
3. Dibujar una compuerta AND para cada término que tiene un 1 en la columna de resultado.
4. Conectar las compuertas AND a las entradas apropiadas.
5. Alimentar la salida de todas las compuertas AND a una compuerta OR.

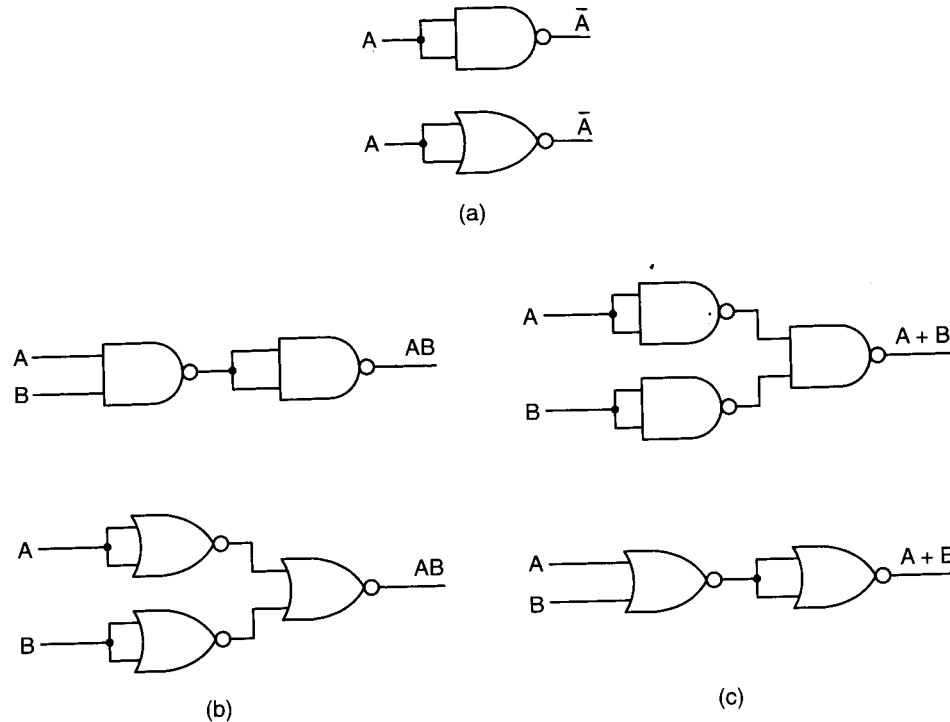
Aunque hemos mostrado cómo puede implementarse cualquier función booleana empleando compuertas NOT, AND y OR, a menudo es conveniente implementar los circuitos utilizando un solo tipo de compuerta. Por fortuna, es fácil convertir los circuitos generados por el algoritmo anterior en una forma NAND pura o NOR pura. Para efectuar tal conversión, lo único que necesitamos es una forma de implementar NOT, AND y OR empleando un solo tipo de compuerta. El renglón superior de la figura 3-4 muestra cómo pueden implementarse estas tres compuertas empleando sólo compuertas NAND; el renglón inferior muestra cómo hacerlo usando sólo compuertas NOR. (Estos métodos son sencillos, pero no son los únicos.)

Una forma de implementar una función booleana usando sólo compuertas NAND o sólo compuertas NOR es seguir primero el procedimiento anterior para construirla con NOT, AND y OR. Luego se sustituyen las compuertas de varias entradas con circuitos equivalentes que sólo tengan compuertas de dos entradas. Por ejemplo,  $A + B + C + D$  pueden calcularse como  $(A + B) + (C + D)$ , utilizando tres compuertas OR de dos entradas. Por último, las compuertas NOT, AND y OR se sustituyen por los circuitos de la figura 3-4.

Aunque este procedimiento no nos lleva a circuitos óptimos, en el sentido de tener un número mínimo de compuertas, sí nos demuestra que siempre es posible encontrar una solución. Se dice que las compuertas NAND y NOR son **completas (o universales)** porque cualquier función booleana puede calcularse empleando cualquiera de ellas. Ninguna otra compuerta tiene esta propiedad, y esto es otra razón por las que a menudo se les prefiere como bloques de construcción de circuitos.

### 3.1.4 Equivalencia de circuitos

Los diseñadores de circuitos a menudo tratan de reducir el número de compuertas en sus productos a fin de reducir el costo de los componentes, el espacio que ocupan en las tarjetas de circuitos impresos, el consumo de electricidad, etc. Para reducir la complejidad de un



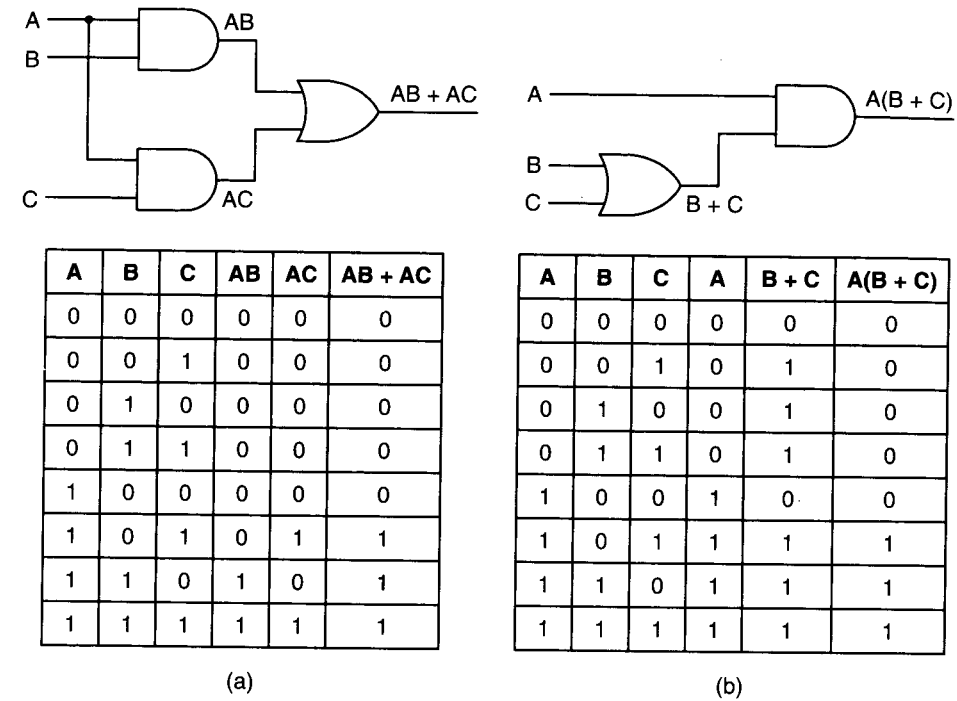
**Figura 3-4.** Construcción de compuertas (a) NOT, (b) AND y (c) OR utilizando sólo compuertas NAND o sólo compuertas NOR.

circuito, el diseñador debe encontrar otro circuito que calcule la misma función que el original pero lo haga con menos compuertas (o tal vez con compuertas más sencillas, por ejemplo, compuertas de dos entradas en lugar de cuatro). En la búsqueda de circuitos equivalentes, el álgebra booleana puede ser una herramienta valiosa.

Como ejemplo del uso del álgebra booleana, considere el circuito y la tabla de verdad para  $AB + AC$  que se muestra en la figura 3-5(a). Aunque no hemos hablado de ello, muchas de las reglas del álgebra ordinaria también se cumplen en el álgebra booleana. En particular,  $AB + AC$  puede factorizarse para dar  $A(B + C)$  empleando la ley distributiva. La figura 3-5(b) muestra el circuito y la tabla de verdad para  $A(B + C)$ . Puesto que dos funciones son equivalentes si y sólo si tienen la misma salida para todas las posibles entradas, si examinamos las tablas de verdad de la figura 3-5 es fácil ver que  $A(B + C)$  es equivalente a  $AB + AC$ . A pesar de esta equivalencia, es obvio que el circuito de la figura 3-5(b) es mejor que el de la figura 3-5(a) porque contiene menos compuertas.

En general, un diseñador de circuitos parte de una función booleana y luego le aplica las leyes del álgebra booleana en un intento por encontrar una función equivalente más sencilla. El circuito puede construirse a partir de la función final.

Para utilizar este enfoque, necesitamos algunas identidades del álgebra booleana. La figura 3-6 muestra algunas de las principales. Resulta interesante que cada ley tiene dos



**Figura 3-5.** Dos funciones equivalentes. (a)  $AB + AC$ . (b)  $A(B + C)$ .

formas, una de las cuales es el **dual** de la otra. Si intercambiamos AND y OR y también 0 y 1, cualquier forma puede producirse a partir de la otra. Todas las leyes pueden demostrarse fácilmente construyendo sus tablas de verdad. Con excepción de la ley de DeMorgan, la ley de absorción y la forma AND de la ley distributiva, los resultados son más o menos intuitivos. La ley de DeMorgan puede extenderse a más de dos variables, por ejemplo,  $\overline{ABC} = \bar{A} + \bar{B} + \bar{C}$ .

La ley de DeMorgan sugiere una notación alternativa. En la figura 3-7(a) se muestra la forma AND para indicar negación mediante burbujas de inversión, tanto en la entrada como en la salida. Así, una compuerta OR con entradas invertidas equivale a una compuerta NAND. Si examinamos la figura 3-7(b), la forma dual de la ley de DeMorgan, debe ser obvio que una compuerta NOR puede dibujarse como una compuerta AND con entradas invertidas. Si negamos ambas formas de la ley de DeMorgan, llegamos a la figura 3-7(c) y (d), que muestra representaciones equivalentes de las compuertas AND y OR. Existen símbolos análogos para las formas de múltiples variables de la ley de DeMorgan (por ejemplo, una compuerta NAND de  $n$  entradas se convierte en una compuerta OR con  $n$  entradas invertidas).

Utilizando las identidades de la figura 3-7, y las identidades análogas para compuertas con varias entradas, es fácil convertir la representación de suma de productos de una tabla de verdad a una forma NAND pura o NOR pura. Por ejemplo, consideremos la función OR EXCLUSIVO de la figura 3-8(a). El circuito estándar de suma de productos se muestra en la

| Nombre           | Función AND                         | Función OR                          |
|------------------|-------------------------------------|-------------------------------------|
| Ley de identidad | $1A = A$                            | $0 + A = A$                         |
| Ley nula         | $0A = 0$                            | $1 + A = 1$                         |
| Ley idempotencia | $AA = A$                            | $A + A = A$                         |
| Ley inversa      | $A\bar{A} = 0$                      | $A + \bar{A} = 1$                   |
| Ley conmutativa  | $AB = BA$                           | $A + B = B + A$                     |
| Ley asociativa   | $(AB)C = A(BC)$                     | $(A + B) + C = A + (B + C)$         |
| Ley distributiva | $A + BC = (A + B)(A + C)$           | $A(B + C) = AB + AC$                |
| Ley de absorción | $A(A + B) = A$                      | $A + AB = A$                        |
| Ley de DeMorgan  | $\overline{AB} = \bar{A} + \bar{B}$ | $\overline{A + B} = \bar{A}\bar{B}$ |

Figura 3-6. Algunas identidades del álgebra booleana.

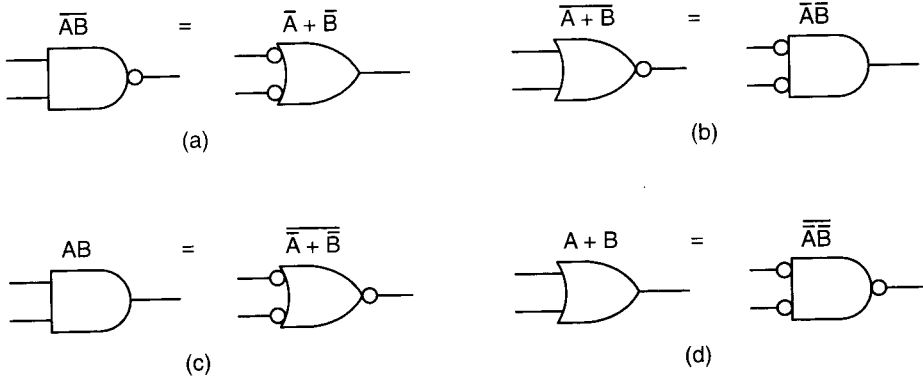


Figura 3-7. Símbolos alternativos para algunas compuertas: (a) NAND. (b) NOR. (c) AND. (d) OR.

figura 3-8(b). Para convertirlo a una forma NAND, las líneas que conectan la salida de las compuertas AND a la entrada de la compuerta OR se deben redibujar con dos burbujas de inversión, como se muestra en la figura 3-8(c). Por último, utilizando la figura 3-7(a), llegamos a la figura 3-8(d). Las variables  $\bar{A}$  y  $\bar{B}$  pueden generarse a partir de  $A$  y  $B$  utilizando compuertas NAND o NOR con sus entradas unidas. Observe que las burbujas de inversión se mueven a lo largo de una línea a voluntad; por ejemplo, de las salidas de las compuertas de entrada de la figura 3-8(d) a las entradas de la compuerta de salida.

Como nota final acerca de la equivalencia de circuitos, demostraremos el sorprendente resultado de que la misma compuerta física puede calcular diferentes funciones, dependiendo de las convenciones empleadas. En la figura 3-9(a) mostramos la salida de cierta compuerta,  $F$ , para diferentes combinaciones de entradas. Tanto las entradas como las salidas se

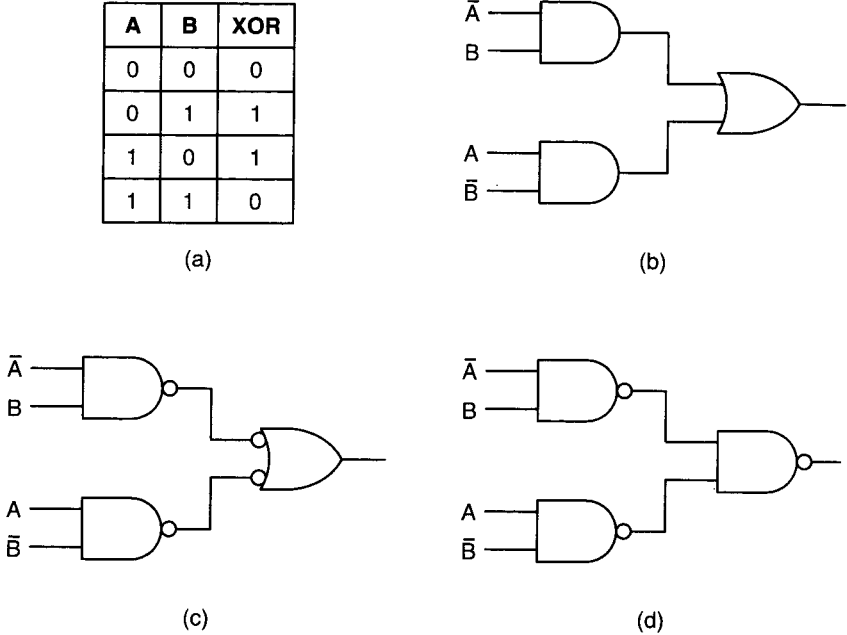


Figura 3-8. (a) Tabla de verdad de la función XOR. (b)-(d) Tres circuitos para calcularla.

indican en volts. Si adoptamos la convención de que 0 volts es un 0 lógico y 3.3 volts o 5 volts es un 1 lógico, llamada **lógica positiva**, obtenemos la tabla de verdad de la figura 3-9(b), la función AND. Pero si adoptamos **lógica negativa**, en la que 0 volts es un 1 lógico y 3.3 o 5 volts es un 0 lógico, obtenemos la tabla de verdad de la figura 3-9(c), la función OR.

| A              | B              | F              |
|----------------|----------------|----------------|
| 0 <sup>V</sup> | 0 <sup>V</sup> | 0 <sup>V</sup> |
| 0 <sup>V</sup> | 5 <sup>V</sup> | 0 <sup>V</sup> |
| 5 <sup>V</sup> | 0 <sup>V</sup> | 0 <sup>V</sup> |
| 5 <sup>V</sup> | 5 <sup>V</sup> | 5 <sup>V</sup> |

(a)

| A | B | F |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

(b)

| A | B | F |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 0 | 0 | 0 |

(c)

Figura 3-9. (a) Características eléctricas de un dispositivo. (b) Lógica positiva. (c) Lógica negativa.

Así pues, la convención que se escoja para transformar voltajes en valores lógicos es crucial. Excepto si se especifica lo contrario, de aquí en adelante usaremos **lógica positiva**, de modo que los términos 1 lógico, verdadero y alto son sinónimos, lo mismo que los términos 0 lógico, falso y bajo.

## 3.2 CIRCUITOS LÓGICOS DIGITALES BÁSICOS

En las secciones anteriores vimos cómo implementar tablas de verdad y otros circuitos sencillos utilizando compuertas individuales. En la práctica, ya son pocos los circuitos que se construyen compuerta por compuerta, aunque esto solía ser común. Hoy en día, los bloques de construcción usuales son módulos que contienen varias compuertas. En las secciones que siguen examinaremos más de cerca esos bloques de construcción para ver cómo se usan y cómo pueden construirse a partir de compuertas individuales.

### 3.2.1 Circuitos integrados

Las compuertas no se fabrican ni se venden individualmente, sino en unidades llamadas **circuitos integrados**, también conocidos como **IC** (*Integrated Circuits*) o **chips**. Un IC es un trozo cuadrado de silicio de unos 5 mm × 5 mm en el que se han depositado algunas compuertas. Los IC pequeños suelen montarse en paquetes de plástico o cerámica rectangulares con una anchura de 5 a 15 mm y una longitud de 20 a 50 mm. En los lados largos hay dos filas paralelas de terminales de unos 5 mm de largo que se pueden insertar en zócalos o soldarse a tarjetas de circuitos impresos. Cada terminal está conectada a la entrada o salida de alguna compuerta del chip, a la alimentación eléctrica, o a tierra. Los paquetes con dos filas de terminales en el exterior e IC en el interior se conocen técnicamente como **paquetes duales en línea** o **DIP** (*Dual Inline Packages*), pero todo mundo los llama chips, lo que hace borrosa la distinción entre el trozo de silicio y su paquete. Los paquetes más comunes tienen 14, 16, 18, 20, 22, 24, 28, 40, 64 o 68 terminales. En el caso de chips grandes es común usar paquetes cuadrados con terminales en los cuatro lados o en la base.

Los chips pueden dividirse a grandes rasgos en clases basadas en el número de compuertas que contienen, como se indica en seguida. Obviamente, este esquema de clasificación es en extremo burdo, pero en ocasiones resulta útil.

- Circuito SSI (integrado a pequeña escala, *Small Scale Integrated*): 1 a 10 compuertas.
- Circuito MSI (integrado a mediana escala, *Medium Scale Integrated*): 10 a 100 compuertas.
- Circuito LSI (integrado a gran escala, *Large Scale Integrated*): 100 a 100,000 compuertas.
- Circuito VLSI (integrado a muy grande escala, *Very Large Scale Integrated*): >100,000 compuertas.

Estas clases tienen diferentes propiedades y se usan de distintas formas.

Un chip SSI por lo regular contiene entre dos y seis compuertas independientes, cada una de las cuales se puede usar en forma individual, al estilo de las secciones anteriores. La figura 3-10 ilustra esquemáticamente un chip SSI común que contiene cuatro compuertas NAND. Cada una de éstas tiene dos entradas y una salida, lo que requiere un total de 12 terminales para las cuatro compuertas. Además, el chip necesita alimentación ( $V_{CC}$ ) y tierra (GND), que todas las compuertas comparten. En general, el paquete tiene una muesca cerca de la terminal 1 para identificar la orientación. Para evitar confusión en los diagramas de circuitos, por convención no se muestran la alimentación, la tierra ni las compuertas que no se utilizan.

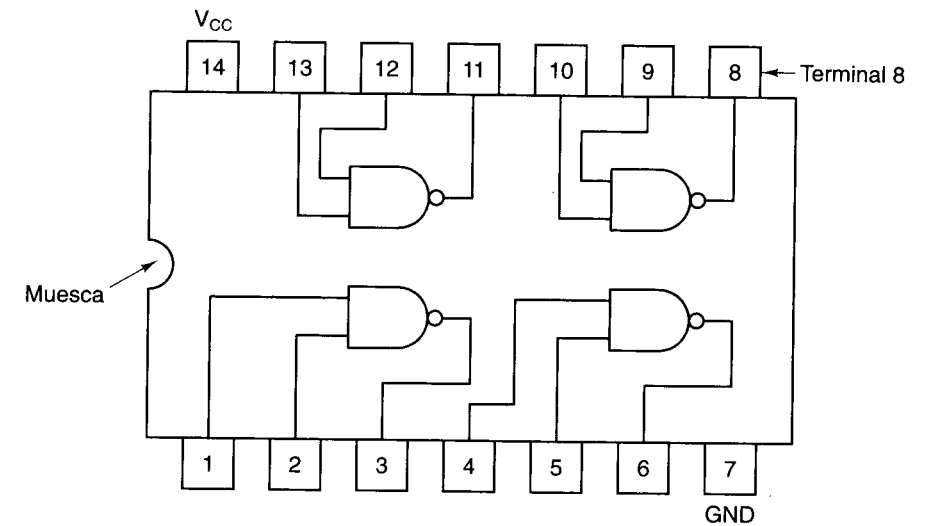


Figura 3-10. Chip SSI que contiene cuatro compuertas.

Es posible obtener muchos otros chips como éste por unos cuantos centavos de dólar cada uno. Cada chip SSI tiene un puñado de compuertas y hasta unas 20 terminales. En los años setenta, las computadoras se construían con grandes cantidades de estas compuertas, pero hoy en día toda una CPU y una cantidad sustancial de memoria (caché) se integran en un solo chip.

Para nuestros fines, todas las compuertas son ideales en el sentido de que la salida aparece tan pronto como se aplica la entrada. En realidad, los chips tienen un **retraso de compuerta** finito que incluye tanto el tiempo de propagación de la señal a través del chip y el tiempo de conmutación. Los retardos típicos son de 1 a 10 ns.

Actualmente es posible colocar casi 10 millones de transistores en un chip. Dado que cualquier circuito se puede construir con compuertas NAND, podríamos pensar que un fabricante podría crear un chip muy general con cinco millones de compuertas NAND. Desafortunadamente, semejante chip necesitaría 15,000,002 terminales. Con el espaciado estándar de 0.1 pulg entre terminales, el chip tendría más de 18 km de largo, lo cual podría tener un efecto negativo sobre las ventas. Obviamente, la única forma de aprovechar la tecnología es diseñar circuitos con una fuerte relación compuertas/terminales. En las secciones que siguen examinaremos circuitos MSI sencillos que combinan varias compuertas internamente para proporcionar una función útil que sólo requiere un número limitado de conexiones externas (terminales metálicas).

### 3.2.2 Circuitos combinacionales

Muchas aplicaciones de la lógica digital requieren un circuito con varias entradas y varias salidas en el que las salidas estén determinadas de forma única por las entradas vigentes. Un circuito así se denomina **circuito combinacional**. No todos los circuitos tienen esta propiedad. Por ejemplo, un circuito que contiene elementos de memoria bien podría generar salidas que depen-

den de los valores almacenados además de las variables de entrada. Un circuito que implementa una tabla de verdad, como el de la figura 3-3(a), es un ejemplo típico de circuito combinacional. En esta sección examinaremos algunos circuitos combinacionales de uso frecuente.

### Multiplexores

En el nivel de lógica digital, un **multiplexor** es un circuito con  $2^n$  entradas de datos, una salida de datos y  $n$  entradas de control que seleccionan una de las entradas de datos. La entrada de datos seleccionada se "pasa por compuertas" (se encamina) hacia la salida. La figura 3-11 es un diagrama esquemático de un multiplexor de ocho entradas. Las tres líneas de control,  $A$ ,  $B$  y  $C$ , codifican un número de 3 bits que especifica cuál de las ocho líneas de entrada se encamina a la compuerta OR y de ahí a la salida. Sea cual sea el valor que esté en las líneas de control, siete de las compuertas AND siempre producirán 0; la otra podría producir 0 o 1, dependiendo del valor de la línea de entrada seleccionada. Cada compuerta AND se habilita con una combinación distinta de las entradas de control. El circuito del multiplexor se muestra en la figura 3-11. Después de añadir alimentación y tierra, puede meterse en un encapsulado de 14 terminales.

Con el multiplexor podemos implementar la función de mayoría de la figura 3-3(a) como se muestra en la figura 3-12(b). Para cada combinación de  $A$ ,  $B$  y  $C$ , se selecciona una de las líneas de entrada de datos. Cada entrada se conecta con  $V_{CC}$  (1 lógico) o con tierra (0 lógico). El algoritmo para conectar las entradas es sencillo: la entrada  $D_i$  tiene el mismo valor que el renglón  $i$  de la tabla de verdad. En la figura 3-3(a) los renglones 0, 1, 2 y 4 son 0, así que las entradas correspondientes se conectan a tierra; los demás renglones son 1, así que se conectan con el 1 lógico. De esta forma, cualquier tabla de verdad de tres variables se puede implementar utilizando el chip de la figura 3-12(a).

Ya vimos cómo puede usarse un chip multiplexor para seleccionar una de varias entradas y cómo puede implementar una tabla de verdad. Otra de sus aplicaciones es como convertidor de datos paralelos a seriales. Si colocamos 8 bits de datos en las líneas de entrada y luego alimentamos las líneas de control sucesivamente con los números binarios 000 a 111, los ocho bits saldrán por la línea de salida en serie. Un uso típico de la conversión paralelo a serial es en un teclado, donde cada digitación define implícitamente un número de 7 u 8 bits que se debe enviar serialmente por una línea telefónica.

Lo contrario de un multiplexor es un **desmultiplexor**, que encamina su única señal de entrada a una de  $2^n$  salidas, dependiendo de los valores de las  $n$  líneas de control. Si el valor binario en las líneas de control es  $k$ , se selecciona la salida  $k$ .

### Decodificadores

Como segundo ejemplo, examinaremos un circuito que acepta un número de  $n$  bits como entrada y lo utiliza para seleccionar (es decir, poner en 1) una y sólo una de las  $2^n$  líneas de salida. Semejante circuito, que se ilustra para  $n = 3$  en la figura 3-13, se llama **decodificador**.

Para ver en qué casos podría ser útil un decodificador, imagine una memoria que consista en ocho chips, cada uno de los cuales contiene 1 MB. El chip 0 tiene las direcciones de 0 a

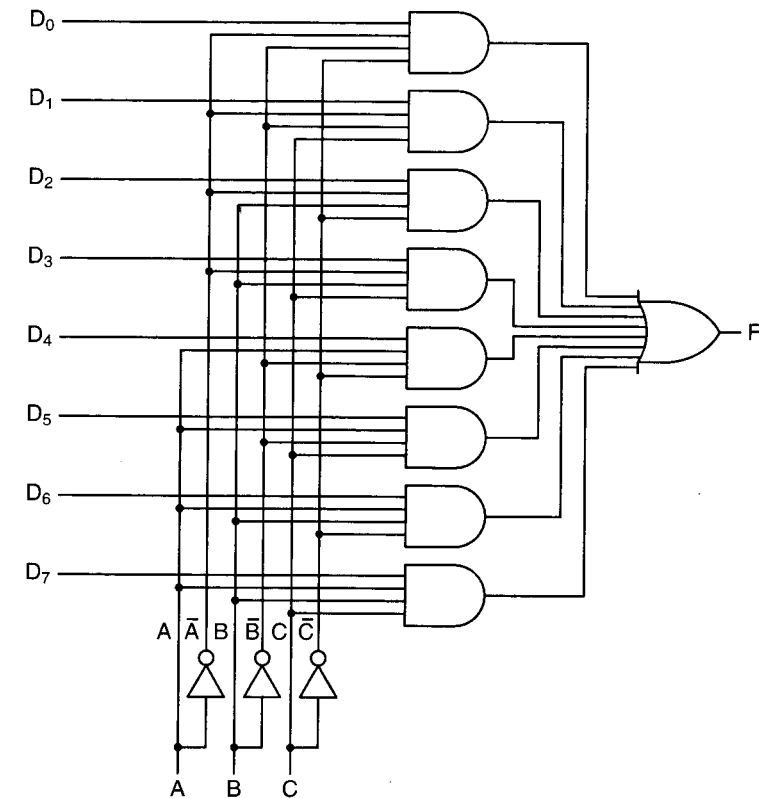


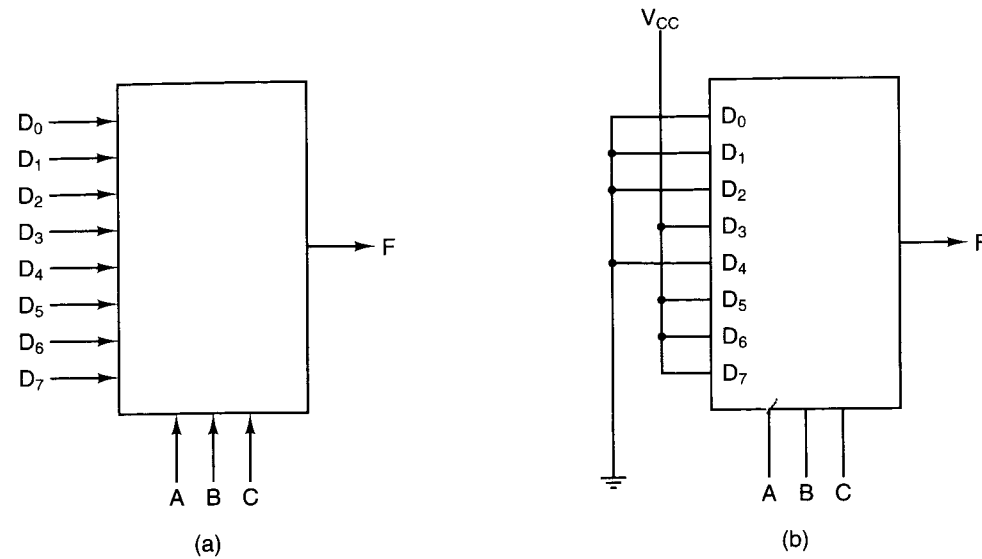
Figura 3-11. Circuito multiplexor de ocho entradas.

1 MB, el chip 1 tiene las direcciones de 1 MB a 2 MB, etc. Cuando se presenta una dirección a la memoria, los 3 bits de orden alto sirven para seleccionar uno de los ocho chips. Utilizando el circuito de la figura 3-13, estos 3 bits son las tres entradas  $A$ ,  $B$  y  $C$ . Dependiendo de las entradas, una y sólo una de las ocho líneas de salida,  $D_0, \dots, D_7$ , es 1; el resto son 0. Cada línea de salida habilita uno de los ocho chips de memoria. Puesto que sólo una línea de salida se pone en 1, sólo se habilita un chip.

El funcionamiento del circuito de la figura 3-13 es sencillo. Cada compuerta AND tiene tres entradas, de las cuales la primera es  $A$  o bien  $\bar{A}$ , la segunda es  $B$  o bien  $\bar{B}$  y la tercera es  $C$  o bien  $\bar{C}$ . Cada compuerta se habilita con una combinación de entradas distinta:  $D_0$  con  $\bar{A}\bar{B}\bar{C}$ ,  $D_1$  con  $\bar{A}\bar{B}C$ , etcétera.

### Comparadores

Otro circuito útil es el **comparador**, que compara dos palabras de entrada. El sencillo comparador de la figura 3-14 acepta dos entradas,  $A$  y  $B$ , cada una de 4 bits, y produce 1 si son iguales y 0 en caso contrario. El circuito se basa en la compuerta XOR (OR EXCLUSIVA),



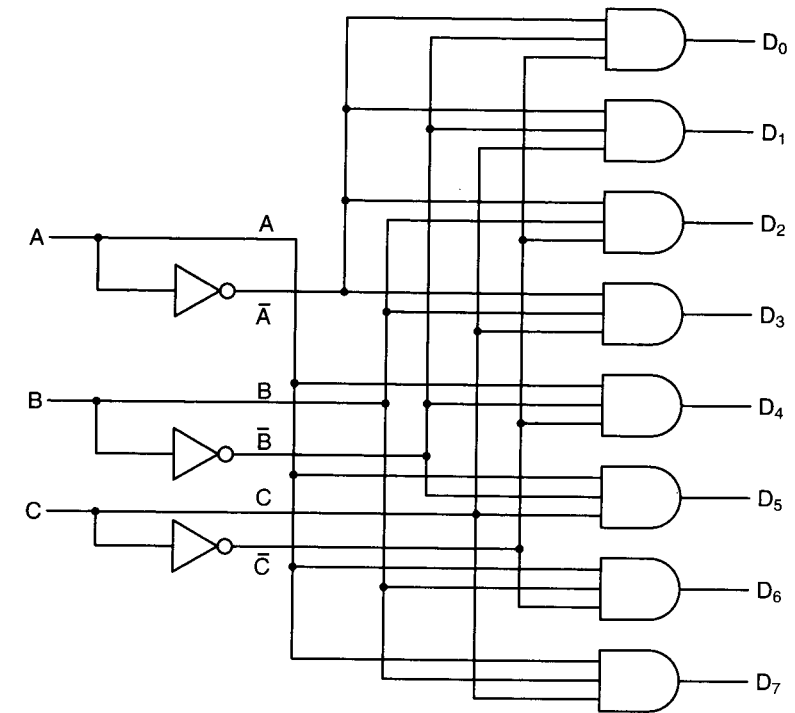
**Figura 3-12.** (a) Multiplexor MSI. (b) El mismo multiplexor alambrado para calcular la función de mayoría.

que produce un 0 si sus entradas son iguales, y un 1 si no lo son. Si las dos palabras de entrada son iguales, las cuatro compuertas XOR deberán producir 0. Estas cuatro señales se conectan después a una compuerta OR; si el resultado es 0, las palabras de entrada son iguales; si es 1, no lo son. En nuestro ejemplo usamos una compuerta NOR como etapa final para invertir el sentido de la prueba: 1 significa igualdad; 0 significa desigualdad.

### Arreglos lógicos programables

Ya vimos que es posible construir funciones arbitrarias (tablas de verdad) calculando términos producto con compuertas AND y haciendo pasar los resultados por una compuerta OR. Un chip muy general para formar sumas de productos es el **arreglo lógico programable** o **PLA** (*Programmable Logic Array*), del cual se muestra un ejemplo pequeño en la figura 3-15. Este chip tiene líneas de entrada para 12 variables. El complemento de cada entrada se genera internamente, para dar un total de 24 señales de entrada. El corazón del circuito es un arreglo de 50 compuertas AND, cada una de las cuales puede tener en potencia cualquier subconjunto de las 24 señales de entrada como su entrada. La determinación de qué señal de entrada va a qué compuerta AND se efectúa con un arreglo de  $24 \times 50$  bits proporcionado por el usuario. Cada línea de entrada a las 50 compuertas AND contiene un fusible. Cuando el circuito sale de la fábrica, los 1200 fusibles están intactos. Para programar el arreglo, el usuario quema fusibles selectos aplicando un voltaje alto al chip.

La parte de salida del circuito consiste en seis compuertas OR, cada una de las cuales tiene hasta 50 entradas que corresponden a las 50 salidas de las compuertas AND. En este caso también, un arreglo (de  $50 \times 6$ ) proporcionado por el usuario indica cuáles de las posibles



**Figura 3-13.** Un circuito decodificador de 3 a 8.

conexiones realmente existen. El chip tiene 12 terminales de entrada, 6 terminales de salida, alimentación y tierra, para un total de 20.

Como ejemplo del uso de un PLA, volvamos a considerar el circuito de la figura 3-3(b). Este circuito tiene tres entradas, cuatro compuertas AND, una compuerta OR y tres inversores. Una vez hechas las conexiones internas apropiadas, nuestro PLA puede calcular la misma función utilizando tres de sus 12 entradas, cuatro de sus 50 compuertas AND y una de sus seis compuertas OR. (Las cuatro compuertas AND deberán calcular  $\bar{A}BC$ ,  $A\bar{B}C$ ,  $AB\bar{C}$  y  $ABC$ , respectivamente; la compuerta OR acepta estos cuatro términos de producto como entradas.) De hecho, el mismo PLA podría configurarse a modo de calcular simultáneamente un total de cuatro funciones de complejidad similar. En el caso de estas funciones sencillas el número de variables de entrada es el factor limitante; si la función es más compleja, el factor limitante podría ser el número de compuertas AND y OR.

Aunque los PLA programables en campo que acabamos de describir se siguen usando, para muchas aplicaciones son preferibles los PLA hechos a la medida. El cliente (al mayoreo) diseña estos PLA y el fabricante los produce según las especificaciones del cliente. Tales PLA cuestan menos que los programables en campo.

Ahora podemos comparar las tres diferentes formas de implementar la tabla de verdad de la figura 3-3(a) que hemos visto. Si utilizamos componentes SSI, necesitamos cuatro chips. Como alternativa, bastaría con un chip multiplexor MSI, como se muestra en la figura

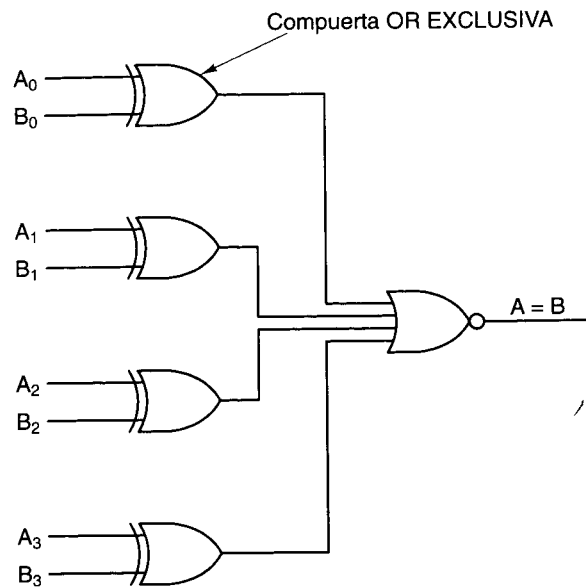


Figura 3-14. Comparador simple de 4 bits.

3-12(b). Por último, podríamos usar una cuarta parte de un chip PLA. Obviamente, si se necesitan muchas funciones, el PLA es más eficiente que los otros dos métodos. En el caso de circuitos sencillos, los chips SSI y MSI, más baratos, podrían ser preferibles.

### 3.2.3 Circuitos aritméticos

Ha llegado el momento de pasar de los circuitos MSI de propósito general vistos anteriormente a los circuitos combinacionales MSI que sirven para efectuar operaciones aritméticas. Comenzaremos con un sencillo desplazador de 8 bits, y luego veremos cómo se construyen los sumadores. Por último, examinaremos las unidades aritmética y lógica, que desempeñan un papel central en cualquier computadora.

#### Desplazadores

Nuestro primer circuito MSI de aritmética es un desplazador de ocho entradas y ocho salidas (vea la figura 3-16). Se presentan ocho bits de entrada en las líneas  $D_0, \dots, D_7$ . La salida, que no es más que la entrada desplazada un bit, se obtiene en las líneas  $S_0, \dots, S_7$ . La línea de control,  $C$ , determina la dirección del desplazamiento, 0 hacia la izquierda y 1 hacia la derecha.

Para ver cómo funciona el circuito, observe los pares de compuertas AND para todos los bits excepto las compuertas del final. Cuando  $C = 1$ , el miembro derecho de cada par se enciende, y pasa el bit de entrada correspondiente a la salida. Puesto que la compuerta AND derecha está conectada a la salida de la compuerta OR que está a su derecha, se efectúa un

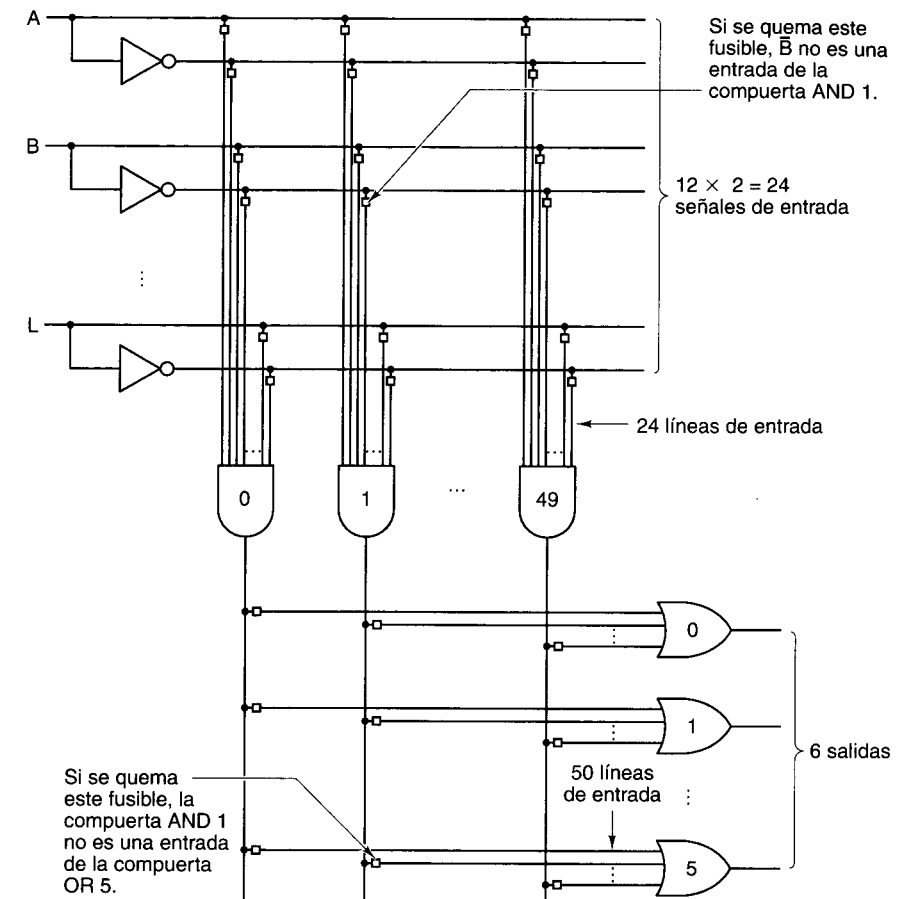


Figura 3-15. Arreglo lógico programable de 12 entradas y 6 salidas. Los cuadritos representan fusibles que pueden quemarse para determinar la función que se calculará. Los fusibles están dispuestos en dos matrices; la de arriba para las compuertas AND y la de abajo para las compuertas OR.

desplazamiento hacia la derecha. Cuando  $C = 0$ , es el miembro izquierdo del par de compuertas AND el que se enciende, y se efectúa un desplazamiento a la izquierda.

#### Sumadores

Una computadora que no puede sumar enteros es casi inconcebible. Por tanto, un circuito de hardware para efectuar una suma es un componente indispensable de toda CPU. La tabla de verdad para la suma de enteros de un bit se muestra en la figura 3-17(a). Hay dos salidas: la suma de las entradas,  $A$  y  $B$ , y el acarreo a la siguiente posición (hacia la izquierda). En la figura 3-17(b) se ilustra un circuito para calcular tanto el bit de suma como el bit de acarreo. Este sencillo circuito se conoce generalmente como **medio sumador**.



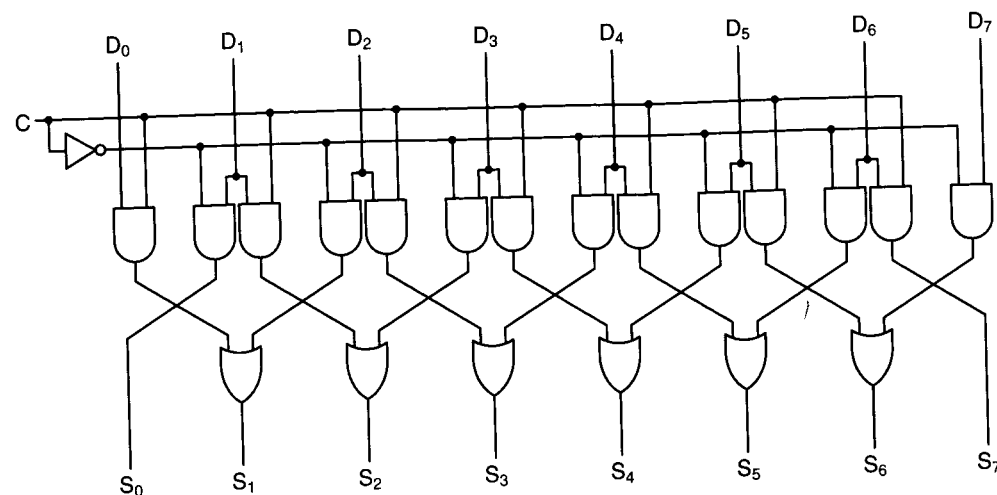


Figura 3-16. Desplazador a la izquierda/derecha de un bit.

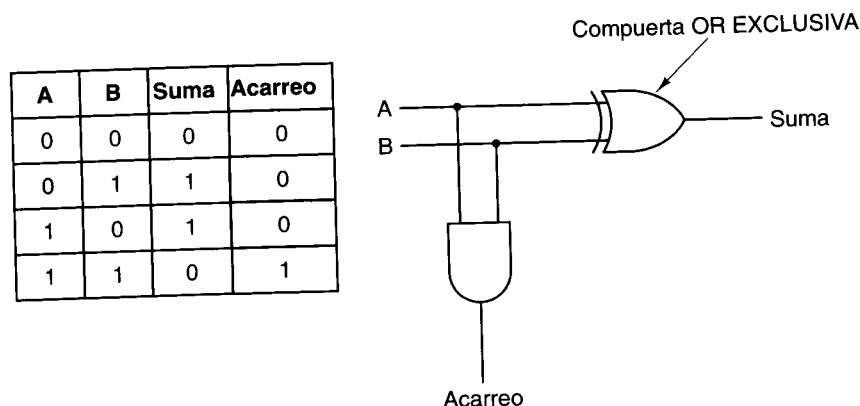


Figura 3-17. (a) Tabla de verdad para la suma de un bit. (b) Circuito de un medio sumador.

Aunque un medio sumador es suficiente para sumar los bits de orden bajo de dos palabras de entrada de varios bits, no sirve para una posición de bit interior de la palabra porque no maneja el acarreo que llega de la posición que está a su derecha. Se necesita el **sumador completo** de la figura 3-18. Si examinamos el circuito nos quedará claro que un sumador completo se construye con dos medio sumadores. La línea de salida *Suma* es 1 si el número de unos en *A*, *B* y el *Acarreo de entrada* es impar. *Acarreo de salida* es 1 si *A* y *B* son ambos 1 (entrada izquierda de la compuerta OR) o uno y sólo uno de ellos es 1 y el bit *Acarreo de entrada* es también 1. Juntos, los dos medio sumadores generan los bits tanto de suma como de acarreo.

Si queremos construir un sumador para, digamos, dos palabras de 16 bits, basta con repetir el circuito de la figura 3-18(b) 16 veces. El acarreo de salida de un bit se usa como

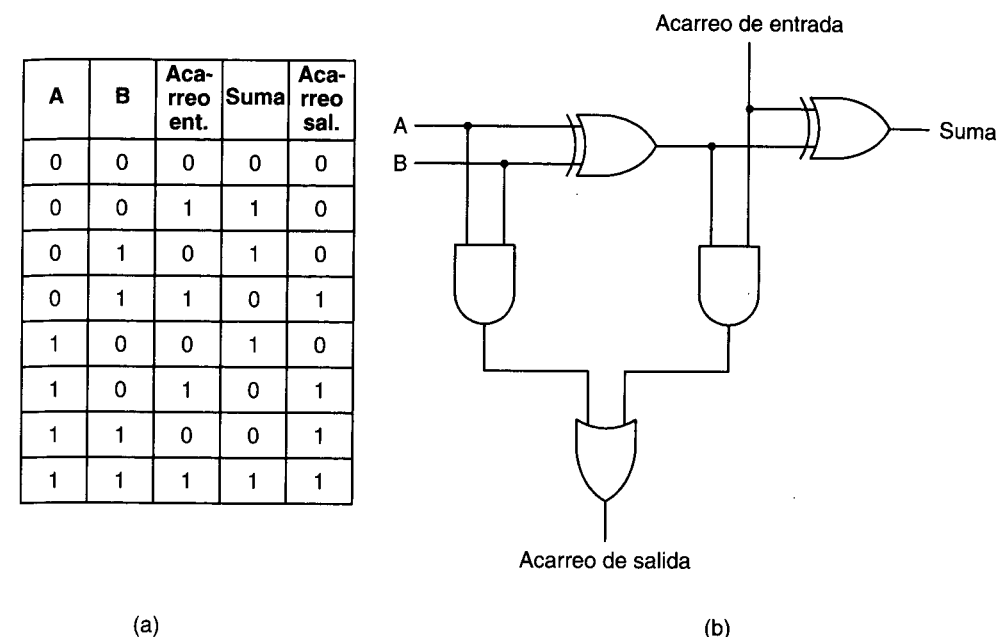


Figura 3-18. (a) Tabla de verdad para un sumador completo. (b) Circuito de un sumador completo.

acarreo de entrada de su vecino izquierdo. El acarreo de entrada del bit de la extrema derecha se conecta a 0. Este tipo de sumador se llama **sumador con propagación de acarreo** porque, en el peor de los casos, que es sumar 1 a 111...111 (binario), la suma no puede completarse hasta que el acarreo se ha propagado como una onda desde el bit de la extrema derecha hasta el de la extrema izquierda. También existen, y generalmente se prefieren, sumadores que no tienen este retraso y por tanto son más rápidos.

Como ejemplo sencillo de sumador más rápido, considere la separación de un sumador de 32 bits en una mitad inferior de 16 bits y una mitad superior de 16 bits. Cuando se inicia la suma, el sumador superior no puede ponerse a trabajar todavía porque no va a conocer su acarreo de entrada sino hasta dentro de 16 tiempos de suma.

Sin embargo, consideremos esta modificación. En lugar de tener una sola mitad superior demos al sumador dos mitades superiores en paralelo duplicando el hardware de la mitad superior. Así el circuito consiste ahora en tres sumadores de 16 bits: una mitad superior y dos mitades superiores, *U0* y *U1* que se ejecutan en paralelo. Se alimenta un 0 a *U0* como acarreo; y se alimenta un 1 a *U1* como acarreo. Ahora ambos pueden comenzar a trabajar al mismo tiempo que la mitad inferior, pero sólo uno dará el resultado correcto. Después de 16 tiempos de suma, se sabrá si el acarreo de entrada de la segunda mitad es 0 o 1, y podrá seleccionarse la mitad superior correcta de entre las dos respuestas disponibles. Este truco reduce el tiempo de suma en un factor de dos. Un sumador de este tipo se llama **sumador por selección de acarreo**. El truco puede repetirse para construir cada sumador de 16 bits con sumadores de 8 bits repetidos, y así sucesivamente.

### Unidades aritmética lógica

Casi todas las computadoras contienen un solo circuito para obtener el AND, el OR y la suma de dos palabras de máquina. Por lo regular, un circuito de este tipo para palabras de  $n$  bits se construye con  $n$  circuitos idénticos para las posiciones de bit individuales. La figura 3-19 es un ejemplo sencillo de un circuito de este tipo, llamado **unidad aritmética lógica** o **ALU** (*Arithmetic Logic Unit*). Una unidad aritmética y lógica puede calcular cualquiera de cuatro funciones:  $A \text{ AND } B$ ,  $A \text{ OR } B$ ,  $\bar{B}$ , o  $A + B$ , dependiendo de si las líneas de entrada de selección de función  $F_0$  y  $F_1$  contienen 00, 01, 10 o 11 (binario). Cabe señalar que aquí  $A + B$  significa la suma aritmética de  $A$  y  $B$ , no el AND booleano.

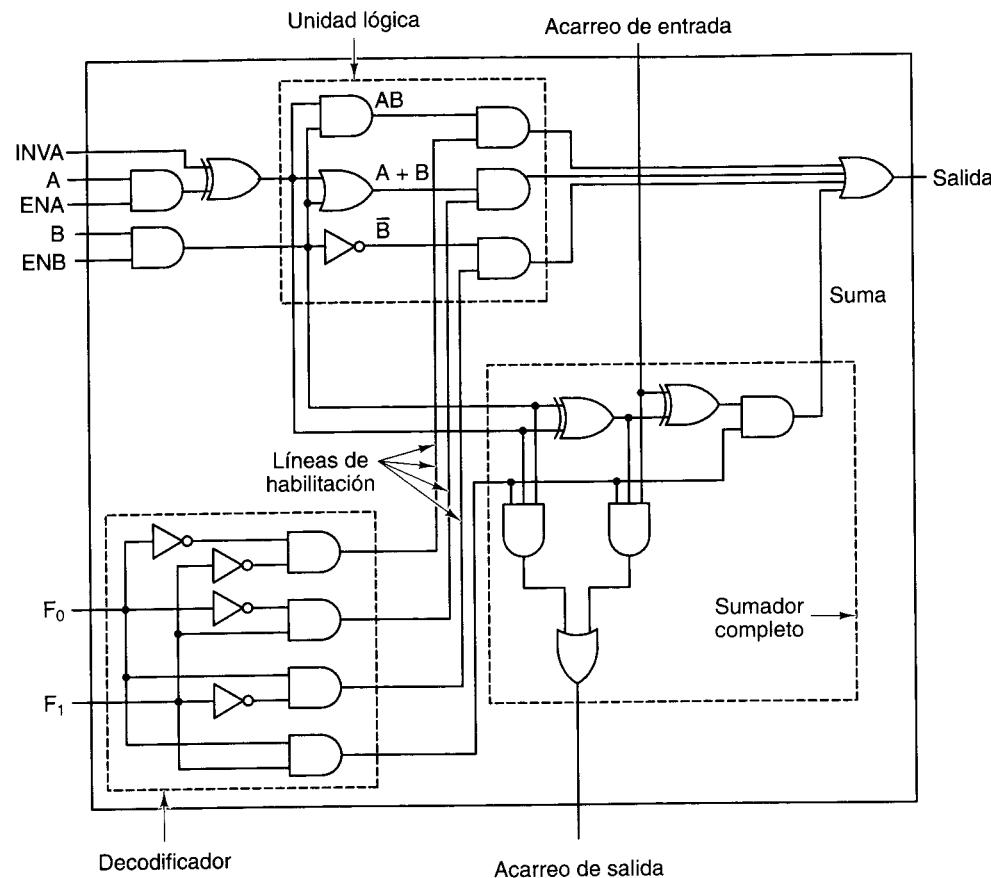


Figura 3-19. Una ALU de un bit.

La esquina inferior izquierda de nuestra ALU contiene un decodificador de 2 bits que genera señales de habilitación para las cuatro operaciones, con base en las señales de control  $F_0$  y  $F_1$ . Dependiendo de los valores de  $F_0$  y  $F_1$ , una y sólo una de las cuatro líneas de habilitación

tación se selecciona. Un valor de 1 en una de estas líneas permite que la salida de la función seleccionada pase hasta la compuerta OR final y de ahí a la salida.

La esquina superior izquierda contiene la lógica necesaria para calcular  $A \text{ AND } B$ ,  $A \text{ OR } B$  y  $\bar{B}$ , pero sólo uno de estos resultados pasa a la compuerta OR final, dependiendo de las líneas de habilitación que salen del decodificador. Dado que una y sólo una de las líneas del decodificador es 1, una y sólo una de las cuatro compuertas AND que alimentan a la compuerta OR estará habilitada; las otras tres producirán 0, sean cuales sean los valores de  $A$  y  $B$ .

Además de poder usar  $A$  y  $B$  como entradas de operaciones aritméticas o lógicas, es posible hacer que cualquiera de las dos sea 0 negando ENA o ENB, respectivamente. También es posible obtener  $\bar{A}$  poniendo INVA. Veremos algunos usos para INVA, ENA y ENB en el capítulo 4. En condiciones normales, tanto ENA como ENB son 1 para habilitar ambas entradas, e INVA es 0. En este caso,  $A$  y  $B$  simplemente se alimentan a la unidad lógica sin modificación.

La esquina inferior derecha de la ALU contiene un sumador completo para calcular la suma de  $A$  y  $B$ , e incluye manejo de acarreo porque es probable que se conecten juntos varios de estos circuitos para efectuar operaciones con palabras enteras. Es posible conseguir circuitos como el de la figura 3-19, llamado **porción de bit** (*bit slice*). Estas porciones permiten al diseñador de una computadora construir una ALU de la capacidad deseada. La figura 3-20 muestra una ALU de 8 bits formada por ocho ALU de porción de bit. La señal INC sólo es útil en operaciones de suma; si está presente, incrementa (es decir, suma 1 a) el resultado, lo que permite calcular sumas como  $A + 1$  y  $A + B + 1$ .

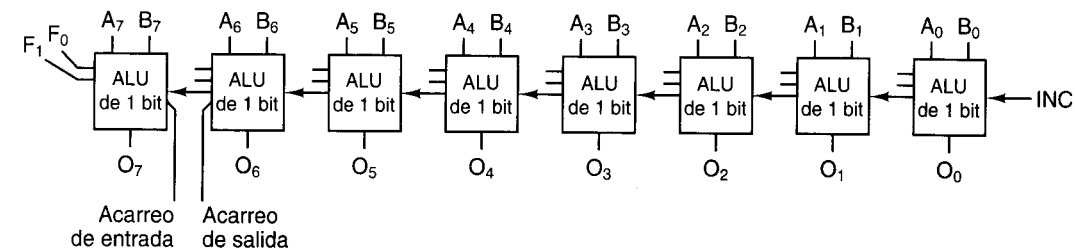


Figura 3-20. Ocho "tajadas" de ALU de un bit conectadas para formar una ALU de ocho bits. Por sencillez, no se muestran las señales de habilitación y de inversión.

### 3.2.4 Relojes

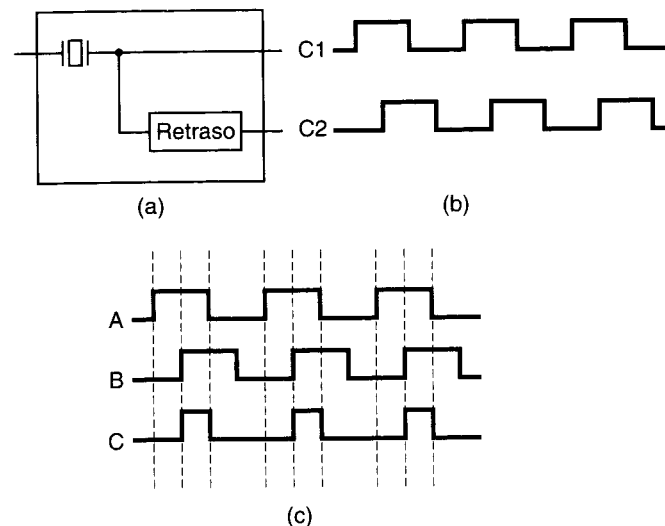
En muchos circuitos digitales el orden en que ocurren los sucesos es crucial. En ocasiones un suceso debe preceder a otro, a veces dos sucesos deben ocurrir simultáneamente. Para que los diseñadores puedan establecer las relaciones de temporización requeridas, muchos circuitos digitales emplean relojes que hacen posible la sincronización. Un **reloj** en este contexto es un circuito que emite una serie de pulsaciones con una anchura de pulsación precisa y un intervalo preciso entre pulsaciones consecutivas. El periodo entre los flancos correspondientes de dos pulsaciones consecutivas se denomina **tiempo de ciclo del reloj**. Las frecuencias de pulsación

suelen ser de 1 a 500 MHz, lo que corresponde a ciclos de reloj de 1000 ns a 2 ns. Para lograr gran exactitud, la frecuencia del reloj normalmente se controla con un oscilador de cristal.

En una computadora pueden ocurrir muchos sucesos durante un solo ciclo de reloj. Si dichos sucesos deben ocurrir en un orden específico, el ciclo de reloj debe dividirse en subciclos. Una forma común de obtener una definición más fina que la del reloj básico consiste en sacar una derivación de la línea primaria del reloj e insertar un circuito que tenga un retraso conocido. Esto genera una señal de reloj secundaria desfasada respecto a la primaria, como se muestra en la figura 3-21(a). El diagrama de temporización de la figura 3-21(b) proporciona cuatro referencias de tiempo para sucesos discretos:

1. Flanco ascendente de C1.
2. Flanco descendente de C1.
3. Flanco ascendente de C2.
4. Flanco descendente de C2.

Si se vinculan diferentes sucesos con los distintos flancos, se podrá lograr la sucesión requerida. Si se necesitan más de cuatro referencias de tiempo dentro de un ciclo de reloj, pueden derivarse más líneas secundarias de la primaria, con diferentes retardos.



**Figura 3-21.** (a) Un reloj. (b) Diagrama de temporización del reloj. (c) Generación de un reloj asimétrico.

En algunos circuitos lo que interesa son los periodos más que los instantes discretos en el tiempo. Por ejemplo, podría permitirse que ocurra cierto suceso en cualquier momento en que C1 esté alto, no precisamente en el flanco ascendente. Otro suceso podría suceder sólo cuando C2 está alto. Si se requieren más de dos intervalos, pueden proporcionarse más líneas

de reloj, o puede hacerse que los estados altos de los dos relojes se traslapen de manera parcial en el tiempo. En este último caso pueden distinguirse cuatro intervalos distintos:  $\overline{C1}$  AND  $\overline{C2}$ ,  $\overline{C1}$  AND C2, C1 AND  $\overline{C2}$  y C1 AND C2.

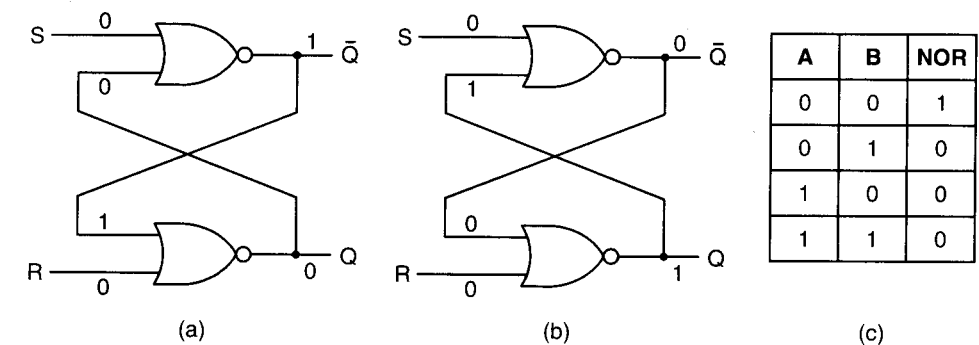
Por cierto, los relojes son simétricos en el sentido de que pasan tanto tiempo en el estado alto como en el estado bajo, como se muestra en la figura 3-21(b). Si se desea generar un tren de pulsaciones asimétrico, el reloj básico se desplaza utilizando un circuito de retraso y luego se hace AND con la señal original, como se muestra en la figura 3-21(c) para producir C.

### 3.3 MEMORIA

Un componente indispensable de toda computadora es su memoria. Sin memoria no podría haber computadoras como las que conocemos. La memoria sirve para almacenar las instrucciones que se van a ejecutar, y también los datos. En las secciones que siguen examinaremos los componentes básicos de un sistema de memoria comenzando desde el nivel de compuerta, para ver cómo funcionan y cómo se combinan para producir memorias grandes.

#### 3.3.1 Latches

Para crear una memoria de un bit necesitamos un circuito que de alguna manera “recuerde” los valores de entrada anteriores. Podemos construir un sistema así a partir de dos compuertas NOR, como se ilustra en la figura 3-22(a). También hay circuitos análogos que se basan en compuertas NAND, pero no los mencionaremos más porque son idénticos en lo conceptual a las versiones con NOR.



**Figura 3-22.** (a) Latch NOR en el estado 0. (b) Latch NOR en el estado 1. (c) Tabla de verdad para el NOR.

El circuito de la figura 3-22(a) se llama **latch SR**. Este circuito tiene dos entradas, *S* (*Set*) para establecer el latch (es decir, ponerlo en 1), y *R* (*Reset*) para restablecerlo (es decir, borrarlo o ponerlo en 0). También hay dos salidas, *Q* y  $\overline{Q}$ , que son complementarias, como veremos en breve. A diferencia de los circuitos combinacionales, las salidas de un latch no están determinadas de forma única por las entradas vigentes.

Para saber la razón, supongamos que tanto  $S$  como  $R$  son 0, que es lo más común. Por ahora, asumamos también que  $Q = 0$ . Puesto que  $Q$  se realimenta a la compuerta NOR superior, las dos entradas de ésta son 0, así que su salida,  $\bar{Q}$  es 1. El 1 se realimenta a la compuerta inferior, cuyas entradas son entonces 1 y 0, lo que da  $Q = 0$ . Este estado es al menos consistente y se muestra en la figura 3-22(a).

Imaginemos ahora que  $Q$  no es 0, sino 1, y que  $R$  y  $S$  siguen siendo 0. Las entradas de la compuerta superior son 0 y 1, así que produce una salida  $\bar{Q}$  de 0, la cual se realimenta a la compuerta inferior. Este estado, que se muestra en la figura 3-22(b), también es consistente. Un estado en el que ambas salidas son iguales a 0 es inconsistente, porque obliga a ambas compuertas a tener dos ceros como entradas, y ello produciría una salida de 1, no de 0. Asimismo, es imposible que ambas salidas sean iguales a 1, porque eso obligaría a que las entradas fueran 0 y 1, lo que produce 0, no 1. Nuestra conclusión es sencilla: con  $R = S = 0$ , el latch tiene dos estados estables, a los que llamaremos 0 y 1, que dependen de  $Q$ .

Examinemos ahora el efecto de las entradas sobre el estado del latch. Supongamos que  $S$  se vuelve 1 mientras  $Q = 0$ . Las entradas de la compuerta superior son entonces 1 y 0, y esto hace que la salida  $\bar{Q}$  sea 0. Este cambio hace que las dos entradas de la compuerta inferior sean 0, y por tanto que su salida sea 1. Así, establecer  $S$  (o sea, ponerlo en 1) conmuta el estado de 0 a 1. Poner  $R$  en 1 cuando el latch está en el estado 0 no tiene ningún efecto porque la salida de la compuerta NOR inferior es 0 tanto para las entradas 10 como para 11.

Por un razonamiento similar, es fácil ver que poner  $S$  en 1 mientras  $Q = 1$  no surte ningún efecto, pero poner  $R$  en 1 hace que el latch pase al estado  $Q = 0$ . En síntesis, cuando  $S$  se pone en 1 momentáneamente, el latch termina en el estado  $Q = 1$ , sin importar cuál haya sido su estado anterior. De igual manera, poner  $R$  en 1 hace que el latch pase al estado  $Q = 0$ . El circuito "recuerda" si fue  $S$  o  $R$  la última que estuvo en 1. Podemos aprovechar esta propiedad para construir memorias de computadora.

### Latches SR con reloj

En muchos casos es conveniente impedir que el latch cambie de estado como no sea en ciertos momentos específicos. Para ello, modificamos un poco el circuito básico, como se muestra en la figura 3-23, para obtener un **latch SR con reloj**.

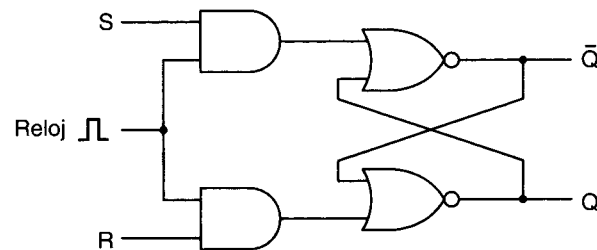


Figura 3-23. Latch SR con reloj.

Este circuito tiene una entrada adicional, el reloj, que normalmente es 0. Con el reloj en 0, ambas compuertas AND producen 0, sean cuales sean los valores de  $S$  y  $R$ , y el latch no cambia

de estado. Cuando el reloj está en 1, el efecto de las compuertas AND desaparece y el latch es sensible a  $S$  y  $R$ . A pesar de su nombre, la señal de reloj no tiene que provenir de un reloj. También se usan ampliamente los términos **enable** (habilitar) y **strobe** (señal estroboscópica) para indicar que la entrada de reloj es 1; es decir, que el circuito es sensible al estado de  $S$  y  $R$ .

Hasta ahora hemos evitado considerar el problema que sucede cuando tanto  $S$  como  $R$  son 1, y hemos tenido una buena razón para hacerlo: el circuito se vuelve no determinista cuando por fin  $R$  y  $S$  regresan a 0. El único estado consistente para  $S = R = 1$  es  $Q = \bar{Q} = 0$ , pero tan pronto como ambas entradas regresen a 0 el latch deberá saltar a uno de sus dos estados estables. Si una de las dos entradas regresa a 0 antes que la otra, la que se quedó en 1 más tiempo gana, porque cuando sólo una de las entradas es 1, determina el estado del latch. Si ambas entradas regresan a 0 al mismo tiempo (lo cual es muy poco probable), el latch salta a uno de sus estados estables al azar.

### Latches D con reloj

Una buena forma de resolver la ambigüedad del latch SR (que surge cuando  $S = R = 1$ ) es evitar que ocurra. La figura 3-24 muestra un circuito de latch que sólo tiene una entrada,  $D$ . Puesto que la entrada a la compuerta AND inferior siempre es el complemento de la entrada de la compuerta AND superior, nunca surge el problema de que ambas entradas sean 1. Cuando  $D = 1$  y el reloj es 1, el latch pasa al estado  $Q = 1$ . Cuando  $D = 0$  y el reloj es 1, el latch pasa al estado  $Q = 0$ . En otras palabras, cuando el reloj es 1, se muestrea el valor actual de  $D$  y se almacena en el latch. Este circuito, llamado **latch D con reloj**, es una verdadera memoria de un bit. El valor almacenado siempre está disponible en  $Q$ . Para cargar el valor actual de  $D$  en la memoria, se aplica una pulsación positiva a la línea del reloj.

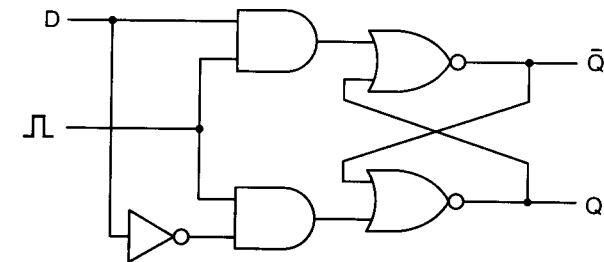


Figura 3-24. Latch D con reloj.

Este circuito requiere 11 transistores. Circuitos más avanzados (pero menos obvios) pueden almacenar 1 bit con menos transistores, hasta un mínimo de seis. En la práctica éstos son los diseños normalmente usados.

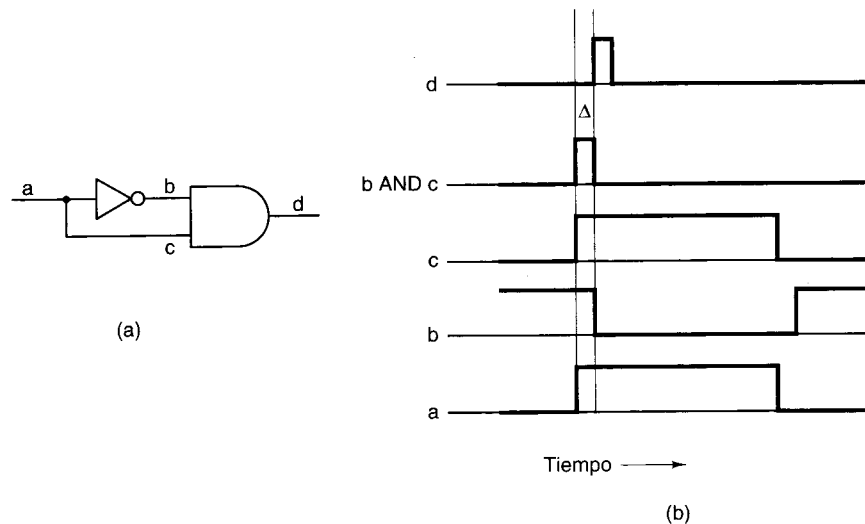
### 3.3.2 Flip-flops

En muchos circuitos es necesario muestrear el valor que hay en una línea dada en un instante dado, y almacenarlo. En esta variante, llamada **flip-flop**, la transición de estado no ocurre cuando el reloj es 1, sino durante la transición del reloj de 0 a 1 (flanco ascendente) o de 1 a

0 (flanco descendente). Así, la longitud del pulso de reloj no importa, en tanto las transiciones sean rápidas.

Conviene hacer hincapié en la diferencia entre un flip-flop y un latch. Un flip-flop **se dispara por flanco**, mientras que un latch **se dispara por nivel**. Cabe advertir, empero, que en la literatura hay veces en que se confunden estos términos. Muchos autores usan “flip-flop” cuando se refieren a un latch, y viceversa.

Hay varias formas de diseñar un flip-flop. Por ejemplo, si hubiera alguna forma de generar una pulsación muy corta en el flanco ascendente de la señal del reloj, esa pulsación podría alimentarse a un latch D. De hecho, sí hay una forma de hacerlo, y el circuito correspondiente se muestra en la figura 3-25(a).

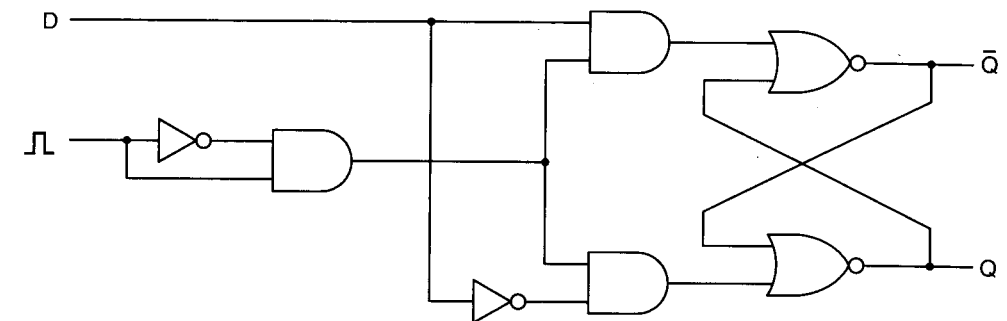


**Figura 3-25.** (a) Generador de pulsaciones. (b) Temporización en cuatro puntos del circuito.

A primera vista, podría parecer que la salida de la compuerta AND siempre sería cero, ya que el AND de cualquier señal con su inverso es cero, pero la situación es un poco más compleja. El inversor tiene un retraso de propagación pequeño, pero distinto de cero, y ese retraso es lo que hace que el circuito funcione. Suponga que medimos el voltaje en los cuatro puntos de medición *a*, *b*, *c* y *d*. La señal de entrada, medida en *a*, es un pulso largo de reloj, como se muestra en la figura 3-25(b) en la parte inferior. Arriba se muestra la señal en *b*. Observe que está invertida y también tiene un pequeño retraso, por lo regular de unos cuantos nanosegundos dependiendo del tipo de inversor que se use.

La señal en *c* también tiene un retraso, pero éste es sólo por el tiempo de propagación de la señal (a la velocidad de la luz). Si la distancia física entre *a* y *c* es, por ejemplo, de 20 micras, el retraso de propagación es de 0.0001 ns, que sin duda es insignificante en comparación con el tiempo que la señal tarda en propagarse a través del inversor. Por ello, en la práctica podemos considerar que la señal en *c* es idéntica a la señal en *a*.

Cuando se obtiene el AND de las entradas de la compuerta AND, *b* y *c*, el resultado es un pulso corto, como se muestra en la figura 3-25(b), donde la anchura de la pulsación,  $\Delta$ , es igual al retardo de compuerta del inversor, que por lo regular es de 5 ns o menos. La salida de la compuerta AND no es más que esta pulsación desplazada por el retraso de la compuerta AND, como se aprecia en la parte superior de la figura 3-25(b). Este desplazamiento en el tiempo sólo implica que el latch D se activará con un retraso fijo después del flanco ascendente del reloj, pero no tiene efecto sobre la anchura de la pulsación. En una memoria con un tiempo de ciclo de 50 ns, una pulsación de 5 ns que le indica cuándo debe muestrear la línea *D* podría ser lo bastante corta, en cuyo caso el circuito completo puede ser el de la figura 3-26. Vale la pena señalar que este diseño de flip-flop es bonito porque es fácil de entender, pero en la práctica normalmente se usan flip-flops más complejos.



**Figura 3-26.** Flip-flop D.

Los símbolos estándar para los latches y flip-flops se muestran en la figura 3-27. La figura 3-27(a) es un latch cuyo estado se carga cuando el reloj, *CK*, es 1, en contraste con la figura 3-27(b), que es un latch cuyo reloj normalmente es 1 pero baja a 0 momentáneamente para cargar el estado de *D*. Las figuras 3-27(c) y (d) son flip-flops, no latches, lo que se indica con el símbolo angular en la entrada del reloj. El de la figura 3-27(c) cambia de estado con el flanco ascendente del pulso de reloj (transición 0 a 1), mientras que el de la figura 3-27(d) cambia de estado con el flanco descendente (transición 1 a 0). Muchos latches y flip-flops, pero no todos, también tienen  $\bar{Q}$  como salida, y algunos tienen dos entradas adicionales, *Establecer* o *Preestablecer* (*Set* o *Preset*, para forzar el estado a  $Q = 1$ ) y *Restablecer* o *Borrar* (*Reset* o *Clear*, para forzar el estado a  $Q = 0$ ).

### 3.3.3 Registros

Los flip-flops se pueden conseguir en diversas configuraciones. Una de las más sencillas, que contiene dos flip-flops D independientes con señales de borrar y preestablecer, se ilustra en la figura 3-28(a). Aunque se empaquetan en el mismo chip de 14 terminales, los dos flip-flops son independientes. Una configuración muy distinta es el flip-flop octal de la figura 3-28(b). Aquí los ocho (de ahí el término “octal”) flip-flops D no sólo carecen de las líneas  $\bar{Q}$  y *preset*, sino que todas las líneas de reloj se juntan y se alimentan de la terminal 11. Los flip-flops

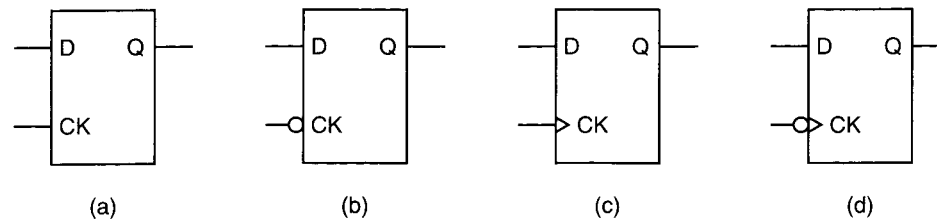


Figura 3-27. Latches y flip-flops D.

Los mismos son del tipo del de la figura 3-27(d), pero las burbujas de inversión en los flip-flops se cancelan con el inversor conectado a la terminal 11, de modo que los flip-flops se cargan durante la transición ascendente. Las ocho señales de borrar también se juntan, de modo que cuando la terminal 1 baja a 0 todos los flip-flops se fuerzan a su estado 0. Por si se está preguntando por qué la terminal 11 se invierte en la entrada y luego se invierte otra vez en cada señal CK, una señal de entrada podría no tener la suficiente corriente para alimentar a los ocho flip-flops; el inversor de entrada se está usando en realidad como amplificador.

Si bien una razón para juntar las líneas de reloj y de borrar en la figura 3-28(b) es ahorrar terminales, en esta configuración el chip se usa de forma diferente de como se usarían ocho flip-flops independientes: se utiliza como un solo registro de ocho bits. También podrían usarse dos de estos chips en paralelo para formar un registro de 16 bits conectando sus respectivas terminales 1 y 11. Examinaremos los registros y sus aplicaciones con mayor detalle en el capítulo 4.

### 3.3.4 Organización de la memoria

Aunque ya avanzamos de la sencilla memoria de un bit de la figura 3-24 a la memoria de ocho bits de la figura 3-28(b), para construir memorias grandes se requiere una organización diferente, en la cual sea posible direccionar palabras individuales. Una organización de memoria muy utilizada que satisface este criterio se muestra en la figura 3-29. Este ejemplo ilustra una memoria con cuatro palabras de 3 bits. Cada operación lee o escribe una palabra de 3 bits completa. Si bien la capacidad total de memoria de 12 bits apenas rebasa la de nuestro flip-flop octal, este diseño requiere menos terminales y, lo que es más importante, es fácil extenderla a memorias grandes.

Aunque la memoria de la figura 3-29 podría parecer complicada a primera vista, en realidad es muy sencilla gracias a su estructura regular. Este circuito tiene ocho líneas de entrada y tres líneas de salida. Tres de las entradas son datos:  $I_0$ ,  $I_1$  e  $I_2$ ; dos son para la dirección:  $A_0$  y  $A_1$ ; y tres son de control:  $CS$  para seleccionar el chip (*Chip Select*),  $RD$  para distinguir entre lectura (*Read*) y escritura, y  $OE$  para habilitar la salida (*Output Enable*). Las tres salidas son de datos:  $D_0$ ,  $D_1$  y  $D_2$ . En principio, esta memoria podría colocarse en un paquete de 14 terminales, incluida la de potencia y la de tierra, en comparación con 20 terminales para el flip-flop octal.

Para seleccionar este chip de memoria, la lógica externa debe poner  $CS$  en 1 y también poner  $RD$  en 1 si se desea leer o en 0 si se desea escribir. Las dos líneas de dirección deben ajustarse de modo que indiquen cuál de las cuatro palabras de 3 bits se desea leer o escribir.

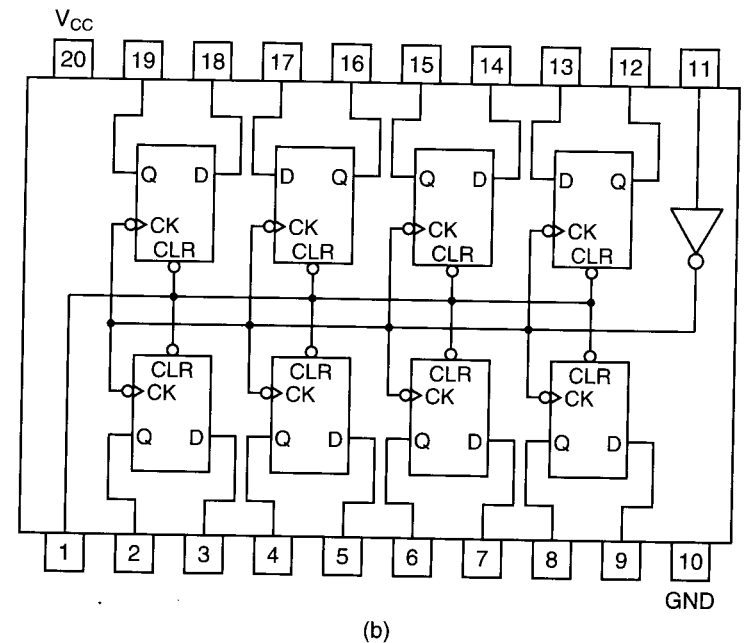
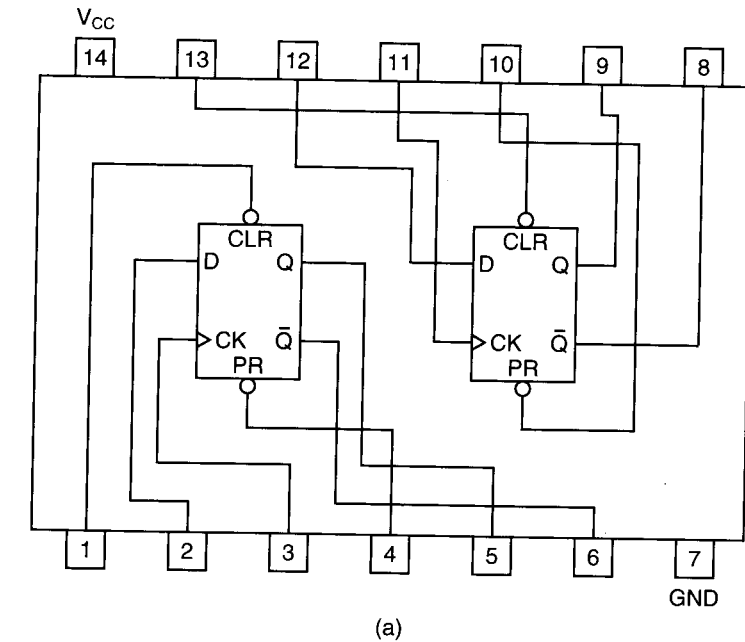
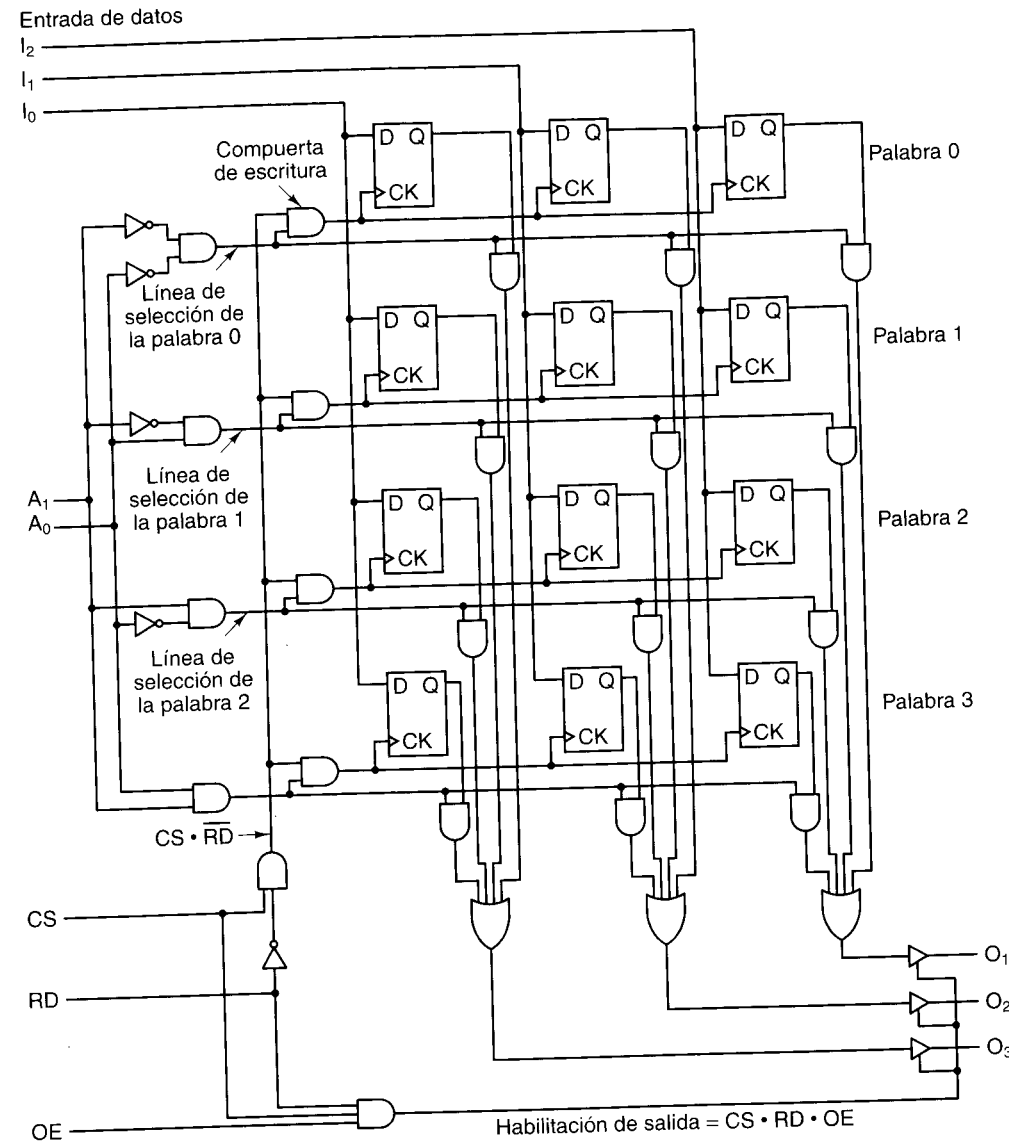


Figura 3-28. (a) Flip-flop D dual. (b) Flip-flop octal.



**Figura 3-29.** Diagrama lógico de una memoria 4 x 3. Cada renglón es una de las cuatro palabras de 3 bits. Una operación de lectura o escritura siempre lee o escribe una palabra completa.

En el caso de una operación de lectura, las líneas de entrada de datos no se usan, pero la palabra seleccionada se coloca en las líneas de salida de datos. Para una operación de escritura, los bits presentes en las líneas de entrada de datos se cargan en la palabra de memoria seleccionada; las líneas de salida de datos no se usan.

Ahora examinemos con detenimiento la figura 3-29 para observar el modo de funcionamiento. Las cuatro compuertas AND de selección de palabra a la izquierda de la memoria forman un decodificador. Los inversores de entrada se colocaron de modo que cada compuerta se habilite (su salida sea alta) con una dirección diferente. Cada compuerta alimenta una línea de selección de palabra, de arriba hacia abajo, para las palabras 0, 1, 2 y 3. Si el chip se seleccionó para escritura, la línea vertical rotulada  $CS \cdot \overline{RD}$  estará alta y habilitará una de las cuatro compuertas de escritura, dependiendo de cuál línea de selección de palabra esté alta. La salida de la compuerta de escritura alimenta todas las señales CK para la palabra seleccionada, cargando los datos de entrada en los flip-flops de esa palabra. Sólo se efectúa una escritura si CS está alta y RD está baja, y aun entonces sólo se escribe la palabra seleccionada por  $A_0$  y  $A_1$ ; las demás palabras no se modifican.

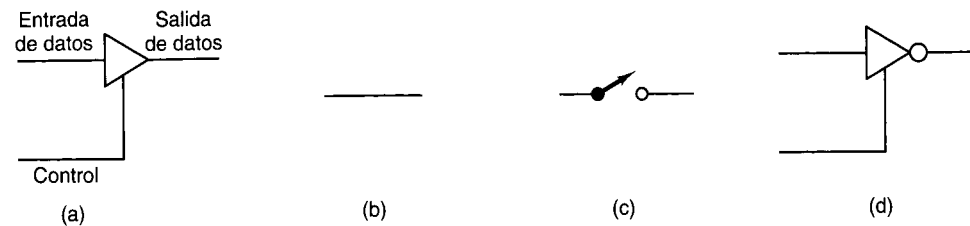
La lectura es similar a la escritura. La decodificación de la dirección es exactamente la misma que en una escritura, pero ahora la línea  $CS \cdot \overline{RD}$  está baja, lo que deshabilita todas las compuertas de escritura y ninguno de los flip-flops se modifica. En vez de ello, la línea de selección de palabra habilita las compuertas AND conectadas a los bits Q de la palabra seleccionada. Así, la palabra seleccionada pasa sus datos a las compuertas OR de cuatro entradas que están en la parte inferior de la figura, mientras que las otras tres palabras pasan ceros. Por consiguiente, la salida de las compuertas OR es idéntica al valor almacenado en la palabra seleccionada. Las tres palabras que no se seleccionaron no contribuyen a la salida.

Aunque podríamos haber diseñado un circuito en el que las tres compuertas OR se alimentaran directamente a las tres líneas de datos de salida, hacerlo así a veces causa problemas. En particular, hemos mostrado que las líneas de entrada de datos y las de salida de datos son diferentes, pero en las memorias reales se emplean las mismas líneas. Si hubiéramos conectado las compuertas OR a las líneas de salida de datos, el chip trataría de producir datos (o sea, forzar cada línea a tener un valor específico) aun en las escrituras, y esto interferiría los datos de entrada. Por esta razón, es deseable tener una forma de conectar las compuertas OR a las líneas de salida de datos durante las lecturas pero desconectarlas totalmente durante las escrituras. Lo que necesitamos es un conmutador electrónico que puede establecer o romper una conexión en unos cuantos nanosegundos.

Por fortuna, existen tales conmutadores. En la figura 3-30(a) se muestra el símbolo de un **buffer no inversor**. Este buffer tiene una entrada de datos, una salida de datos y una entrada de control. Cuando la entrada de control está en estado alto, el buffer actúa como un alambre, como se muestra en la figura 3-30(b). Cuando la entrada de control está baja, el buffer actúa como un circuito abierto, como se muestra en la figura 3-30(c); es como si alguien desconectara la salida de datos del resto del circuito con una pinza de corte. La diferencia respecto a la técnica de la pinza de corte es que la conexión se puede restaurar después en unos cuantos nanosegundos con sólo hacer que la señal de control sea alta otra vez.

La figura 3-30(d) muestra un **buffer inversor**, que actúa como un inversor normal cuando la señal de control es alta y desconecta la salida del circuito cuando la señal de control es baja. Ambos tipos de buffers son **dispositivos de tres estados**, porque pueden producir 0, 1 o ninguna de las dos (circuito abierto). Los buffers también amplifican las señales, por lo que pueden alimentar a muchas entradas simultáneamente. En muchos casos se usan buffers en los circuitos por esta razón, aun cuando no se necesitan sus propiedades de interrupción.





**Figura 3-30.** (a) Buffer no inversor. (b) Efecto de (a) cuando el control está alto. (c) Efecto de (a) cuando el control está bajo. (d) Buffer inversor.

Volviendo al circuito de memoria, ahora deberá quedar claro para qué sirven los tres buffers no inversores en las líneas de salida de datos. Cuando CS, RD y OE están todas altas, la señal de habilitación de salida también está alta; esta señal habilita los buffers, y se coloca una palabra en las líneas de salida. Cuando alguna de las señales CS, RD u OE está baja, las salidas de datos se desconectan del resto del circuito.

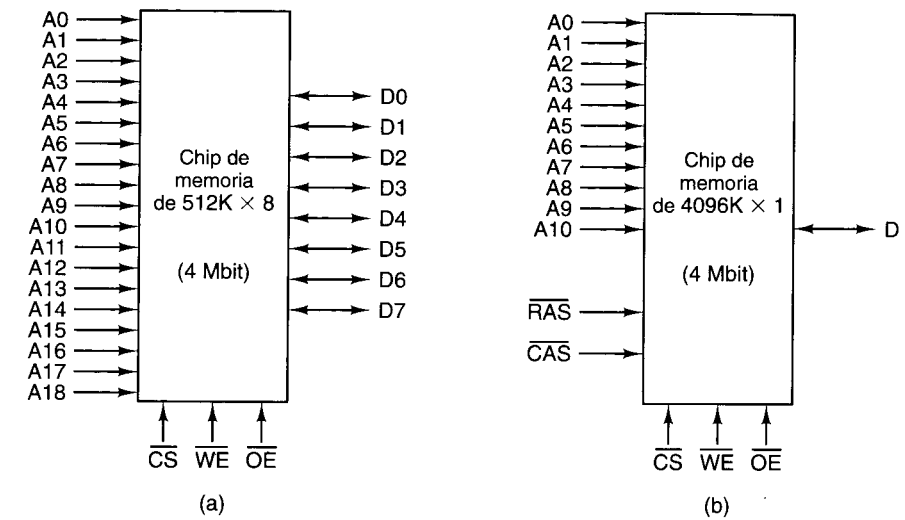
### 3.3.5 Chips de memoria

Lo bueno de la memoria de la figura 3-29 es que fácilmente se puede extender a un mayor tamaño. Tal como la dibujamos, la memoria es de  $4 \times 3$ , es decir, cuatro palabras de tres bits cada una. Para extenderla a  $4 \times 8$  basta con añadir cinco columnas más de cuatro flip-flops cada una, además de otras cinco líneas de entrada y otras cinco líneas de salida. Para convertir esta memoria de  $4 \times 3$  a  $8 \times 3$  necesitaremos añadir otras cuatro filas de tres flip-flops cada una, además de una línea de dirección  $A_2$ . Con este tipo de estructura, el número de palabras en la memoria debe ser una potencia de 2 si se desea una eficiencia máxima, pero el número de bits en una palabra puede ser el que se desee.

Puesto que la tecnología de circuitos integrados se presta a fabricar chips cuya estructura interna sea un patrón bidimensional repetitivo, los chips de memoria son una aplicación ideal de esa tecnología. Al mejorar la tecnología, el número de bits que se pueden colocar en un chip aumenta continuamente, duplicándose por lo regular cada 18 meses (ley de Moore). Los chips más grandes no siempre hacen que los más pequeños se vuelvan obsoletos, debido a las diferentes características de capacidad, rapidez, potencia, precio y facilidad de conexión. En general, los chips más grandes en el mercado tienen precios inflados y por tanto son más costosos por bit que los chips más viejos y pequeños.

Para un tamaño de memoria dado, hay diversas formas de organizar el chip. La figura 3-31 muestra dos posibles organizaciones de un chip de 4 Mbit:  $512K \times 8$  y  $4096 \times 1$ . (Por cierto, los tamaños de los chips de memoria generalmente se dan en bits, no en bytes, y adoptaremos esa convención aquí.) En la figura 3-31(a) se requieren 19 líneas de dirección para direccionar uno de los  $2^{19}$  bytes, y se requieren ocho líneas de datos para cargar o almacenar el byte seleccionado.

Es pertinente incluir aquí una nota sobre terminología. En algunas terminales, el voltaje alto hace que ocurra alguna acción. En otras, el voltaje bajo es el que causa la acción. Para evitar confusiones, diremos siempre que una señal se **habilita o aserta** (en lugar de decir que



**Figura 3-31.** Dos formas de organizar un chip de memoria de 4 Mbit.

se vuelve alta o se vuelve baja) para indicar que se ajusta de modo que cause alguna acción. Así, en algunas terminales, habilitar la señal significa hacerla alta; en otras, significa hacerla baja. Las terminales que se habilitan bajando la señal reciben nombres que llevan una barra o testa arriba. Por tanto, una señal llamada CS se habilita llevándola a nivel alto, pero una llamada  $\overline{CS}$  se habilita llevándola a nivel bajo. Lo contrario de habilitar es **deshabilitar**. Cuando no está sucediendo nada especial, las terminales están deshabilitadas.

Volvamos ahora a nuestro chip de memoria. Puesto que una computadora normalmente tiene muchos chips de memoria, se requiere una señal para seleccionar el chip que se necesita en ese momento, de modo que responda y los demás no lo hagan. Se incluye la señal  $\overline{CS}$  (Chip Select) para este fin. La señal se habilita para activar el chip. Además, se necesita una forma de distinguir las lecturas de las escrituras. La señal  $\overline{WE}$  (Write Enable, habilitar escritura) sirve para indicar que se van a escribir datos, no a leerse. Por último, la señal  $\overline{OE}$  (Output Enable) se habilita para alimentar las señales de salida; si no está habilitada, la salida del chip se desconecta del circuito.

En la figura 3-31(b) se emplea un esquema de direccionamiento distinto. Internamente, este chip está organizado como una matriz  $2048 \times 2048$  de celdas de un bit, lo que da 4 Mbit. Para direccionar el chip, primero se selecciona una localidad (renglón) colocando su número de 11 bits en las terminales de dirección. Luego se habilita la señal  $\overline{RAS}$  (Row Address Strobe, habilitar dirección de fila). Después, se coloca un número de columna en las terminales de dirección y se habilita  $\overline{CAS}$  (Column Address Strobe, habilitar dirección de columna). El chip responde aceptando o produciendo un bit de datos.

Los chips de memoria grandes a menudo se construyen como matrices  $n \times n$  que se direccionan por renglón y columna. Esta organización reduce el número de terminales requeridas pero también hace más lento el direccionamiento del chip, ya que se necesitan dos ciclos de direccionamiento, uno para el renglón y otro para la columna. Para recuperar algo de la velocidad que se pierde por este diseño, algunos chips de memoria pueden aceptar una

dirección de fila seguida de una sucesión de direcciones de columna para acceder bits consecutivos de un renglón.

Hace años, los chips de memoria más grandes en general estaban organizados como la figura 3-31(b). Ahora que las palabras de memoria han crecido de 8 bits a 32 bits o más, los chips con una anchura de un bit dejaron de ser prácticos. Para construir una memoria con palabras de 32 bits a partir de chips de  $4096K \times 1$  se requieren 32 chips en paralelo. Estos 32 chips tienen una capacidad total de por lo menos 16 MB. En cambio, si se usan chips de  $512K \times 8$  sólo se requieren cuatro chips en paralelo, y pueden tenerse memorias pequeñas, de 2 MB o más. Para evitar tener 32 chips de memoria, casi todos los fabricantes de chips producen familias con anchuras de 1, 4, 8 y 16 bits.

### 3.3.6 Las memorias RAM y las ROM

Todas las memorias que hemos estudiado hasta aquí pueden leerse y escribirse. Tales memorias se llaman **RAM** (memoria de acceso aleatorio, *Random Access Memory*), aunque el nombre es engañoso porque es posible acceder aleatoriamente a todos los chips, no sólo los de RAM, pero el término ya está demasiado bien establecido como para deshacerse de él a estas alturas. Hay dos variedades de RAM, estática y dinámica. Una **RAM estática (SRAM, Static RAM)** se construye internamente empleando circuitos similares a nuestro flip-flop D básico. Estas memorias tienen la propiedad de que su contenido se conserva en tanto se sigue alimentando el circuito: segundos, minutos, horas, y hasta días. Las RAM estáticas son muy rápidas. Un tiempo de acceso típico es de unos cuantos nanosegundos. Por esta razón, las SRAM son populares como memoria caché de nivel 2.

Las **RAM dinámicas (DRAM)**, en cambio, no usan flip-flops. En vez de ello, una RAM dinámica es una matriz de celdas, cada una de las cuales contiene un transistor y un diminuto condensador. Los condensadores pueden cargarse o descargarse, lo que permite almacenar ceros y unos. Puesto que la carga eléctrica tiende a fugarse, cada bit de una RAM dinámica debe **refrescarse** (cargarse de nuevo) cada pocos milisegundos para evitar que los datos se pierdan. Puesto que el proceso de refresco corre a cargo de la lógica externa, las RAM dinámicas requieren interfaces más complejas que las estáticas, aunque en muchas aplicaciones esta desventaja se compensa por su mayor capacidad.

Puesto que las RAM dinámicas sólo necesitan un transistor y un condensador por bit (en comparación con seis transistores por bit para la mejor RAM estática), las RAM dinámicas tienen una densidad muy alta (muchos bits por chip). Por esta razón, las memorias principales casi siempre se construyen con RAM dinámicas. Sin embargo, esta gran capacidad tiene un precio: las RAM dinámicas son lentas (decenas de nanosegundos). La combinación de una caché de RAM estática y una memoria principal de RAM dinámica intenta combinar las propiedades ventajosas de cada una.

Existen varios tipos de chips de RAM dinámica. El tipo más antiguo que continúa en uso es la **DRAM FPM (modo de página rápida, Fast Page Mode)**. Internamente, la organización es una matriz de bits; el hardware presenta una dirección de renglón y luego recorre las direcciones de columna, como describimos con  $\overline{RAS}$  y  $\overline{CAS}$  en el contexto de la figura 3-31(b).

La DRAM FPM está siendo reemplazada gradualmente por la **DRAM edo (salida de datos extendida, Extended Data Output)**, que permite iniciar una segunda referencia a la

memoria antes de que la anterior se haya completado. Este sencillo uso de conductos no hace que una sola referencia a la memoria sea más rápida, pero sí mejora el ancho de banda de la memoria, que produce más palabras por segundo.

Tanto los chips FPM como los EDO son asincrónicos, lo que significa que las líneas de dirección y de datos no están controladas por un mismo reloj. En contraste, la **SDRAM (DRAM sincrónica)** es un híbrido de RAM estática y dinámica y es controlado por un solo reloj sincrónico. La SDRAM se usa a menudo en las cachés grandes y podría ser la tecnología preferida para las memorias principales en el futuro.

Las RAM no son el único tipo de chips de memoria. En muchas aplicaciones, como juguetes, aparatos domésticos y automóviles, el programa y algunos de los datos deben conservarse en la memoria aunque se interrumpa el suministro de electricidad. Además, una vez instalados, ni el programa ni los datos llegan a modificarse. Estos requisitos han dado pie a la creación de las **ROM** (memorias sólo de lectura, *Read-Only Memories*), que no pueden modificarse ni borrarse, intencionalmente o por accidente. Los datos de una ROM se insertan durante su fabricación, básicamente exponiendo un material fotosensible a través de una máscara que contiene el patrón de bits deseado y luego eliminando por grabado la superficie expuesta (o la no expuesta). La única forma de modificar el programa de una ROM es cambiar todo el chip.

Las ROM son mucho más económicas que las RAM cuando se producen en grandes cantidades, pues así el costo de elaborar la máscara se diluye. Sin embargo, estas memorias son inflexibles, porque no pueden modificarse después de su fabricación, y el tiempo que tarda en surtirse un pedido de ROM puede ser de semanas. A fin de facilitar el desarrollo de productos nuevos basados en ROM, se inventó la **PROM** (ROM programable, *Programmable ROM*). Una PROM es parecida a una ROM, excepto que puede programarse (una vez) en el campo, lo que elimina el retraso por surtido. Muchas PROM contienen una matriz de diminutos fusibles. Un fusible específico puede quemarse seleccionando su renglón y columna y aplicando después un voltaje alto a una terminal especial del chip.

El siguiente avance en este campo fue la **EPROM** (PROM borrrable, *Erasable PROM*), que no sólo puede programarse en el campo, sino también borrarse en el campo. Cuando la ventana de cuarzo de una EPROM se expone a una luz ultravioleta intensa durante 15 minutos, todos los bits se ponen en 1. Si se esperan muchos cambios durante el ciclo de diseño, las EPROM son mucho más económicas que las PROM porque pueden reutilizarse. Las EPROM suelen tener la misma organización que las RAM estáticas. La EPROM 27C040 de 4 bits, por ejemplo, emplea la organización de la figura 3-31(a), que es típica de una RAM estática.

Aun mejor que la EPROM es la **EEPROM**, que puede borrarse por aplicación de pulsaciones, sin tener que colocarse en una cámara especial para exponerla a luz ultravioleta. Además, una EEPROM puede reprogramarse instalada, mientras que una EPROM tiene que insertarse en un dispositivo especial para programar las EPROM. En el aspecto negativo, las EEPROM suelen ser mucho más pequeñas (por lo regular 1/64 que las EPROM comunes) y dos veces más lentas. Las EEPROM no pueden competir con las DRAM ni las SRAM porque son 10 veces más lentas, 100 veces más pequeñas y mucho más costosas; sólo se utilizan en situaciones en las que su no volatilidad es crucial.

Una clase de EEPROM más reciente es la **memoria flash**. A diferencia de la EPROM, que se borra por exposición a luz ultravioleta, y la EEPROM, que se puede borrar por byte, la

memoria flash puede borrarse y reescribirse por bloque. Al igual que la EEPROM, la memoria flash puede borrarse sin sacarla del circuito. Varios fabricantes producen pequeñas tarjetas de circuitos impresos con decenas de megabytes de memoria flash para usarse como cinta de video y almacenar imágenes en cámaras digitales, además de muchos otros usos. Algún día podrán usarse las memorias flash para sustituir a los discos, lo cual representaría un avance importante, en vista de sus tiempos de acceso de 100 ns. El principal problema de ingeniería en la actualidad es que las memorias flash se desgastan después de unos 10,000 borrados, mientras que los discos duran años, por más frecuentemente que se reescriban. En la figura 3-32 se presenta un resumen de los diversos tipos de memoria.

| Tipo   | Categoría         | Borrado   | Alterable por byte | Volátil | Uso típico                   |
|--------|-------------------|-----------|--------------------|---------|------------------------------|
| SRAM   | Lectura/escritura | Eléctrico | Sí                 | Sí      | Caché nivel 2                |
| DRAM   | Lectura/escritura | Eléctrico | Sí                 | Sí      | Memoria principal            |
| ROM    | Sólo lectura      | Imposible | No                 | No      | Aparatos en vol. grandes     |
| PROM   | Sólo lectura      | Imposible | No                 | No      | Equipos en vol. pequeños     |
| EPROM  | Princ. lectura    | Luz UV    | No                 | No      | Prototipos de dispositivos   |
| EEPROM | Princ. lectura    | Eléctrico | Sí                 | No      | Prototipos de dispositivos   |
| Flash  | Lectura/escritura | Eléctrico | No                 | No      | Película para cámara digital |

Figura 3-32. Comparación de diversos tipos de memoria.

3.4 CHIPS DE CPU Y BUSES

Ahora que poseemos información acerca de los chips SSI, MSI y de memoria, podemos comenzar a juntar todas las piezas y examinar sistemas completos. En esta sección, examinaremos primero algunos aspectos generales de las CPU desde la perspectiva del nivel de lógica digital, incluida la **conexión de terminales** o **pinout** (el significado de las señales de las distintas terminales). Puesto que las CPU están íntimamente relacionadas con el diseño de los buses que usan, también proporcionaremos en esta sección una introducción al diseño de buses. En secciones posteriores daremos ejemplos detallados de CPU y de sus buses.

3.4.1 Chips de CPU

Todas las CPU modernas están contenidas en un solo chip. Esto hace que su interacción con el resto del sistema esté bien definida. Cada chip de CPU tiene un conjunto de terminales, a través de las cuales debe pasar toda su comunicación con el mundo exterior. Algunas terminales proporcionan señales de la CPU; otras aceptan señales del mundo exterior; algunas pueden hacer las dos cosas. Si entendemos la función de todas las terminales, podremos conocer cómo la CPU interactúa con la memoria y los dispositivos de E/S en el nivel de lógica digital.

Las terminales de un chip de CPU se pueden dividir en tres tipos: dirección, datos y control. Estas terminales se conectan a terminales similares de los chips de memoria y de E/S a través de una colección de alambres paralelos llamado bus. Para buscar una instrucción, la CPU primero coloca la dirección de memoria de esa instrucción en sus terminales de dirección, y luego habilita una o más líneas de control para informar a la memoria que quiere leer (por ejemplo) una palabra. La memoria contesta colocando la palabra solicitada en las terminales de datos de la CPU y habilitando una señal para indicar que ya terminó. Cuando la CPU ve esta señal, acepta la palabra y ejecuta la instrucción.

La instrucción podría requerir leer o escribir palabras de datos, en cuyo caso se repite todo el proceso para cada palabra adicional. Más adelante veremos los detalles de cómo funciona la lectura y la escritura. Por ahora, lo que es importante entender es que la CPU se comunica con la memoria y los dispositivos de E/S tanto transmitiendo como recibiendo señales en sus terminales. No es posible ninguna otra comunicación.

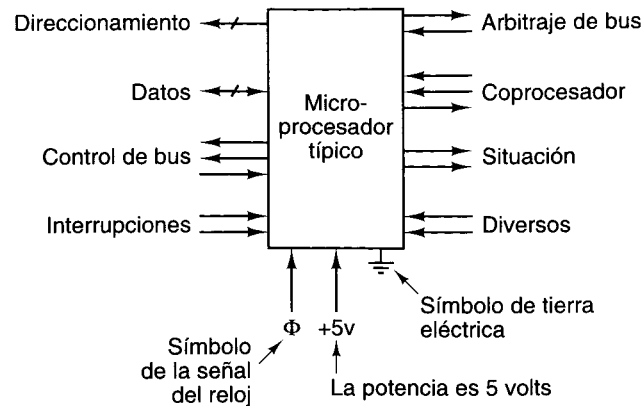
Dos de los parámetros fundamentales que determinan el desempeño de una CPU son el número de líneas de dirección y el número de líneas de datos. Un chip que tiene *m* líneas de dirección puede direccionar hasta 2<sup>*m*</sup> localidades de memoria. Los valores comunes de *m* son 16, 20, 32 y 64. De igual manera, un chip con *n* líneas de datos puede leer o escribir una palabra de *n* bits en una sola operación. Los valores comunes de *n* son 8, 16, 32, 36 y 64. Una CPU con ocho líneas de datos requiere cuatro operaciones para leer una palabra de 32 bits, mientras que una con 32 líneas de datos puede realizar la misma tarea en una sola operación. Así, el chip con 32 terminales de datos es mucho más rápido, pero siempre es más costoso.

Además de las líneas de dirección y de datos, toda CPU tiene algunas líneas de control, las cuales regulan el flujo y la temporización de los datos que entran en la CPU y salen de ella, además de tener varios otros usos. Todas las CPU tienen terminales para la alimentación (casi siempre +3.3 volts o + 5 volts), tierra y una señal de reloj (una onda cuadrada), pero las demás terminales varían considerablemente de un chip a otro. No obstante, las líneas de control pueden agruparse a grandes rasgos en las siguientes categorías principales:

- 1. Control de bus.
- 2. Interrupciones.
- 3. Arbitraje de bus.
- 4. Señalización de coprocesador.
- 5. Situación.
- 6. Diversos.

Describiremos brevemente cada una de estas categorías a continuación. Cuando examinemos los chips Pentium II, UltraSPARC II y picoJava II más adelante, daremos más detalles. En la figura 3-33 se muestra un chip de CPU genérico que utiliza estos grupos de señales.

Las terminales de control de bus son principalmente salidas de la CPU al bus (por tanto, entradas a los chips de memoria y de E/S) que dicen si la CPU quiere leer o escribir en la



**Figura 3-33.** Conexión lógica de terminales de una CPU genérica. Las flechas indican señales de entrada y de salida. Las líneas diagonales cortas indican que se usan varias terminales. En el caso de una CPU específica, se daría un número para indicar cuántas.

memoria o hacer alguna otra cosa. La CPU utiliza estas terminales para controlar el resto del sistema y decirle lo que quiere hacer.

Las terminales de interrupciones son entradas a la CPU provenientes de dispositivos de E/S. En casi todos los sistemas, la CPU puede ordenarle a un dispositivo de E/S que inicie una operación y luego dedicarse a hacer algo útil mientras el lento dispositivo de E/S está haciendo su trabajo. Una vez completada la E/S, el chip controlador de E/S habilita una señal en una de estas terminales para interrumpir a la CPU y pedirle que atienda al dispositivo de E/S (digamos, para ver si ocurrieron errores de E/S). Algunas CPU tienen una terminal de salida para indicar que recibieron la señal de interrupción.

Las terminales de arbitraje de bus son necesarias para regular el tráfico en el bus e impedir que dos dispositivos traten de usarlo al mismo tiempo. Para fines de arbitraje, la CPU cuenta como un dispositivo.

Algunos chips de CPU están diseñados para operar con coprocesadores, como chips de punto flotante, de gráficos o de otro tipo. Para facilitar la comunicación entre la CPU y el coprocesador, se incluyen terminales especiales para realizar y atender diversas solicitudes.

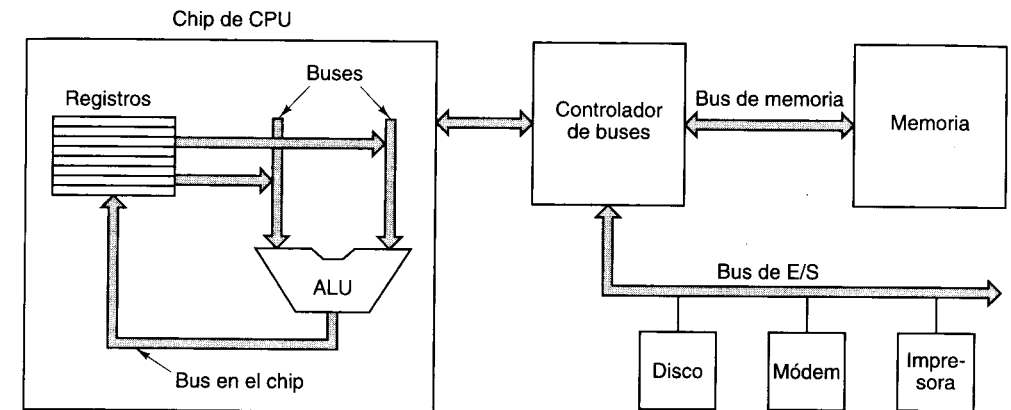
Además de estas señales, algunas CPU tienen varias terminales de uso diverso. Algunas proporcionan o aceptan información de estado, otras son útiles para restablecer la computadora, y otras más están presentes para asegurar la compatibilidad con chips de E/S antiguos.

### 3.4.2 Buses de computadora

Un **bus** es un camino eléctrico común entre varios dispositivos. Los buses pueden clasificarse por su función. Pueden usarse internamente en la CPU para transportar datos a y de la ALU, o externos a la CPU para conectarla con la memoria o los dispositivos de E/S. Cada tipo de bus tiene sus propios requisitos y propiedades. En esta sección y en las que siguen nos con-

centraremos en los buses que conectan a la CPU con la memoria y los dispositivos de E/S. En el siguiente capítulo examinaremos con más detenimiento los buses internos de la CPU.

Las primeras computadoras personales tenían un solo bus externo o **bus de sistema** que constaba de 50 a 100 hilos de cobre paralelos grabados en la tarjeta madre, con conectores espaciados a intervalos regulares para insertar tarjetas de memoria y de E/S. Las computadoras personales modernas generalmente tienen un bus de propósito especial entre la CPU y la memoria y (por lo menos) un bus más para los dispositivos de E/S. En la figura 3-34 se ilustra un sistema mínimo, con un bus de memoria y un bus de E/S.



**Figura 3-34.** Sistema de cómputo con varios buses.

En la literatura los buses a veces se dibujan como flechas gruesas, como en esta figura. La distinción entre una flecha gruesa y una sola línea atravesada por una línea diagonal corta y un número de bits junto a ella es sutil. Cuando todos los bits son del mismo tipo, digamos bits de dirección o bits de datos, es común usar la representación con la línea diagonal corta. Cuando intervienen líneas de dirección, datos y control, la flecha gruesa es más común.

Si bien los diseñadores de la CPU están en libertad de usar el tipo de bus que les plazca dentro del chip, si se quiere que tarjetas diseñadas por terceros puedan conectarse al bus del sistema deben existir reglas bien definidas que digan cómo funciona el bus, mismas que todos los dispositivos conectados a él deberán obedecer. Estas reglas son el **protocolo de bus**. Además, debe haber especificaciones mecánicas y eléctricas para que las tarjetas de terceros ajusten en la caja de tarjetas y tengan conectores que coincidan con los de la tarjeta madre tanto mecánicamente como en términos de voltajes, temporización, etcétera.

En el mundo de las computadoras se usan ampliamente varios buses. Unos de los más conocidos, tanto actuales como históricos (con ejemplos) son el Ómnibus (PDP-8), Unibus (PDP-11), Multibus (8086), el bus de IBM PC (PC/XT), el bus ISA (PC/AT), el bus EISA (80386), Microchannel (PS/2), el bus PCI (muchas PC), el bus SCSI (muchas PC y estaciones de trabajo), Nubus (Macintosh), Bus Serial Universal (PC modernas), FireWire (electrónica para consumidor), el bus VME (equipo de laboratorio de física) y el bus CAMAC (física de alta energía). Es probable que el mundo fuera un mejor lugar si todos menos uno se

desaparecieran repentinamente de la faz de la Tierra (está bien, de acuerdo, ¿qué tal todos menos dos?). Desafortunadamente, la estandarización en esta área es poco probable porque ya se ha invertido mucho en todos estos sistemas incompatibles.

Iniciemos ahora nuestro estudio del funcionamiento de los buses. Algunos dispositivos que se conectan a un bus son activos y pueden iniciar transferencias de bus, mientras que otros son pasivos y esperan solicitudes. Los activos se llaman **amos** o **maestros**; los pasivos se llaman **esclavos**. Cuando la CPU ordena a un controlador de disco que lea o escriba un bloque, la CPU está actuando como amo y el controlador de disco está actuando como esclavo. Sin embargo, unos momentos después el controlador de disco podría actuar como amo al ordenar a la memoria que acepte las palabras que está leyendo de la unidad de disco. En la figura 3-35 se enumeran varias combinaciones típicas de amo y esclavo. En ninguna circunstancia la memoria puede ser un amo.

| Amo          | Esclavo         | Ejemplo                                               |
|--------------|-----------------|-------------------------------------------------------|
| CPU          | Memoria         | Traer instrucciones y datos                           |
| CPU          | Dispositivo E/S | Iniciar transferencia de datos                        |
| CPU          | Coprocesador    | Delegación de instrucciones de la CPU al coprocesador |
| E/S          | Memoria         | DMA (acceso directo a memoria)                        |
| Coprocesador | CPU             | Obtención de operandos de la CPU por el coprocesador  |

Figura 3-35. Ejemplos de amos (controladores) y esclavos de bus.

Las señales binarias que los dispositivos de cómputo producen, a menudo no son lo bastante fuertes como para impulsar un bus, sobre todo si éste es relativamente largo y tiene conectados muchos dispositivos. Por esta razón, casi todos los amos de bus se conectan al bus con un chip llamado **controlador** o **driver de bus**, que en esencia es un amplificador digital. Así mismo, casi todos los esclavos se conectan al bus con un **receptor de bus**. En el caso de dispositivos que pueden actuar como amo y como esclavo, se emplea un chip combinado llamado **transceptor de bus**. Estos chips de interfaz con el bus suelen ser dispositivos de tres estados, para poder “flotar” (desconectarse) cuando no se necesitan, o estar conectados de forma un tanto distinta, llamada **de colector abierto**, cuyo efecto es el mismo. Cuando dos o más dispositivos de una línea de colector abierto habilitan la línea al mismo tiempo, el resultado es un OR booleano de todas las señales. Este sistema se conoce como **OR alambrado**. En la mayor parte de los buses, algunas de las líneas son de tres estados y otras, que necesitan la propiedad de OR alambrado, son de colector abierto.

Al igual que una CPU, un bus también tiene líneas de dirección, de datos y de control. Sin embargo, no necesariamente hay una correspondencia uno a uno entre las terminales de la CPU y las señales del bus. Por ejemplo, algunas CPU tienen tres terminales que codifican si se está efectuando una lectura de memoria, una escritura de memoria, una lectura de E/S, una escritura de E/S o alguna otra operación. Un bus típico podría tener una línea para lectura de memoria, una segunda para escritura de memoria, otra para lectura de E/S, una cuarta para escritura de E/S, etc. En tal caso se necesitaría un chip decodificador entre la CPU y el bus para coordinar ambos lados, es decir, para convertir la señal codificada de tres bits en señales individuales que se alimenten a las líneas del bus.

El diseño y la operación de buses son temas lo bastante complejos como para que se hayan escrito libros enteros sobre ellos (Stanley y Anderson, 1995b; Solari, 1993; y Solari y Willse, 1998). Los aspectos principales del diseño de un bus son su anchura, temporización, arbitraje y operaciones. Cada una de estas cuestiones tiene un impacto considerable sobre la rapidez y el ancho de banda del bus. Examinaremos cada uno de estos aspectos en las cuatro secciones que siguen.

3.4.3 Ancho de bus

El ancho de bus es el parámetro de diseño más obvio, Cuantas más líneas de dirección tenga un bus, más memoria podrá direccionar la CPU directamente. Si un bus tiene  $n$  líneas de dirección, una CPU podrá usarlo para direccionar  $2^n$  localidades de memoria distintas. Para manejar memorias grandes los buses necesitan muchas líneas de dirección. Sencillo, ¿no?

El problema es que los buses anchos necesitan más líneas que los angostos; además, ocupan más espacio físico (por ejemplo, en la tarjeta madre) y necesitan conectores más grandes. Todos estos factores hacen que el bus sea más caro. Por tanto, hay un equilibrio entre tamaño máximo de memoria y costo del sistema. Un sistema con un bus de direcciones de 64 líneas y  $2^{32}$  bytes de memoria costará más que uno con 32 líneas de dirección y los mismos  $2^{32}$  bytes de memoria. La posibilidad de una expansión futura no es gratuita.

El resultado de esta observación es que muchos diseñadores de sistemas tienden a ser miopes, y las consecuencias pueden ser graves. La IBM PC original contenía una CPU 8088 y un bus de direcciones de 20 bits, como se muestra en la figura 3-36(a). Estos 20 bits permitían a la PC direccionar 1 MB de memoria.

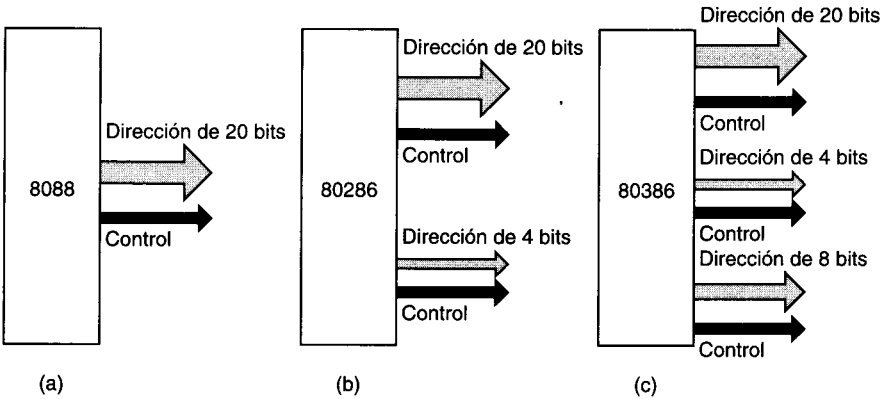


Figura 3-36. Crecimiento de un bus de direcciones con el tiempo.

Cuando surgió el chip de CPU (el 80286), Intel decidió incrementar el espacio de direcciones a 16 MB, por lo que fue necesario añadir cuatro líneas más de bus de dirección (sin meterse con las 20 originales, por razones de compatibilidad con modelos anteriores), como se muestra en la figura 3-36(b). Lamentablemente, fue necesario añadir más líneas de control para ocuparse de las nuevas líneas de dirección. Cuando apareció el 80386, se añadieron

otras ocho líneas de dirección, junto con más líneas de control, como se muestra en la figura 3-36(c). El diseño resultante (el bus EISA) es mucho menos elegante de lo que habría sido si hubiera tenido 32 líneas desde el principio.

El número de líneas de dirección no es lo único que tiende a aumentar con el tiempo; lo mismo sucede con el número de líneas de datos, pero por una razón un poco distinta. Hay dos formas de incrementar el ancho de banda de datos de un bus: reducir el tiempo de ciclo de bus (más transferencias por segundo) o incrementar la capacidad del bus de datos (más bits/transferencia). Es posible acelerar el bus, pero no es fácil porque las señales de las diferentes líneas viajan a velocidades ligeramente distintas, problema que se conoce como **sesgo de bus**. Cuanto más rápido es el bus, más importante es el efecto del sesgo de bus.

Otro problema que surge al hacer más rápido un bus es que se vuelve incompatible con lo existente. Las tarjetas antiguas diseñadas para el bus más lento no funcionan con el nuevo. Hacer obsoletas las tarjetas viejas pone de mal humor tanto a los dueños de las tarjetas como a sus fabricantes. Por ello, la estrategia usual para mejorar el desempeño es añadir más líneas de datos, algo análogo a lo que se muestra en la figura 3-36. Sin embargo, como era de esperarse, este crecimiento incremental no produce un diseño claro al final. La IBM PC y sus sucesoras, por ejemplo, pasaron de 8 líneas de datos a 16 y luego a 32, con prácticamente el mismo bus.

Para aliviar el problema de los buses excesivamente anchos, algunos diseñadores optan por usar un **bus multiplexado**. En este diseño, en lugar de tener líneas de direcciones y de datos por separado, hay, por ejemplo, 32 líneas para ambos, direcciones y datos. Al principio de una operación de bus las líneas se usan para la dirección; más adelante, se usan para datos. En el caso de una escritura en memoria, por ejemplo, esto implica que las líneas de dirección deben configurarse y propagarse a la memoria antes de que los datos puedan colocarse en el bus. Con líneas separadas, la dirección y los datos pueden enviarse al mismo tiempo. La multiplexión de las líneas reduce la anchura (y el costo) del bus, pero hace más lento al sistema. Los diseñadores tienen que sopesar cuidadosamente todas estas opciones al tomar sus decisiones.

### 3.4.4 Temporización del bus

Los buses se pueden dividir en dos categorías según su temporización. Un **bus sincrónico** tiene una línea alimentada por un oscilador de cristal. La señal de esta línea consiste en una onda cuadrada cuya frecuencia generalmente está entre 5 MHz y 100 MHz. Todas las actividades del bus tardan un número entero de estos ciclos, llamados **ciclos de bus**. El otro tipo de bus, el **bus asincrónico**, no tiene un reloj maestro. Los ciclos de bus pueden tener la longitud que se requiera y no tiene que ser la misma entre todos los pares de dispositivos. A continuación examinaremos los dos tipos de buses.

#### Buses sincrónicos

Como ejemplo del funcionamiento de un bus sincrónico, considere la temporización de la figura 3-37(a). En este ejemplo, usaremos un reloj de 40 MHz, lo que da un ciclo de bus de 25 ns. Si bien esto podría parecer algo lento en comparación con las velocidades de CPU de 500 MHz y más, pocos buses de PC existentes son mucho más rápidos. Por ejemplo, el bus

ISA que se usa en todas las PC basadas en Intel opera a 8.33 MHz, e incluso el popular bus PCI por lo regular opera a 33 MHz o bien 66 MHz. Ya explicamos las razones por las que los buses actuales son lentos: problemas técnicos de diseño como el sesgo de bus, y la necesidad de compatibilidad con modelos anteriores.

En nuestro ejemplo supondremos también que una lectura de la memoria tarda 40 ns desde el momento en que la dirección es estable. Como veremos en breve, con estos parámetros se requieren tres ciclos de bus para leer una palabra. El primer ciclo inicia en el flanco ascendente de  $T_1$  y el tercero termina en el flanco ascendente de  $T_4$ , como se muestra en la figura. Observe que ninguno de los flancos ascendentes o descendentes se dibujó vertical, porque ninguna señal eléctrica puede cambiar su valor en un tiempo cero. En este ejemplo supondremos que una señal tarda 1 ns en cambiar. Las líneas de reloj, ADDRESS (dirección), DATA (datos),  $\overline{MREQ}$  (solicitud de memoria),  $\overline{RD}$  (leer) y  $\overline{WAIT}$  (esperar) aparecen en la misma escala de tiempo.

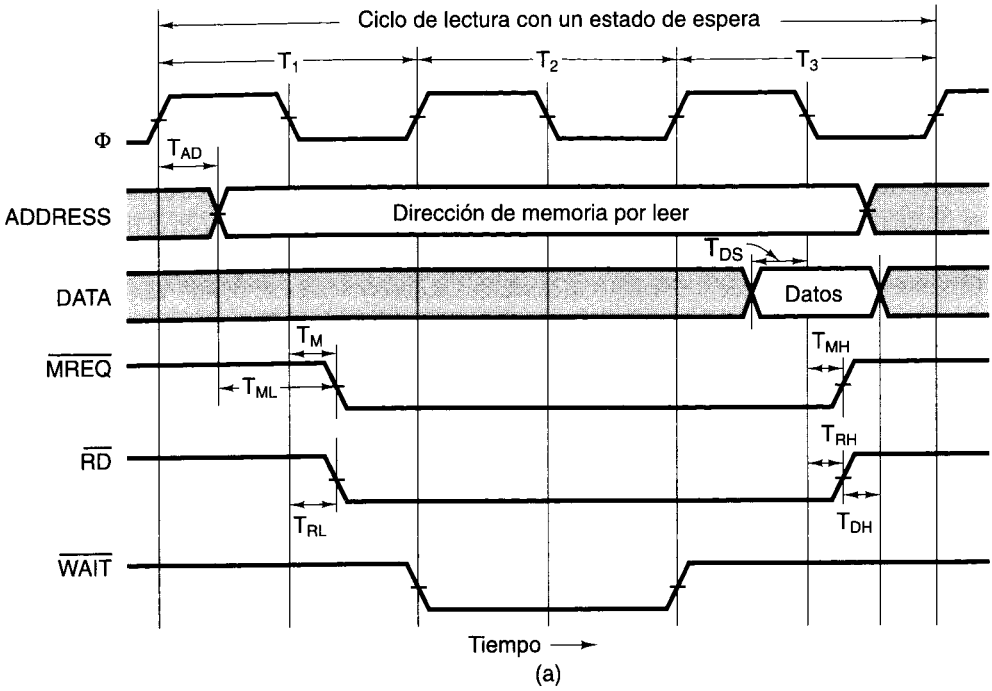
El inicio de  $T_1$  está definido por el flanco ascendente del reloj. Ya iniciado  $T_1$ , la CPU coloca la dirección de la palabra que quiere en las líneas de dirección. Puesto que la dirección no es un valor individual, como el reloj, no podemos mostrarlo como una sola línea en la figura; en vez de ello, se muestra como dos líneas, con un cruce en el momento en que cambia la dirección. Además, el sombreado previo al cruce indica que el valor antes del cruce no es importante. Utilizando la misma convención de sombreado, vemos que el contenido de las líneas de datos no es significativo sino hasta ya bien entrado  $T_3$ .

Una vez que las líneas de dirección han tenido tiempo de estabilizarse en sus nuevos valores, se asertan  $\overline{MREQ}$  y  $\overline{RD}$ . La primera indica que se está accediendo a la memoria (no a un dispositivo de E/S), y la segunda se habilita si se trata de una lectura y se deshabilita si se trata de una escritura. Puesto que la operación de memoria tarda 40 ns después de que la dirección es estable (ya iniciado el primer ciclo de reloj), no puede proporcionar los datos solicitados durante  $T_2$ . Para avisarle a la CPU que no los espere, la memoria habilita la línea  $\overline{WAIT}$  al principio de  $T_2$ . Esta acción insertará **estados de espera** (ciclos de bus extra) hasta que la memoria termine y deshabilite  $\overline{WAIT}$ . En nuestro ejemplo se insertó un estado de espera ( $T_2$ ) porque la memoria es demasiado lenta. Al principio de  $T_3$ , cuando la memoria está segura de que tendrá los datos durante el ciclo en curso, deshabilita  $\overline{WAIT}$ .

Durante la primera mitad de  $T_3$  la memoria coloca los datos en las líneas de datos. En el flanco descendente de  $T_3$  la CPU habilita (es decir, lee) las líneas de datos y almacena el valor en un registro interno. Después de leer los datos, la CPU invalida  $\overline{MREQ}$  y  $\overline{RD}$ . Si es necesario, se puede iniciar otro ciclo de memoria en el siguiente flanco ascendente del reloj.

En la especificación de temporización de la figura 3-37(b) se aclaran ocho símbolos que ocurren en el diagrama de temporización.  $T_{AD}$ , por ejemplo, es el intervalo de tiempo entre el flanco ascendente del reloj  $T_1$  y el establecimiento de las líneas de dirección. Según la especificación de temporización,  $T_{AD} \leq 11$  ns. Esto implica que el fabricante de la CPU garantiza que durante cualquier ciclo de lectura la CPU producirá la dirección que se va a leer en un tiempo de 11 ns después del punto medio del flanco ascendente de  $T_1$ .

Las especificaciones de temporización también requieren que los datos estén disponibles en las líneas de datos al menos  $T_{DS}$  (5 ns) antes del flanco descendente de  $T_3$ , para que tengan tiempo de estabilizarse antes de que la CPU habilite su entrada. La combinación de



| Símbolo         | Parámetro                                                         | Mín | Máx | Unidad |
|-----------------|-------------------------------------------------------------------|-----|-----|--------|
| T <sub>AD</sub> | Retraso de salida de dirección                                    |     | 11  | ns     |
| T <sub>ML</sub> | Dirección estable antes de MREQ                                   | 6   |     | ns     |
| T <sub>M</sub>  | Retraso de MREQ desde el flanco desc. $\Phi$ en T <sub>1</sub>    |     | 8   | ns     |
| T <sub>RL</sub> | Retraso de RD desde el flanco desc. de $\Phi$ en T <sub>1</sub>   |     | 8   | ns     |
| T <sub>DS</sub> | Tiempo de preparación de datos antes del flanco desc. de $\Phi$   | 5   |     | ns     |
| T <sub>MH</sub> | Retraso de MREQ desde el flanco desc. de $\Phi$ en T <sub>3</sub> |     | 8   | ns     |
| T <sub>RH</sub> | Retraso de RD desde el flanco desc. de $\Phi$ en T <sub>3</sub>   |     | 8   | ns     |
| T <sub>DH</sub> | Tiempo de retención de datos desde la deshabilitación de RD       | 0   |     | ns     |

(b)

Figura 3-37. (a) Temporización de lectura en un bus síncrono. (b) Especificación de algunos tiempos críticos.

restricciones sobre T<sub>AD</sub> y T<sub>DS</sub> implica que, en el peor de los casos, la memoria sólo dispondrá de 62.5 – 11 – 5 = 46.5 ns desde el momento en que aparezca la dirección hasta que tenga que producir los datos. Puesto que bastan 40 ns, aun en el peor de los casos, una memoria de 40 ns

siempre puede responder durante T<sub>3</sub>. Una memoria de 50 ns, en cambio, tendría que insertar un segundo estado de espera y responder durante T<sub>4</sub>.

La especificación de temporización garantiza además que la dirección estará preparada al menos 6 ns antes de que se habilite MREQ. Este tiempo puede ser importante si MREQ controla la selección de chip en la memoria, porque algunas memorias requieren un tiempo de preparación de la dirección previo a la selección del chip. Es obvio que el diseñador del sistema no debe escoger un chip de memoria que requiera un tiempo de preparación de 10 ns.

Las restricciones sobre T<sub>M</sub> y T<sub>RL</sub> implican que tanto MREQ como RD se asertarán en un plazo de 8 ns después del reloj descendente T<sub>1</sub>. En el peor de los casos, el chip de memoria tendrá sólo 25 + 25 – 8 – 5 = 37 ns después de la aserción de MREQ y RD para colocar sus datos en el bus. Esta restricción es adicional respecto (e independiente) del intervalo de 40 ns que se requiere después de que la dirección se estabiliza.

T<sub>MH</sub> y T<sub>RH</sub> nos dicen cuánto tardan MREQ y RD en deshabilitarse una vez que se ha habilitado la entrada de los datos. Por último, T<sub>DH</sub> nos dice cuánto tiempo la memoria debe mantener los datos en el bus después de que RD se ha deshabilitado. En lo que a nuestra CPU de ejemplo concierne, la memoria puede quitar los datos del bus tan pronto como RD se ha deshabilitado; sin embargo, en algunas CPU reales los datos deben mantenerse estables durante un tiempo corto después de eso.

Cabe señalar que la figura 3-37 es una versión muy simplificada de las restricciones de temporización reales. En la práctica siempre se especifican muchos más tiempos críticos. No obstante, esto da cierta idea de cómo funciona un bus síncrono.

Como último punto que vale la pena mencionar es que las señales de control pueden ser asignadas, como altas o bajas. Compete a los diseñadores del bus determinar qué es lo más conveniente, pero la elección es esencialmente arbitraria. Uno puede considerarlo como un equivalente de hardware de la opción de un programador para representar bloques de disco libres en un mapa de bits como ceros *versus* unos.

Buses asincrónicos

Aunque es fácil trabajar con buses síncronos gracias a sus intervalos de tiempo discretos, también tienen ciertos problemas. Por ejemplo, todo funciona en múltiplos del reloj del bus. Si una CPU y una memoria pueden completar una transferencia en 3.1 ciclos, tendrán que prolongarlo a 4.0 porque están prohibidas las fracciones de ciclo.

Peor aún, una vez que se ha escogido un ciclo de bus, y que se han construido tarjetas de memoria y de E/S para él, es difícil aprovechar mejoras futuras en la tecnología. Por ejemplo, suponga que, unos cuantos años después de construirse el sistema de la figura 3-37, salen al mercado memorias con tiempos de acceso de 20 ns en lugar de 40 ns. Ya no sería necesario el estado de espera, y la máquina trabajaría más rápidamente. Ahora suponga que aparecen memorias de 10 ns. El desempeño del sistema no mejoraría, porque el tiempo mínimo para una lectura es de 2 ciclos con este diseño.

Expresando este hecho en términos un poco diferentes, si un bus síncrono tiene una colección heterogénea de dispositivos, algunos rápidos y otros lentos, el bus tiene que ajustarse al más lento, y los rápidos no podrán aprovechar todo su potencial.



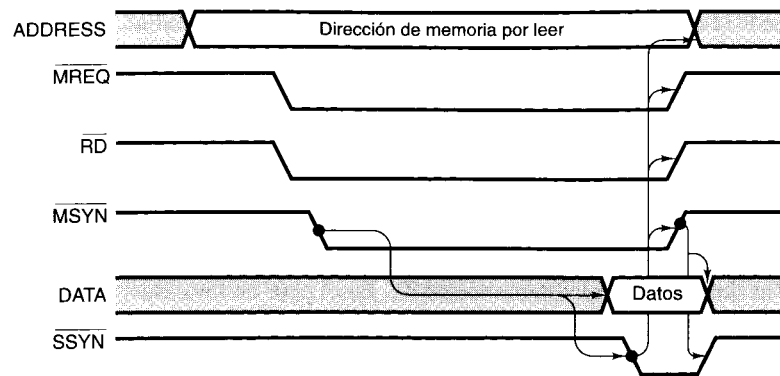


Figura 3-38. Funcionamiento de un bus asincrónico.

La tecnología mixta puede manejarse cambiando a un bus asincrónico, es decir, uno que no tenga reloj maestro, como el de la figura 3-38. En lugar de vincular todo con el reloj, una vez que el amo del bus ha habilitado la dirección,  $\overline{MREQ}$ ,  $\overline{RD}$  y todo lo demás que necesite habilitar, habilita una señal especial que llamaremos  $\overline{MSYN}$  (sincronización de amo, *Master SYNchronization*). Cuando el esclavo percibe esto, realiza su trabajo lo más rápidamente que puede. Cuando termina, habilita  $\overline{SSYN}$  (sincronización de esclavo, *Slave SYNchronization*).

Tan pronto como el amo reconoce que se habilitó  $\overline{SSYN}$ , sabe que los datos están disponibles, así que los almacena y luego deshabilita las líneas de dirección junto con  $\overline{MREQ}$ ,  $\overline{RD}$  y  $\overline{MSYN}$ . Cuando el esclavo detecta la deshabilitación de  $\overline{MSYN}$ , sabe que el ciclo se completó y deshabilitó  $\overline{SSYN}$ , con lo que estamos otra vez en la situación original, con todas las señales deshabilitadas, esperando el siguiente amo.

Los diagramas de temporización de buses asincrónicos (y a veces también los sincrónicos) utilizan flechas para indicar causa y efecto, como en la figura 3-38. La habilitación de  $\overline{MSYN}$  hace que las líneas de datos se habiliten y también hace que el esclavo habilite  $\overline{SSYN}$ . La habilitación de  $\overline{SSYN}$ , a su vez, causa la deshabilitación de las líneas de dirección,  $\overline{MREQ}$ ,  $\overline{RD}$  y  $\overline{MSYN}$ . Por último, la deshabilitación de  $\overline{MSYN}$  causa la deshabilitación de  $\overline{SSYN}$ , con lo que termina la lectura.

Un conjunto de señales que interactúa de esta manera se llama **saludo completo**. La parte fundamental consiste en cuatro sucesos:

1. Habilitar  $\overline{MSYN}$ .
2. Habilitar  $\overline{SSYN}$  como respuesta a  $\overline{MSYN}$ .
3. Deshabilitar  $\overline{MSYN}$  como respuesta a  $\overline{SSYN}$ .
4. Deshabilitar  $\overline{SSYN}$  como respuesta a la invalidación de  $\overline{MSYN}$ .

Debe ser obvio que los saludos completos son independientes de la temporización. Cada suceso es causado por un suceso anterior, no por un pulso de reloj. Si un par amo-esclavo dado es lento, no afectará en absoluto a un par amo-esclavo subsecuente que sea mucho más rápido.

La ventaja de un bus asincrónico ha quedado clara, pero el hecho es que casi todos los buses son sincrónicos. La razón es que es más fácil construir un sistema sincrónico. La CPU simplemente habilita sus señales, y la memoria simplemente reacciona. No hay retroalimentación (causa y efecto), pero si los componentes se escogieron debidamente, todo funcionará bien sin saludos. Además, existe una inversión considerable en la tecnología de los buses sincrónicos.

### 3.4.5 Arbitraje del bus

Hasta ahora, hemos supuesto tácitamente que sólo hay un controlador de bus, la CPU. En realidad, los chips de E/S tienen que convertirse en controladores de bus para leer y escribir en la memoria, y también para causar interrupciones. Los coprocesadores también podrían necesitar convertirse en controladores de bus. Entonces surge la pregunta: "¿Qué sucede si dos o más dispositivos quieren convertirse en controlador del bus al mismo tiempo? La respuesta es que se requiere algún mecanismo de **arbitraje de bus** para evitar el caos.

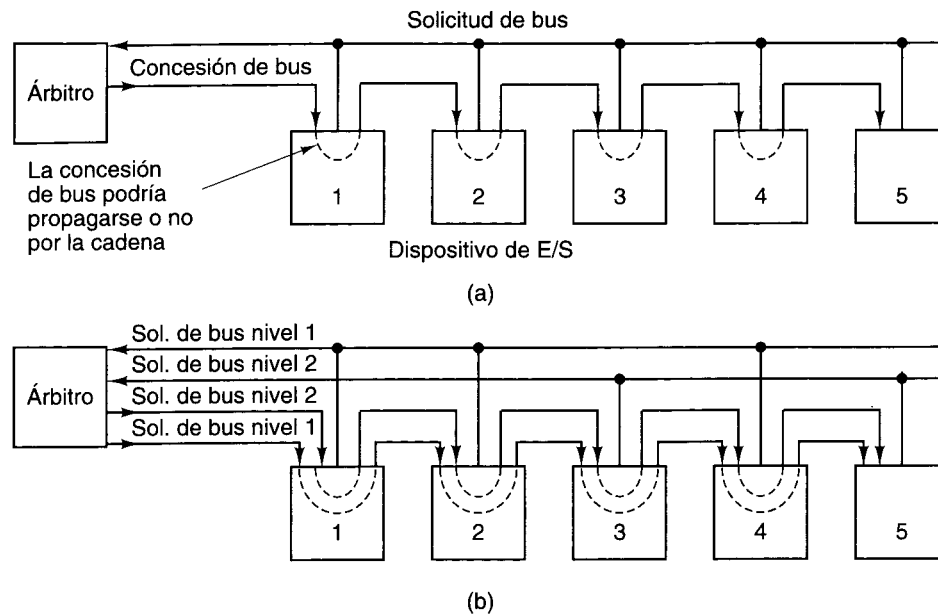
Los mecanismos de arbitraje pueden ser centralizados o descentralizados. Consideremos primero el arbitraje centralizado. Una forma especialmente sencilla de arbitraje centralizado se muestra en la figura 3-39(a). En este esquema, un solo árbitro de bus determina quién sigue. Muchas CPU tienen el árbitro integrado a su chip, pero a veces se requiere un chip aparte. El bus contiene una sola línea de solicitud de OR alambrado que uno o más dispositivos pueden habilitar en cualquier momento. El árbitro no tiene forma de saber cuántos dispositivos solicitaron el bus. Las únicas categorías que puede distinguir son: hay solicitudes y no hay solicitudes.

Cuando el árbitro detecta una solicitud de bus, emite una concesión habilitando la línea de otorgamiento del bus. Esta línea está conectada a todos los dispositivos de E/S en serie, como una serie barata de luces de Navidad. Cuando el dispositivo que está más cercano físicamente al árbitro percibe la concesión, verifica si él emitió una solicitud. Si lo hizo, se apodera del bus pero no propaga la concesión línea abajo. Si no hizo una solicitud, propaga la concesión al siguiente dispositivo en la línea, que se comporta de la misma manera, y así hasta que algún dispositivo acepta la concesión y se apodera del bus. Este esquema se llama **encadenamiento circular**, y tiene la propiedad de que en la práctica asigna prioridades a los dispositivos con base en su cercanía al árbitro. El dispositivo más cercano gana.

Para supeditar las prioridades implícitas basadas en la distancia al árbitro, muchos buses tienen varios niveles de prioridad. Para cada nivel de prioridad hay una línea de solicitud de bus y una línea de otorgamiento de bus. El bus de la figura 3-39(b) tiene dos niveles, 1 y 2 (los buses reales a menudo tienen 4, 8 o 16 niveles). Cada dispositivo se conecta a uno de los niveles de solicitud de bus, y los dispositivos para los que el tiempo es más crítico se conectan a los de más alta prioridad. En la figura 3-39(b) los dispositivos 1, 2 y 4 usan la prioridad 1 mientras que los dispositivos 3 y 5 usan la prioridad 2.

Si hay solicitudes en varios niveles de prioridad al mismo tiempo, el árbitro emite una concesión sólo en el de más alta prioridad. Entre dispositivos con la misma prioridad se usa el encadenamiento circular. En la figura 3-39(b), si hay conflictos, el dispositivo 2 vence al





**Figura 3-39.** (a) Árbitro de bus centralizado de un nivel con encadenamiento circular. (b) El mismo árbitro, pero con dos niveles.

dispositivo 4, el cual vence al 3. El dispositivo 5 tiene la prioridad más baja porque está al final de la cadena de más baja prioridad.

Por cierto, no es técnicamente necesario conectar la línea de otorgamiento de bus del nivel 2 en serie a los dispositivos 1 y 2, porque no pueden hacer solicitudes en ese nivel. No obstante, por comodidad de implementación, se conectan todas las líneas de otorgamiento a todos los dispositivos, en lugar de conectar un cableado especial que dependa de cuál dispositivo tiene cuál prioridad.

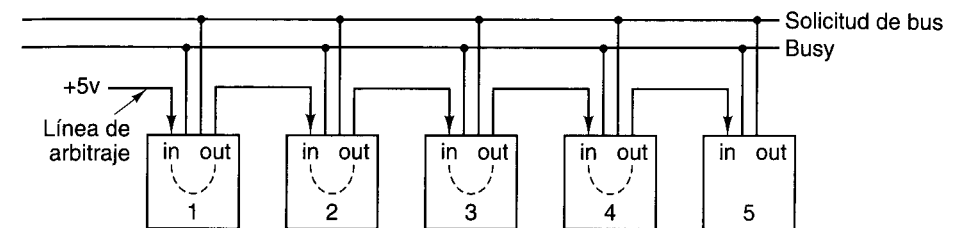
Algunos árbitros tienen una tercera línea que un dispositivo habilita cuando aceptó una concesión y se apoderó del bus. Tan pronto como el dispositivo habilita esta línea de acuse, las líneas de solicitud y de otorgamiento pueden deshabilitarse. El resultado es que otros dispositivos pueden solicitar el bus mientras el primer dispositivo está usando el bus. Para cuando termina la transferencia actual en el bus, ya se ha seleccionado el siguiente controlador de bus, el cual puede iniciar tan pronto como se deshabilita la línea de acuse, momento en el cual se puede iniciar la siguiente ronda de arbitraje. Este esquema requiere una línea de bus extra y más lógica en cada dispositivo, pero aprovecha mejor los ciclos de bus.

En sistemas en los que la memoria está en el bus principal, la CPU debe competir con todos los dispositivos de E/S por el bus en casi cada ciclo. Una solución común para esta situación es dar a la CPU la prioridad más baja, de modo que reciba el bus sólo cuando nadie más lo quiere. La idea aquí es que la CPU siempre puede esperar, pero los dispositivos de E/S con frecuencia deben adquirir el bus rápidamente o perder datos de entrada. Los discos que giran a alta velocidad no pueden esperar. Este problema se evita en muchas computadoras

modernas colocando la memoria en un bus distinto del de los dispositivos de E/S, para que éstos no tengan que competir por el acceso al bus.

Otra posibilidad es el arbitraje de bus descentralizado. Por ejemplo, una computadora podría tener 16 líneas de solicitud de bus por prioridades. Cuando un dispositivo quiere usar el bus, habilita su línea de solicitud. Todos los dispositivos monitorean todas las líneas de solicitud, así que al término de cada ciclo de bus cada dispositivo sabe si fue el solicitante con más alta prioridad y por tanto si tiene permiso de usar el bus durante el siguiente ciclo. En comparación con el arbitraje de bus centralizado, este método de arbitraje requiere más líneas de bus pero evita los posibles costos del árbitro; además, limita el número de dispositivos, que no puede ser mayor que el número de líneas de solicitud.

Otro tipo de arbitraje de bus descentralizado, que se muestra en la figura 3-40, sólo usa tres líneas, por más dispositivos que estén presentes. La primera línea de bus es una de OR alambrado para solicitar el bus. La segunda se llama BUSY (ocupado) y es asertada por el amo de bus vigente. La tercera línea sirve para arbitrar el bus, y se conecta en serie a todos los dispositivos. La cabeza de esta cadena se mantiene habilitada conectándola a la fuente de alimentación de 5 volts.



**Figura 3-40.** Arbitraje de bus descentralizado.

Si ningún dispositivo quiere el bus, la línea de arbitraje asertada se propaga por todos los dispositivos. Si un dispositivo quiere adquirir el bus, primero ve si el bus está inactivo y la señal de arbitraje que está recibiendo, IN, está asertada. Si IN está invalidada, el dispositivo no podrá convertirse en controlador del bus e invalidará OUT. En cambio, si IN está habilitada, el dispositivo deshabilitará OUT. Esto hará que su vecino cadena abajo vea IN deshabilitado y deshabilite su OUT. Así, todos los dispositivos cadena abajo ven IN deshabilitado y por tanto deshabilitan OUT. Al final, sólo un dispositivo tendrá IN habilitado y OUT deshabilitado. Este dispositivo se convertirá en el controlador del bus, habilitará BUSY y OUT, e iniciará su transferencia.

Si lo pensamos, veremos que el dispositivo más a la izquierda que quiera el bus lo recibirá. Así, este esquema es similar al arbitraje de cadena circular original, excepto que no tiene árbitro, por lo que es más económico, más rápido, y no es vulnerable a un fallo del árbitro.

### 3.4.6 Operaciones de bus

Hasta ahora, sólo hemos visto ciclos de bus ordinarios, en los que un controlador (generalmente la CPU) lee desde un esclavo (generalmente la memoria) o escribe en él. De hecho, existen varios tipos de ciclos de bus. A continuación veremos algunos de ellos.

Normalmente, sólo se transfiere una palabra a la vez. Sin embargo, cuando se usa caché, es deseable traer toda una línea de caché (por ejemplo, 16 palabras consecutivas de 32 bits) a la vez. En muchos casos es posible efectuar transferencias de bloques con mayor eficiencia que transferencias individuales sucesivas. Cuando se inicia una lectura de bloque, el controlador del bus le dice al esclavo cuántas palabras debe transferir, por ejemplo, colocando la cuenta de palabras en las líneas de datos durante  $T_1$ . En lugar de devolver sólo una palabra, el esclavo produce una palabra durante cada ciclo hasta agotar la cuenta. La figura 3-41 muestra una versión modificada de la figura 3-37(a), pero ahora con una señal adicional,  $\overline{\text{BLOCK}}$ , que se habilita para indicar que se solicita una transferencia de bloque. En este ejemplo, la lectura de un bloque de 4 palabras tarda 6 ciclos en lugar de 12.

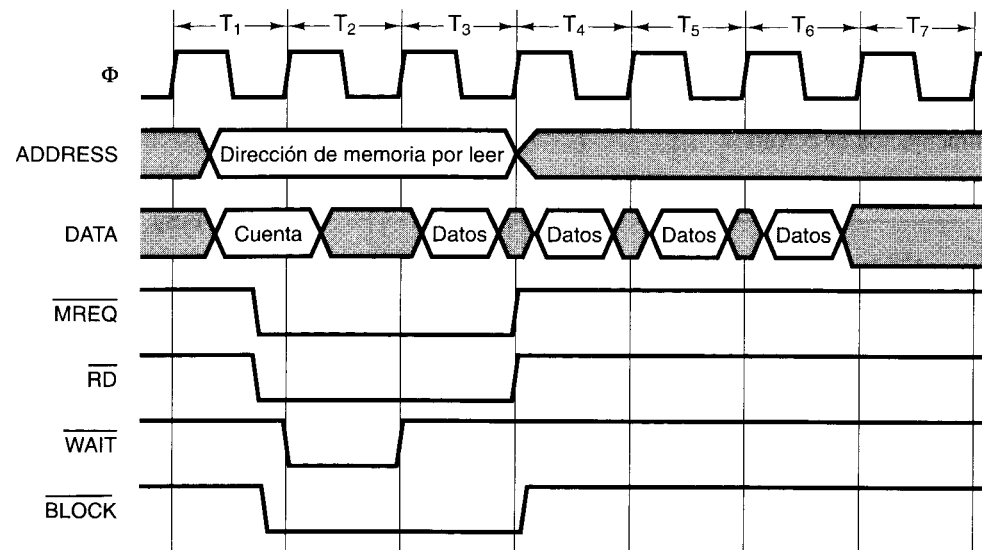


Figura 3-41. Transferencia de un bloque.

También existen otras clases de ciclos de bus. Por ejemplo, en un sistema multiprocesador que tiene dos o más CPU en el mismo bus, a menudo es necesario asegurarse de que sólo una CPU a la vez utilice alguna estructura de datos crítica en la memoria. Una forma común de lograr esto es tener una variable en la memoria que sea 0 cuando ninguna CPU esté usando la estructura de datos y 1 cuando esté en uso. Si una CPU quiere acceder a la estructura de datos, primero deberá leer la variable y si es 0, ponerla en 1. El problema es que, con un poco de mala suerte, dos CPU podrían leerla en ciclos de bus consecutivos. Si los dos ven que la variable es 0, los dos la pondrán en 1 y cada una creerá que es la única CPU que está usando la estructura de datos. Esta serie de sucesos lleva al caos.

Para evitar esta situación, los sistemas multiprocesador a menudo tienen un ciclo de bus especial de leer-modificar-escribir que permite a cualquier CPU leer una palabra de la memoria, inspeccionarla y modificarla, y escribirla de vuelta en la memoria, todo sin desocupar el

bus. Este tipo de ciclo evita que las CPU competidoras puedan usar el bus y así interfieran la operación de la primera CPU.

Otro tipo importante de ciclo de bus sirve para manejar interrupciones. Cuando la CPU ordena a un dispositivo de E/S hacer algo, generalmente espera una interrupción una vez que se concluye el trabajo. La señalización de la interrupción requiere el bus.

Puesto que varios dispositivos podrían querer causar una interrupción simultáneamente, surgen los mismos tipos de problemas de arbitraje que teníamos con los ciclos de bus ordinarios. La solución usual es asignar prioridades a los dispositivos, y usar un árbitro centralizado para dar prioridad a los dispositivos para los que el tiempo es más crítico. Existen chips controladores de interrupciones estándar y se usan ampliamente. La IBM PC y todas sus sucesoras usan el chip Intel 8259A, que se ilustra en la figura 3-42.

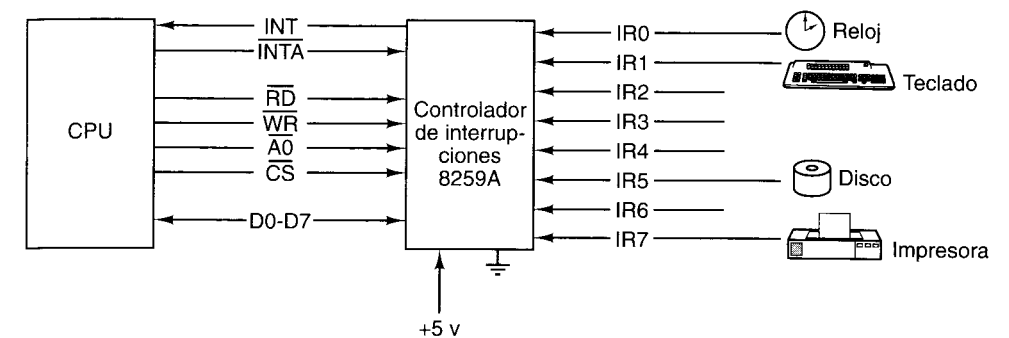


Figura 3-42. Uso del controlador de interrupciones 8259A.

Hasta ocho chips controladores de E/S se pueden conectar directamente a las ocho entradas  $\text{IR}_x$  (solicitud de interrupción, *Interrupt Request*) del 8259A. Cuando cualquiera de estos dispositivos desea causar una interrupción, habilita su línea de entrada. Cuando se habilita una o más entradas, el 8259A habilita  $\text{INT}$  (INTerrupción), que alimenta directamente la terminal de interrupción de la CPU. Cuando la CPU está en condiciones de atender la interrupción, devuelve un pulso al 8259A por  $\text{INTA}$  (acuse de interrupción, *INTerrupt Acknowledge*). En ese momento el 8259A deberá especificar cuál entrada causó la interrupción colocando el número de esa entrada en el bus de datos. Esta operación requiere un ciclo de bus especial. Luego, el hardware de la CPU utiliza ese número como índice de una tabla de apuntadores, llamados **vectores de interrupción**, para encontrar la dirección del procedimiento que debe ejecutar para atender la interrupción.

El 8259A contiene varios registros que la CPU puede leer y escribir empleando ciclos de bus ordinarios y las líneas  $\overline{\text{RD}}$ ,  $\overline{\text{WR}}$  (escribir, *Write*),  $\overline{\text{CS}}$  y  $\overline{\text{AO}}$ . Una vez que el software ha manejado la interrupción y está listo para recibir la siguiente, escribe un código especial en uno de los registros, y esto hace que el 8259A deshabilite  $\text{INT}$ , a menos que tenga otra interrupción pendiente. Estos registros también pueden modificarse para colocar el 8259A en uno de varios modos, enmascarar un conjunto de interrupciones y habilitar otras funciones.

Cuando están presentes más de ocho dispositivos de E/S, los 8259A pueden conectarse en cascada. En el caso más extremo, las ocho entradas pueden conectarse a las salidas de

ocho 8259A adicionales, lo que permite tener hasta 64 dispositivos de E/S en una red de interrupciones de dos etapas. El 8259A tiene unas cuantas líneas dedicadas a manejar esta conexión en cascada, que hemos omitido para simplificar.

Si bien estamos muy lejos de agotar el tema del diseño de buses, el material anterior deberá bastar para entender los fundamentos del funcionamiento de los buses y la interacción entre ellos y las CPU. Pasemos ahora de lo general a lo específico y examinemos algunos ejemplos de CPU reales y sus buses.

### 3.5 EJEMPLOS DE CHIPS DE CPU

En esta sección examinaremos los chips Pentium II, UltraSPARC II y picoJava II con cierto detalle en el nivel de hardware.

#### 3.5.1 El Pentium II

El Pentium II es un descendiente directo de la CPU 8088 que se usó en la IBM PC original. Aunque el Pentium II con sus 7.5 millones de transistores está muy lejos del 8088 con 29,000 transistores, es totalmente compatible con el 8088 y puede ejecutar programas binarios para el 8088 sin modificación (además de programas para todos los procesadores intermedios).

Desde el punto de vista del software, el Pentium II es una máquina de 32 bits completa. Tiene la misma ISA en el nivel de usuario que los chips 80386, 80486, Pentium y Pentium Pro, incluidos los mismos registros, las mismas instrucciones y una implementación completa en el chip del estándar de punto flotante IEEE 754.

Desde el punto de vista del hardware, el Pentium II es algo más porque puede direccionar 64 GB de memoria física y puede transferir datos de y a la memoria en unidades de 64 bits. Aunque el programador no puede observar estas transferencias de 64 bits, hacen que la máquina sea más rápida que una máquina de 32 bits pura.

Internamente, en el nivel de microarquitectura, el Pentium II es básicamente un Pentium Pro al que se añadieron las instrucciones MMX. Las instrucciones en el nivel de ISA se obtienen de la memoria con mucha anticipación y se descomponen en microoperaciones tipo RISC. Estas microoperaciones se almacenan en un buffer, y tan pronto como una dispone de los recursos necesarios para ejecutarse, puede iniciarse. Se pueden iniciar varias microoperaciones en el mismo ciclo, lo que convierte al Pentium II en una máquina superescalar.

El Pentium II tiene una caché de dos niveles. Hay un par de cachés en el chip, 16 KB para instrucciones y 16 KB para datos, además de una caché unificada de segundo nivel de 512 KB. El tamaño de la línea de caché es de 32 bytes. La caché de segundo nivel se ejecuta a la mitad de la frecuencia de reloj de la CPU. Hay relojes de CPU de 233 MHz o más.

Se usan dos buses externos primarios en los sistemas Pentium II, ambos sincrónicos. El bus de memoria sirve para acceder la DRAM principal; el bus PCI se usa para comunicarse con los dispositivos de E/S. A veces se conecta un bus de **legado** (o sea, antiguo) al bus de PCI para poder conectar a él dispositivos periféricos de tecnología anterior.

Un sistema Pentium II puede tener uno o dos CPU que comparten una memoria común. En un sistema de dos CPU, existe el peligro de que si una palabra se lee y se coloca en una caché, y se modifica ahí sin que se escriba de vuelta en la memoria, si la otra CPU trata de leer la palabra obtendrá un valor incorrecto. Se incluye apoyo especial (espionaje) para evitar este problema.

Una diferencia importante entre el Pentium II y todos sus predecesores es su empaquetado. Desde la 8088 hasta el Pentium Pro, todas las CPU de Intel eran chips normales, con terminales en los lados o la base que se podían conectar en zócalos. En contraste, el Pentium II está provisto de lo que Intel llama un **SEC (cartucho de una sola arista, Single Edge Cartridge)**. Como puede verse en la figura 3-43, un SEC es un encapsulado de plástico relativamente grande que contiene la CPU, la caché nivel 2 y un conector de arista para exportar las señales. El SEC de Pentium II tiene 242 conectores.

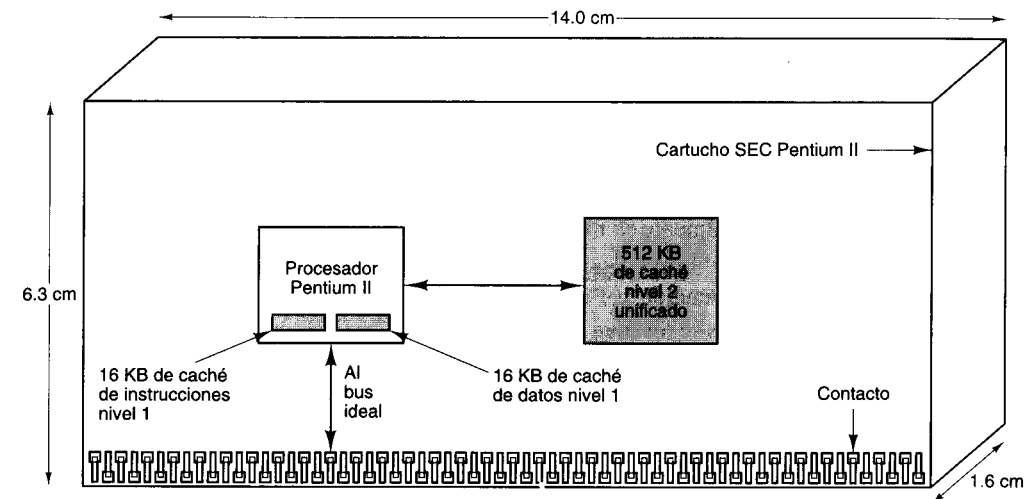


Figura 3-43. El paquete SEC del Pentium II.

Aunque sin duda Intel tuvo sus motivos para cambiar a este modelo de encapsulado, produjo un problema en una dimensión que jamás previó. Al parecer, muchos clientes tienen la costumbre de abrir sus computadoras para buscar el chip de CPU. En los primeros Pentium II que salieron a la venta, los clientes no podían encontrar la CPU y se quejaron a grandes voces ("¡Mi computadora no tiene CPU!"). Intel resolvió este problema pegando una imagen del chip de CPU (en realidad, un holograma) en la parte delantera de todos los SEC que se distribuyeron posteriormente.

El control de energía es un aspecto importante del Pentium II. La cantidad de calor que se desprende depende de la frecuencia de reloj, pero es del orden de 30-50 watts. Esta cantidad es enorme cuando se trata de un chip de computadora. Para tener una idea de cómo se sienten 50 watts, acerque su mano a una bombilla de 50 watts (sin tocarla) que ha estado encendida durante un rato. Por consiguiente, el SEC está configurado para aceptar un

radiador de calor, que es necesario para disipar el calor generado. Esto implica que cuando un Pentium II haya dejado de ser útil como CPU siempre podrá usarse como estufa para acampar.

Según las leyes de la física, cualquier cosa que emita gran cantidad de calor deberá consumir mucha energía. En una computadora portátil en la que la carga de la batería es limitada no es deseable consumir mucha energía. Para subsanar este problema, Intel ha hecho posible poner a la CPU a dormir cuando está ociosa y en una narcolepsia profunda cuando parece que va a estar ociosa durante mucho tiempo. Cuando la CPU está en el sueño profundo, los valores de la caché y los registros se conservan, pero el reloj y todas las unidades internas se desactivan. No se sabe si un Pentium II sueña cuando está profundamente dormido.

### La conexión lógica de terminales del Pentium II

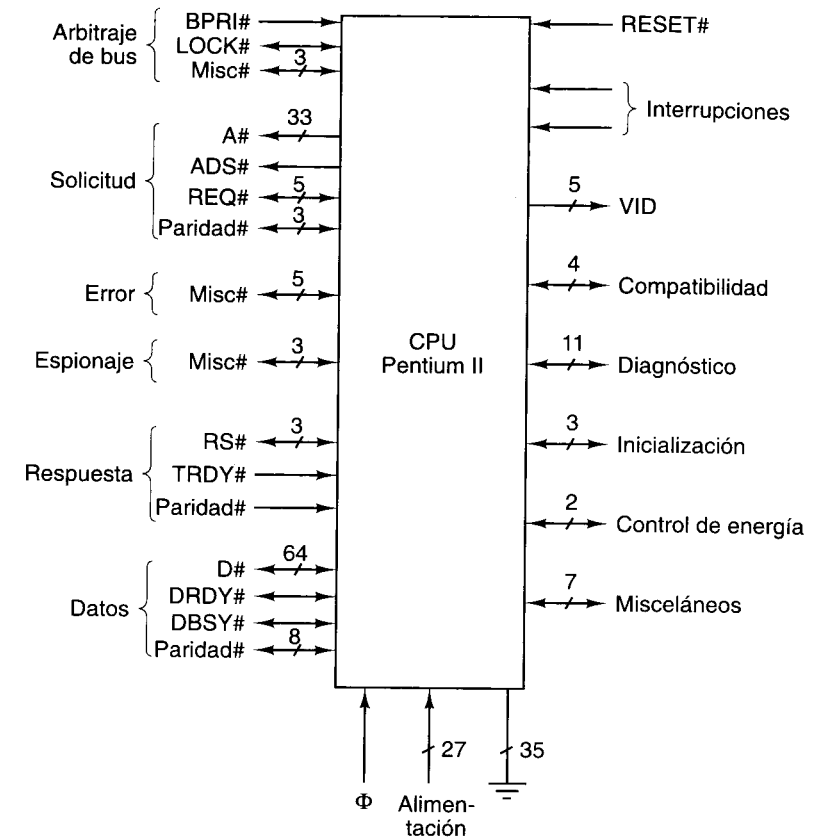
Los 242 conectores de arista del SEC se usan para 170 señales, 27 conexiones de potencia (a diferentes voltajes), 35 tierras y 10 repuestos para uso futuro. Algunas de las señales lógicas emplean dos o más terminales (como la dirección de memoria solicitada), así que sólo hay 53 señales distintas. En la figura 3-44 se muestra la configuración de conexión lógica de terminales (*pinout*) un tanto simplificada. En el lado izquierdo de la figura están los seis grupos principales de señales del bus de memoria; en el lado derecho están señales de diversos tipos. Los nombres que están totalmente en mayúsculas son los nombres reales dados por Intel a las señales. Los demás son nombres colectivos de varias señales relacionadas.

Intel emplea una convención para asignar nombres que es importante entender. Dado que todos los chips actuales se diseñan con computadoras, es necesario poder representar los nombres de las señales como texto ASCII. Es demasiado difícil usar testas para indicar las señales que se habilitan en nivel bajo, por lo que en vez de ello Intel coloca el símbolo # después del nombre. Así,  $\overline{\text{BPRI}}$  se expresa como BPRI#. Como puede verse en la figura, casi todas las señales de Pentium II se habilitan en nivel bajo.

Examinemos las señales, comenzando con las de bus. El primer grupo de señales sirve para solicitar el bus (es decir, efectuar arbitraje). BPRI# permite a un dispositivo hacer una solicitud de alta prioridad, que tiene precedencia respecto a una normal. LOCK# permite a una CPU bloquear el bus para evitar que la otra lo use antes de que la primera termine.

Una vez que una CPU u otro controlador de bus se ha apropiado del bus, puede presentar una solicitud utilizando el siguiente grupo de señales. Las direcciones son de 36 bits, pero los tres bits de orden bajo siempre deben ser 0 y por tanto no se les asignan líneas. Por tanto, A# sólo tiene 33 líneas. Todas las transferencias son de 8 bytes, alineadas a una frontera de 8 bytes. Con 36 bits de dirección, el máximo de memoria direccionable es  $2^{36}$ , que es 64 GB.

Cuando una dirección se coloca en el bus, la señal ADS# se habilita para indicar al objetivo (o sea, la memoria) que las líneas de dirección son válidas. El tipo de ciclo de bus (por ejemplo, leer una palabra o escribir un bloque) se coloca en las líneas REQ#. Dos de las señales de paridad protegen a A# y una protege a ADS# y REQ#. Las cinco líneas de error son utilizadas por el esclavo para informar errores de paridad y por todos los demás dispositivos para informar de otros errores.



**Figura 3-44.** Conexión lógica de terminales del Pentium II. Los nombres en mayúsculas son los nombres oficiales de Intel para señales individuales. Los nombres en mayúsculas y minúsculas son grupos de señales relacionadas o descripciones de señales.

El grupo de Espionaje se usa en sistemas multiprocesador para que una CPU pueda averiguar si una palabra que necesita está en la caché de la otra CPU. Describiremos la forma de espiar del Pentium II en el capítulo 8.

El grupo Respuesta contiene señales que el esclavo utiliza para enviar informes al amo (controlador). RS# contiene el código de situación. TRDY# indica que el esclavo (el objetivo) está listo para aceptar datos del amo. Estas señales también tienen verificación de paridad.

El último grupo del bus sirve para transferir realmente los datos. D# sirve para colocar 8 bytes de datos en el bus. Una vez que están ahí, se aserta DRDY# para anunciar su presencia. DBSY# sirve para decirle al mundo que el bus está ocupado.

RESET# sirve para restablecer la CPU en caso de ocurrir una calamidad. El Pentium II puede configurarse de modo que use las interrupciones igual que el 8088 (para fines de compatibilidad con lo existente) pero también puede usar un nuevo sistema de interrupciones con un dispositivo llamado APIC (**controlador de interrupciones programable avanzado**, *Advanced Programmable Interrupt Controller*).

El Pentium II puede operar a varios voltajes, pero necesita saber cuál. Las señales *VID* sirven para seleccionar automáticamente el voltaje de la fuente de potencia. Las señales de Compatibilidad sirven para engañar a dispositivos antiguos en el bus que piensan que están hablando con un 8088. El grupo de Diagnóstico contiene señales para probar y depurar sistemas según la norma de prueba IEEE 1149.1 JTAG. El grupo de Inicialización se ocupa de arrancar el sistema (*booting*). El grupo de Control de energía permite poner a dormir a la CPU normal y profundamente. Por último, el grupo Misceláneo es una revuelta de señales que incluye una que la CPU habilita si su temperatura interna llega a 130°C (266°F). Si una CPU alcanza alguna vez esta temperatura, probablemente está soñando en retirarse y convertirse en una estufa para acampar.

### Filas de procesamiento en el bus de memoria del Pentium II

Las CPU modernas como el Pentium II son mucho más rápidas que las memorias DRAM modernas. Para evitar que la CPU muera de inanición por falta de datos, es indispensable obtener un rendimiento máximo de la memoria. Por esta razón, el bus de memoria del Pentium II usa las filas de procesamiento, y se pueden estar ejecutando hasta ocho transacciones de bus al mismo tiempo. Ya vimos el concepto de filas de procesamiento en el capítulo 2 en el contexto de una CPU con filas de procesamiento (vea la figura 2-4), pero las memorias también pueden usarlos.

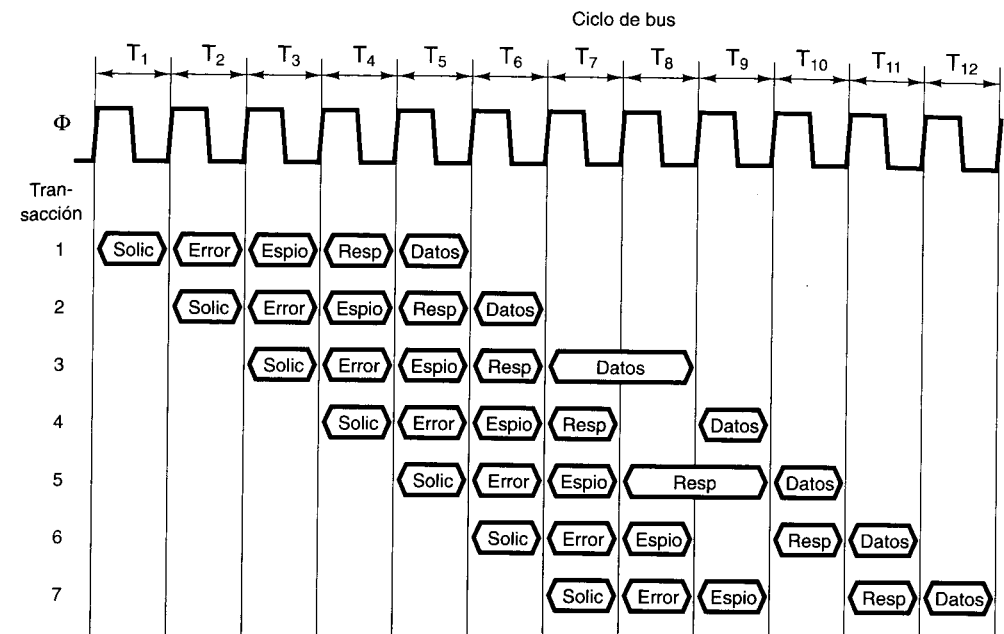
Para ello, las solicitudes de memoria del Pentium II, llamadas **transacciones**, tienen seis etapas:

1. La fase de arbitraje de bus.
2. La fase de solicitud.
3. La fase de informar errores.
4. La fase de espionaje.
5. La fase de respuesta
6. La fase de datos.

No todas las fases se necesitan en todas las transacciones. La fase de arbitraje de bus determina cuál de los posibles controladores de bus toma su turno. La fase de solicitud permite colocar la dirección en el bus y hacer la solicitud. La fase de informar errores permite al esclavo anunciar que la dirección tuvo un error de paridad o que algo más anda mal. La fase de espionaje permite a una CPU espiar a la otra, algo que sólo se necesita en un sistema multiprocesador. La fase de respuesta es aquella donde el controlador se entera de si está a punto de obtener los datos que quiere o no. Por último, la fase de datos permite devolver los datos.

El secreto del bus de memoria con conductos del Pentium II es que cada fase utiliza diferentes señales de bus, de modo que cada una es totalmente independiente de las demás. Los seis grupos de señales requeridas son los que se muestran en la figura 3-44 a la izquierda.

Por ejemplo, una CPU puede tratar de obtener el bus utilizando las señales de arbitraje. Una vez que adquirió el derecho a ser el siguiente, libera estas líneas del bus y comienza a usar las líneas del grupo de Solicitud. Mientras tanto, la otra CPU o algún dispositivo de E/S puede ingresar en la fase de arbitraje del bus, y así sucesivamente. La figura 3-45 muestra cómo varias transacciones de bus pueden estar pendientes al mismo tiempo.



**Figura 3-45.** Colocación de solicitudes en filas de procesamiento en el bus de memoria del Pentium II.

En la figura 3-45 no se muestra la fase de arbitraje de bus porque no siempre se necesita. Por ejemplo, si el dueño actual del bus (por lo regular la CPU) quiere ejecutar otra transacción, no tiene que readquirir el bus; sólo tiene que pedir el bus otra vez después de ceder la propiedad del bus a otro dispositivo solicitante. Las transacciones 1 y 2 son sencillas: cinco fases en cinco ciclos de bus. La transacción 3 introduce una fase de datos más larga, por ejemplo, porque es una transferencia de bloque o porque la memoria direccionada insertó un estado de espera. Por consiguiente, la transacción 4 no puede iniciar su fase de datos cuando quisiera hacerlo. La transacción 4 observa que la señal *DBSY#* todavía está habilitada y simplemente espera que la deshabiliten. En la transacción 5 vemos que la fase de respuesta también puede tardar varios ciclos de bus, lo que retarda la transacción 6. Por último, en la transacción 7 observamos que una vez que se ha introducido una burbuja en la fila de procesamiento permanece ahí si se siguen iniciando consecutivamente nuevas transacciones. Sin embargo, en la práctica es poco probable que la CPU intente iniciar una nueva transacción en cada ciclo de bus, así que las burbujas no duran tanto.

### 3.5.2 El UltraSPARC II

Como segundo ejemplo de chip de CPU, examinaremos la familia UltraSPARC de Sun. Esta familia es la línea de las CPU SPARC de 64 bits de Sun. La familia UltraSPARC se ajusta cabalmente a la arquitectura SPARC versión 9, que también es para las CPU de 64 bits, y se usa en estaciones de trabajo y servidores Sun, así como en varias aplicaciones más. Esta familia incluye el UltraSPARC I, el UltraSPARC II y el UltraSPARC III, cuya arquitectura es muy similar; difieren principalmente en la fecha de introducción y la rapidez del reloj. Para que la explicación que sigue sea más concreta, nos referiremos al UltraSPARC II, pero casi todo aplica también a los demás UltraSPARC.

El UltraSPARC II es una máquina RISC tradicional y es plenamente compatible en lo binario con la arquitectura SPARC V8 de 32 bits. Puede ejecutar programas binarios para SPARC V8 de 32 bits sin modificación porque la arquitectura SPARC V9 es compatible con la arquitectura SPARC V8. El único aspecto en que el UltraSPARC II se desvía de la arquitectura SPARC V9 es en la adición del conjunto de instrucciones para multimedia VIS, que se diseñó para aplicaciones gráficas, decodificación MPEG en tiempo real, etcétera.

El UltraSPARC II se diseñó para construir multiprocesadores de cuatro nodos con memoria compartida sin la adición de circuitos externos, y multiprocesadores más grandes con un mínimo de circuitos externos. En otras palabras, gran parte del “adhesivo” que se necesita para construir un multiprocesador ya está incluido en cada chip UltraSPARC II.

A diferencia del SEC del Pentium II, la CPU UltraSPARC II es un chip autónomo, aunque moderadamente grande, con 5.4 millones de transistores. Tiene 787 terminales en su base, dispuestas como se muestra en la figura 3-46. Esta gran cantidad de terminales se explica en parte por el uso de 64 bits para las direcciones y 128 bits para los datos, pero también se debe en parte a la forma como funcionan las cachés. Además, muchas de las terminales no se usan o son redundantes. Se escogió el número 787 para poder usar un encapsulado estándar en la industria. La industria probablemente considera que es de buena suerte tener un número primo de terminales.

El UltraSPARC II tiene dos cachés internas: 16 KB para instrucciones y 16 KB para datos. Al igual que el Pentium II, también usa una caché de nivel 2 externo al chip, pero a diferencia del Pentium II esa caché no viene empaquetada con el UltraSPARC II en un cartucho patentado. Los diseñadores de sistemas están en libertad de escoger cualquier chip de caché comercial que deseen para la caché de nivel 2.

La decisión de integrar la caché nivel 2 en el Pentium II y separarlo en el UltraSPARC II se debe en parte a las opciones técnicas y en parte a los diferentes modelos comerciales que Intel y Sun usan. En el aspecto técnico, una caché externa es más flexible (las cachés del UltraSPARC II pueden ir de 512 KB a 16 MB; las del Pentium II están fijas en 512 KB), pero podría ser más lento a causa de su mayor distancia de la CPU. Además, se requieren más señales visibles para direccionar la caché (los 242 conectores del cartucho SEC del Pentium II no incluyen señales para la caché, ya que la interacción CPU-caché es interna al cartucho), pero las 787 terminales del UltraSPARC II sí incluyen control de caché.

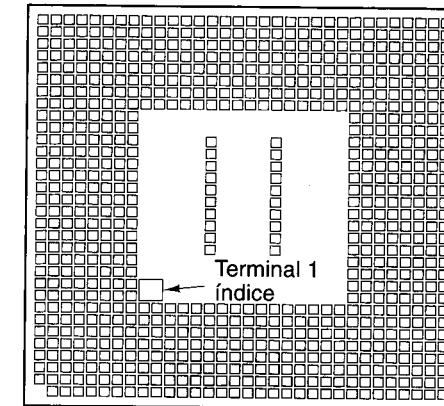


Figura 3-46. El chip de CPU UltraSPARC II.

En el aspecto comercial, Intel es un fabricante de semiconductores y puede diseñar y fabricar su propio chip de caché de nivel 2 y conectarlo a la CPU con una interfaz patentada de alto desempeño. Sun, en cambio, es un fabricante de computadoras, no de chips. Sí diseña algunos de sus chips (como los UltraSPARC), pero subcontrata fabricantes de semiconductores como Texas Instruments y Fujitsu. Cuando puede, Sun prefiere usar chips comerciales que han sido depurados por el mercado tan competitivo. Las SRAM que se usan en las cachés de nivel 2 se pueden conseguir de muchos proveedores de chips, por lo que Sun no sintió una necesidad especial de diseñar su propia SRAM. Esta decisión implica hacer la caché de nivel 2 independiente del chip de la CPU.

Casi todas las estaciones de trabajo Sun tienen un bus asincrónico de 25 MHz llamado **SBus**. Los dispositivos de E/S se pueden conectar al SBus, pero éste es demasiado lento para la memoria, así que Sun creó un mecanismo diferente para que una o más CPU UltraSPARC se comuniquen con una o más memorias: la **UPA (arquitectura de ultrapuerto, Ultra Port Architecture)**. La UPA se puede implementar como un bus, un conmutador, o una combinación de las dos cosas. Diferentes modelos de estaciones de trabajo y servidores usan diferentes implementaciones de la UPA. A la CPU no le importa qué implementación de la UPA se usa porque la interfaz con la UPA está perfectamente definida, y es esta interfaz la que el chip de la CPU debe reconocer (y reconoce).

En la figura 3-47 vemos el corazón de un sistema UltraSPARC II, que muestra el chip de CPU, la interfaz UPA y la caché de nivel 2 (dos SRAM comerciales). La figura también incluye un chip **UDB II (UltraSPARC Data Buffer II)**, cuya función se explicará más adelante. Cuando la CPU necesita una palabra de la memoria, primero la busca en una de sus cachés internas (nivel 1). Si la encuentra, continúa su ejecución a toda velocidad; si no encuentra la palabra en la caché de nivel 1, prueba en la de nivel 2.

Aunque analizaremos el uso de cachés con detalle en el capítulo 4, conviene hablar un poco al respecto aquí. Toda la memoria principal está dividida en líneas de caché (bloques) de